

COMPX204-22B Practical Networking and Cyber Security

Lecture 1 – Intro + Addressing

Matthew Luckie
mluckie@waikato.ac.nz

G.1.28A



1

Overview

This paper provides an overview of the technologies, protocols and security issues involved in computer communications.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

19 July 2022

2

2

Overview

This paper provides an overview of the technologies, protocols and security issues involved in computer communications.

It will introduce programming issues in communications, including error handling and concurrency.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

19 July 2022

3

3

Overview

This paper provides an overview of the technologies, protocols and security issues involved in computer communications.

It will introduce programming issues in communications, including error handling and concurrency.

Practical work will look at the Internet Protocols TCP and IP, and include programming exercises using socket interfaces.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

19 July 2022

4

4

Goals

1. Students will be able to describe aspects of the Internet including TCP/IP, DNS, addressing, routing.
2. Students will be able to write small communications systems in Java including parallel programming to implement servers that support multiple simultaneous clients.
3. Students will be able to use Transport Layer Security (TLS) to implement authentication and encryption in software.
4. Students will be able to write unit tests that check for correct software behaviour with untrusted network input.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

19 July 2022

5

5

Overview

• Pre-requisites

- COMPX102
- Programming course
- Examples are in Java

• Text: none assigned

• Points: 15

- This course represents 1/8th of a year's work
- About 150 hours total
- 2 lectures a week (50 minutes each)
- No scheduled tutorials
- Supervised lab sessions

• Paper resources are available on Moodle

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

19 July 2022

6

6

Contents

- Intro to Linux, Java
- Socket programming, concurrent programming (IO)
- HTTP and Web Servers
- Ethernet
- IPv4, IPv6
- TCP, UDP
- Routing (BGP + OSPF), Multicast, NAT
- Domain names and DNS
- Security
- Network Debugging

7

Assessment

- Internal / Exam ratio is 1:1
 - 4 assignments 30%
 - Ass1: Fri 12th Aug, 5%
 - Ass2: Fri 26th Aug, 8%
 - Ass3: Fri 30th Sep, 10%
 - Ass4: Fri 21st Oct, 7%
 - 1 test @ 10% 10%
 - 50 minutes, Tue 13th Sep
 - Weekly quizzes 10%
 - Final exam 50%

8

Assignments (30% of final grade)

1. Intro to Sockets in Java
2. Sockets and Threads; build a web server
3. Trivial File Transfer with Datagram Sockets
4. Transport Layer Security

9

Specified Labs

- Supervised in-person lab sessions, and remote lab sessions supported with zoom.
- **Masks required to be correctly worn for in-person labs.**
- This is when you can ask for help with lab work
- Hamilton:
 - Monday 9-11am R-block Lab 1
 - Thursday 4-5pm R-block Lab 1
 - Friday 1-3pm R-block Lab 1
- Tauranga:
 - Friday 10-12noon TCBD.3.04

10

Assignments (30% of final grade)

- Two options to have assignments marked:
 - In-person verification during a lab session.
 - Offline marking after the submission due date.
- Both options require you to submit your code using Moodle

11

Tentative Schedule

July	18 – 22	Intro + Adrrs	Sockets	
	25 – 29	Threads	Sockets + Threads	
Aug	1 – 5	Ethernet #1	Ethernet #2	
	8 – 12	DNS / UDP	Unit Testing	A1: 12 th Intro
	15 – 19	TCP #1	TCP #2	
	22 – 26	TCP #3	Revision	A2: 26 th HTTP
Sep	12 – 16	Test #1	Kingitanga	
	19 – 23	Security #1	Security #2	
	26 – 30	Security #3	IP Routing	A3: 30 th TFTP
Oct	3 – 7	OSPF / Mcast	BGP	
	10 – 14	NAT	Autoconf	
	17 – 21	Debugging	Course Wrap	A4: 21 st TLS

12

Class representatives

- Approximately 150 students enrolled in the course
- 2-3 class representatives would be nice
 - Liaise between students in the paper and teaching staff
 - Mediate on relevant issues
 - Pass on academic feedback

13

Learning Objectives

1. Understand what an IP address, a DNS name, and a port number is, and how they are used on the Internet
2. Understand the key differences between IPv4 and IPv6 addresses, and why the two addresses are incompatible

14

Communications

- There are many ways for systems to communicate
 - Infra-red beaming, e.g. TV + stereo remotes
 - Serial cables
 - USB
 - Bluetooth
 - Fibre Optic cables
 - Various wireless systems (WiFi)
- Ethernet-based methods are the most common physical network type

15

Communications

- Systems on a network need to be identified somehow
 - To be able to accurately direct messages to them
 - Think of it like identifying houses in the postal system
- We identify networked systems by their *address*
- There are a number of *addressing schemes*
 - Internet Protocol version 4 (IPv4) addressing
 - Internet Protocol version 6 (IPv6) addressing
 - Ethernet MAC addressing
 - Bluetooth MAC addressing
 - ... and many, many more.
- More than one address may be used in a single communication

16

IPv4

- The most used Internet addressing scheme is IPv4
- Most systems on the Internet have at least one IPv4 address; some have more than one
- An IPv4 address is a 32 bit number, written as four one-byte values
 - e.g. 130.217.250.39
 - This presentation format is known as a dotted-quad.
- Question: what is the range of numbers between each dot? Why?

17

IPv4

- Question: How many addresses are there in the IPv4 *address space*?
- Answer: $2^{32} = 4,294,967,296$

18

IPv4

- Question: why not just use a 32 bit number instead of a dotted-quad?
- Answer:
 - More difficult to remember
 - E.g. 130.217.250.39 => 2,195,323,431
 - Often easier to remember four small numbers than a large one.

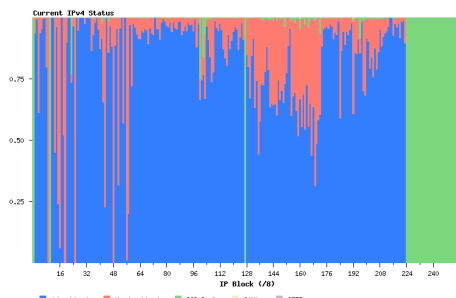
19

IPv4

- At the time IPv4 was specified (early 1980s), it was not conceived that the Internet would need more than 4 billion addresses
- Due to inefficiencies in the allocation system, as well as the massive growth over the last 25 years, networks cannot obtain new IPv4 addresses.

20

IPv4 Address Status: July 2022



source: <https://ipv4.potaroo.net>

21

IPv6

- The fundamental difference between IPv4 and IPv6 is that IPv6 has 128-bit addresses
- 340 282 366 920 938 463 463 374 607 431 768 211 456 addresses
- This will allow for a huge expansion
 - Mobile devices
 - "Internet of Things" – everything could have an IP address
- Unfortunately, IPv4 hosts cannot communicate with IPv6 hosts directly, and vice versa
 - Cannot fit a 128 IPv6 address in the space reserved for a 32-bit IPv4 address.

22

IPv6 presentation format

- IPv6 addresses are represented in 8 groups of 16 bits each, each group reported in hexadecimal form. E.g:

• 2001:df0:4:4000:9a90:96ff:fedc:8110

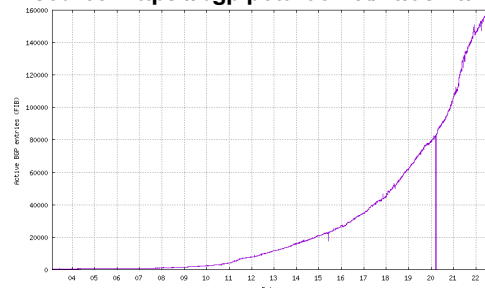
Leading zeros in each group do not need to be printed (i.e. the full hexadecimal printing is 0x0004)

23

IPv6 deployment

- IPv6 protocol was formalized in 1998. Deployment languished for a long time, but is accelerating

source: <https://bgp.potaroo.net/v6/as2.0/>



24

IPv4 vs IPv6

- ISPs can obtain IPv4 from the reseller market:
 - From a reseller market. US\$30 per address (min 256 addresses, i.e., \$7,680 or roughly \$10K NZD)
- ISPs can obtain IPv6 from a Regional Internet Registry:
 - Typically 2^{96} addresses.
 - 79 228 162 514 264 337 593 543 950 336
 - For most, obtain a single allocation, will probably never need more

25

Friendly names

- It is not practical to remember the addresses of each of the computers on the Internet we want to communicate with
 - Especially IPv6!
- Many addresses also have a name associated
 - www.facebook.com : 31.13.78.35
: 2a03:2880:f119:8083:face:b00c:0:25de
 - elearn.waikato.ac.nz : 130.217.226.16
 - www.google.com : 142.250.67.4
: 2404:6800:4006:804::2004

26

Friendly names

- The system for *resolving* (looking up) friendly names to give an IP address is known as the
 - **Domain Name System (DNS)**
- Can make manual DNS queries using the `host` command


```
lab-rg07-17:[100]~>host www.facebook.com
www.facebook.com is an alias for star-mini.c10r.facebook.com.
star-mini.c10r.facebook.com has address 31.13.78.35
star-mini.c10r.facebook.com has IPv6 address
2a03:2880:f119:8083:face:b00c:0:25de
```
- Can make programmatic queries using Sockets

27

Communicating

- We don't communicate with a networked system as such
- Rather, we communicate with a piece of software running on the networked system
 - E.g., to browse to www.google.com, we want to talk to google's web server software
- This is achieved by using a **port number** to identify the application

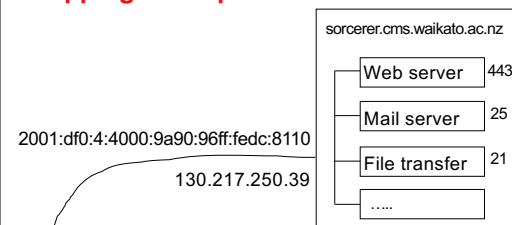
28

Port Numbers

- A port number is a 16-bit number
- Some applications have well known port numbers
 - 443 is used for web (HTTPS)
 - 80 for unencrypted web (HTTP)
 - 53 is used for DNS
 - 21 is used for file transfer protocol (FTP)
 - 110 is used for POP email
 - 25 is used to send mail
- `/etc/services` on Linux and MacOS X systems

29

Wrapping it all up



30

Wrapping it all up

- DNS names map to IP addresses
- IP addresses identify systems on the Internet
- Applications run on a port

© THE UNIVERSITY OF MANCHESTER • 10. MANCHESTER KNOWLEDGE CHAIRS LTD.

19 July 2022

31

31

Learning Objectives

1. Understand what an IP address, a hostname, and a port number is, and how they are used on the Internet
2. Understand the key differences between an IPv4 and an IPv6 address, and why the two addresses are incompatible

© THE UNIVERSITY OF MANCHESTER • 10. MANCHESTER KNOWLEDGE CHAIRS LTD.

19 July 2022

32

32

Summary

- Friendly names
- IP addresses
- Application ports

- Next lecture: using this knowledge to build a simple network application using Java
- First assignment: writing a few small programs with Java sockets

© THE UNIVERSITY OF MANCHESTER • 10. MANCHESTER KNOWLEDGE CHAIRS LTD.

19 July 2022

33

33

COMPX204-22B Practical Networking and Cyber Security

Lecture 2 – Java Sockets

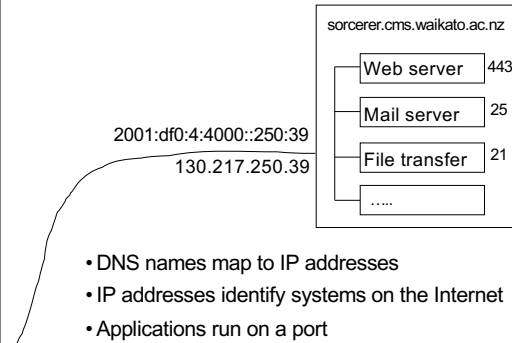
Matthew Luckie
mluckie@waikato.ac.nz



20 July 2022

1

From First Lecture



2

Learning Objectives

1. Be able to write Sockets code that can “resolve” a name to an address, and an address to a name
2. Be able to write Sockets code that can behave like a “client” – connect to a Server, read and write lines of text to that system
3. Be able to write Sockets code that can behave like a “server” – allow clients to connect to the application, and read and write lines of text to that system

Note: the slides beyond here serve as reference, and I won't be using them.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 July 2022

3

3

Java Language

- Object oriented programming language (like C++, C#)
- OO programming is about organizing programs so that they are easier to maintain and debug
- Everything is an “Object”
 - Including the program
 - You ask the object to do something via a “method” call
 - The actual implementation is hidden / opaque to other developers
- Convention: the source code that contains a class Foo is named Foo.java

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 July 2022

4

4

Java Virtual Machine (JVM)

- When you compile C or C++, the compiler produces machine code specific to the target architecture
 - E.g., arm, MIPS, i386, amd64, ...
- When you compile Java, the compiler produces “byte code”
 - When you compile Foo.java with javac, javac produces Foo.class
 - Foo.class is “byte code” that the JVM interprets at run time, producing CPU instructions for the target architecture

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 July 2022

5

5

Using a shell / emacs / Linux

- While integrated development environments (IDEs) are widely available, I emphasize using a shell, a text editor, and Linux
 - Nearly all modern computer science involves Linux / Unix
 - IDEs act as a crutch: what happens if you find yourself asked to write a program in an unfamiliar environment that doesn't have your preferred development environment?
- Text editors, such as emacs, are much more widely deployed.
 - Windows: <https://www.gnu.org/software/emacs/download.html>
 - MacOS: <https://emacsformacosx.com/>

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 July 2022

6

6

Remotely logging in

- SSH: Secure Shell (communication between computers is authenticated and encrypted)
- `ssh <username>@lab-rg06-22.cms.waikato.ac.nz`
 - lab-rg06-01 through lab-rg06-41
 - lab-rg07-01 through lab-rg07-42
- Note: to login to a lab machine, you first have to login via a "bounce" host first. i.e.:
 - `ssh <username>@linux-labs.cms.waikato.ac.nz` then
 - `ssh <username>@lab-rg07-13.cms.waikato.ac.nz`

7

Shells (watch recording of lecture)

- A shell provides low-level access to the operating system.
- Common commands:
 - `ls` "list files in this directory"
 - `pwd` "print working directory"
 - `mkdir` "make directory"
 - `mkdir 201`
 - `cd` "change directory"
 - `cd ..` "change to parent directory"
 - `cd 201` "change to 201 directory"

8

Shells (watch recording of lecture)

- Process control:
 - `ctrl+z` "put process in background" (not recommended)
 - `fg` "foreground process"
 - `ctrl+c` "send process signal to exit"
- Productivity help:
 - recent commands scrollable with up and down arrows
 - `ctrl+r` "find recent command with string in it"
 - `ctrl+r foo` "find recent command with foo in it"
 - Tab completion: shell can guess what you are writing and complete a string for you. Press the tab key.

9

Shells

- It is a bad idea to resize a terminal unless your terminal is at a shell prompt
- Terminal will get confused about how big the screen is if you resize while shell is running a program, and then behave weirdly

10

Emacs (watch recording of lecture)

- Powerful, easy to use text editor
- Uses colour to help with readability
- Will automatically indent code to help with readability
- `emacs -nw -r foo.java`
 - Start emacs without a windowed interface, reverse colours
- `ctrl+x ctrl+s` "save file"
- `ctrl+x ctrl+c` "exit emacs"
- `ctrl+k` "cut everything to right into clipboard"
- `ctrl+y` "paste from clipboard"

11

HelloWorld.java

```
class HelloWorld
{
    public static void main(String [] args)
    {
        System.out.println("Hello world");
    }
}
```

Compile with:
`javac HelloWorld.java`

Run with:
`java HelloWorld`

12

HelloWorld.java

```
class HelloWorld
{
    public static void main(String [] args)
    {
        /* print a line "Hello world" */
        System.out.println("Hello world");

        /* print command line arguments, prefaced with argument number */
        for(int i=0; i<args.length; i++)
            System.out.println(i + " " + args[i]);
    }
}
```

13

Keywords

- **public**: anyone can access this variable / method
- **private**: only this class can access this variable / method
- **protected**: only this and subclasses can access this variable / method
- **void**: no return value
- **static**: only this variable / method for all instances of this whole class
 - **dynamic**: new variables for each class instance

14

Dealing with IP addresses in Java

- **java.net.InetAddress** class
- Useful methods:
 - **getByName** : resolve address with DNS name
 - **getAllByName** : resolve all addresses with DNS name
 - **getHostAddress** : return String containing IP address

Note that this lecture will not discuss **getAllByName**, but you'll use it in assignment one to obtain IPv6 addresses, where available.

15

An example

```
String name = "sorcerer.cms.waikato.ac.nz";
try {
    InetAddress IP =
        InetAddress.getByName(name);
    System.out.println(name + " : " +
        IP.getHostAddress());
}
catch(UnknownHostException e) {
    System.err.println(name + " : unknown
    host");
}
```

16

Output

sorcerer.cms.waikato.ac.nz : 130.217.250.39

17

Doing the reverse

- It is also possible to obtain a DNS name of an IP address (if one exists) using **InetAddress::getHostName()**
- Sometimes if an IP address has a name, this method won't return it
 - if the name lookup takes too long to complete
- If the IP address does not have a name, this method returns the IP address looked up in a String

18

A second example

```
String address = "130.217.250.39";
try {
    InetAddress IP =
        InetAddress.getByName(address);
    System.out.println(address + " : " +
        IP.getHostName());
}
catch(UnknownHostException e) {
    System.err.println(address + " : unknown
    host");
}
```

19

Output

130.217.250.39 : sorcerer.cms.waikato.ac.nz

20

Networked hello world

- The rest of this lecture is concerned with what is known as a Socket
- A Socket is used to establish connections between networked machines on the Internet.
- java.net.Socket

21

Establishing a connection

- This code connects to the HTTPS port (443) on www.wand.net.nz

```
InetAddress wandIP =
    InetAddress.getByName("www.wand.net.nz");
Socket mysocket = new Socket(wandIP, 443);
```

22

Now what?

- We've established a connection, but now we need to be able to send and receive messages on it.
- There are multiple ways to do this in Java.

23

Input and Output streams

- At the lowest level, messages are made up in units of bytes
- You can obtain bytes by obtaining the corresponding input and output streams, and then using the read and write methods available

```
InputStream in = mysocket.getInputStream();
OutputStream out = mysocket.getOutputStream();
```

- But you probably don't want to do this yet.

24

Input and Output lines of text

- To start with, we want to deal with reading and writing text lines from Sockets
 - `java.io.BufferedReader`
 - `java.io.PrintWriter`
- `BufferedReader` returns whole lines of text per call
- `PrintWriter` assembles lines of text and writes to the Socket when a whole line of text is available

25

Writing lines of text

- `PrintWriter` is fairly easy to use

```
PrintWriter out = new
    PrintWriter(mysocket.getOutputStream(), true);
out.println("hello world");
out.close();
```

- The constructor takes an optional boolean parameter that controls whether it flushes on a line basis; you want to use `true` as the parameter.

26

Reading lines of text

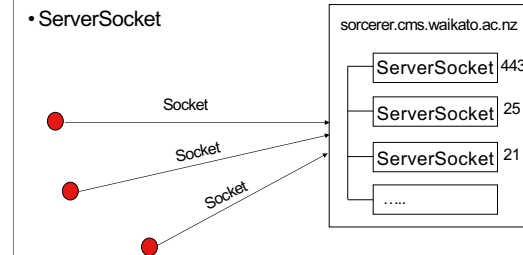
- `BufferedReader` is a bit more complicated
 - The `BufferedReader` constructor takes a `Reader` object, but `Socket.getInputStream()` returns an `InputStream`.
 - The `InputStreamReader` class extends the `Reader` class, and it takes an `InputStream` as a parameter in its constructor, so we'll use that

```
InputStreamReader isr = new
    InputStreamReader(mysocket.getInputStream());
BufferedReader reader = new
    BufferedReader(isr);
```

27

ServerSocket

- There is a special type of socket for accepting new connections to the system in Java.
- `ServerSocket`



28

ServerSocket

```
ServerSocket server = new ServerSocket(51234);
Socket client = server.accept();
```

29

Wrapping it all up: HelloWorldServer.java

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
ServerSocket ss = new ServerSocket(51234);
while(true) {
    Socket client = ss.accept();
    PrintWriter writer = new
        PrintWriter(client.getOutputStream(), true);
    BufferedReader reader = new BufferedReader(
        new InputStreamReader(client.getInputStream()));
    String line = reader.readLine();
    writer.println("You said: " + line);
    client.close();
}
```

30

Wrapping it all up: HelloWorldClient.java

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
InetAddress IP = InetAddress.getByName(args[0]);
Socket me = new Socket(IP, 51234);
PrintWriter writer = new
    PrintWriter(me.getOutputStream(), true);
BufferedReader reader = new BufferedReader(
    new InputStreamReader(me.getInputStream()));
writer.println("Hello, World");
String response = reader.readLine();
me.close();
System.out.println("The other system said:\n" +
    response);
```

© THE UNIVERSITY OF Waikato - 15 Waikato Waikato University

20 July 2020

31

31

In summary

- The InetAddress class deals with resolving names to addresses
- The Socket class represents an individual connection established over the network
- The ServerSocket class is a special type of socket used to allow new connections to be made
- The PrintWriter class allows us to write individual lines of text over a socket
- The BufferedReader class allows us to read lines of text from a socket

© THE UNIVERSITY OF Waikato - 15 Waikato Waikato University

20 July 2020

32

32

Assignment 1 Notes

- Sanity check the command line parameters provided on the command line.

```
$ java SimpleClient lab-rg06-01.cms.waikato.ac.nz 51234
```

- Don't use args[0] or args[1] unless you know they are there.
- Print a usage statement if they are not provided
- You can find out how many elements are valid with the .length property on an array, i.e; args.length

© THE UNIVERSITY OF Waikato - 15 Waikato Waikato University

20 July 2020

33

33

Learning Objectives

1. Be able to write Sockets code that can "resolve" a name to an address, and an address to a name
2. Be able to write Sockets code that can behave like a "client" -- connect to a Server, read and write lines of text to that system
3. Be able to write Sockets code that can behave like a "server" -- allow clients to connect to the application, and read and write lines of text to that system

© THE UNIVERSITY OF Waikato - 15 Waikato Waikato University

20 July 2020

34

34

COMPX204-22B Practical Networking and Cyber Security

Lecture 3 – Java Threads

Matthew Luckie
mluckie@waikato.ac.nz



26 July 2022

1

So far

- The **InetAddress** class deals with resolving names to addresses
- The **Socket** class represents an individual connection established over the network
- The **ServerSocket** class is a special type of socket used to allow new connections to be made
- The **PrintWriter** class allows us to write individual lines of text over a socket
- The **BufferedReader** class allows us to read lines of text from a socket

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

26 July 2022

2

2

Learning Objectives

1. Understand what is meant by “blocked” in a communications software
2. Understand how threads can allow a single process to have multiple points of simultaneous execution
3. Understand the need for synchronized access to variables shared amongst threads

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

26 July 2022

3

3

HelloWorldServer.java

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
ServerSocket ss = new ServerSocket(51234);  
while(true) {  
    Socket client = ss.accept();  
    PrintWriter writer = new  
        PrintWriter(client.getOutputStream(), true);  
    BufferedReader reader = new BufferedReader(  
        new InputStreamReader(client.getInputStream()));  
    String line = reader.readLine();  
    writer.println("You said: " + line);  
    client.close();  
}
```

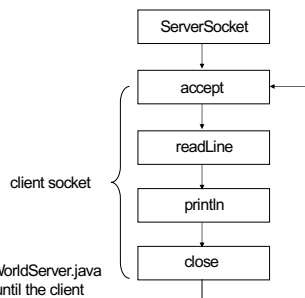
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

26 July 2022

4

4

HelloWorldServer.java



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

26 July 2022

5

5

HelloWorldServer.java

- Major problem: two clients cannot use the server concurrently.
- The problem is that the program *blocks* until the first client to connect sends a line of text through, causing readLine to return

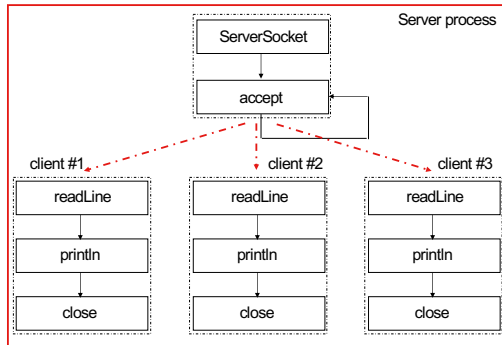
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

26 July 2022

6

6

What we want



7

Solutions

- There are many solutions to this problem
 - “Fork” a new process to deal with the new connection
 - “Synchronous I/O multiplexing” – handle events on a set of sockets as they occur. This is commonly done with “select” or “poll” system calls.
 - “Threads” – a single process with multiple flows of control
- We’ll look at Threads in this course.

8

Threads

- Just as our computer can run multiple programs at the same time, each of which share the processors, we can have two strands of our program run at the same time
- Each strand is a *Thread*
- Jargon
 - Operating system runs processes
 - Programs have threads
 - Threads can be thought of as light-weight processes
 - Threads can share data structures amongst themselves

9

HelloWorldServer2.java

- Goal is to write HelloWorldServer so multiple clients can connect and interact
- HelloWorldServer2:
 - main method will accept new clients as before
 - instead of then reading text from the client, it will then spawn a thread to read the line of text from the client and write it back

10

HelloWorldServer2.java: a skeleton

```

class HelloWorldServerThread extends Thread
{
    public void run()
    {
        /* thread's main procedure – entry point */
    }
    public HelloWorldServerThread(/* parameters */)
    {
        /* initialise thread's parameters */
    }
}

```

11

HelloWorldServer2.java: a skeleton

- Constructor takes parameters to create a thread object. You can think of the parameters like command line arguments to main()
- Thread does not begin execution until the start() method is called

12

HelloWorldServer2.java

```
class HelloWorldServer2 {
    public static void main(String args[])
    {
        try {
            ServerSocket ss = new ServerSocket(51234);
            while(true) {
                Socket client = ss.accept();
                HelloWorldServerThread thread =
                    new HelloWorldServerThread(client);
                thread.start();
            }
        } catch(Exception e) {
            System.err.println("Exception: " + e);
        }
    }
}
```

13

HelloWorldServer2.java

```
class HelloWorldServerThread extends Thread {
    public void run() {
        try {
            PrintWriter writer =
                new PrintWriter(client.getOutputStream(), true);
            BufferedReader reader =
                new BufferedReader(
                    new InputStreamReader(client.getInputStream()));
            String line = reader.readLine();
            writer.println("You said: " + line);
            client.close();
        } catch(Exception e) {
            System.err.println("Exception: " + e);
        }
    }
}
```

14

HelloWorldServer2.java

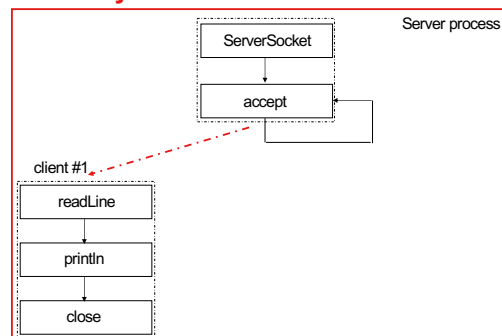
```
class HelloWorldServerThread extends Thread
{
    Socket client;
    public HelloWorldServerThread(Socket s)
    {
        client = s;
    }
}
```

15

In class demo

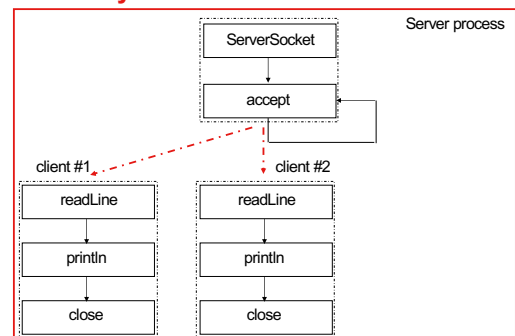
16

Summary so far

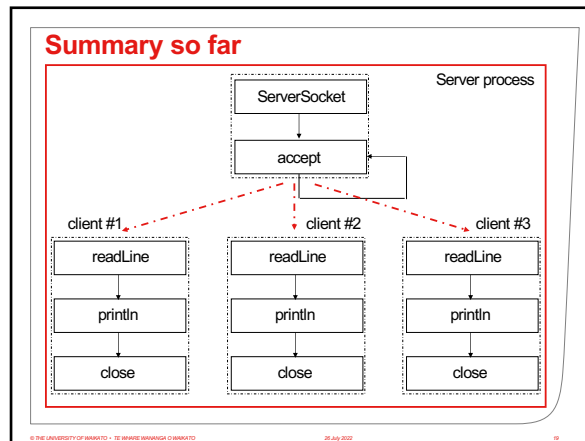


17

Summary so far



18



19

Summary so far

- HelloWorldServer2's main routine accepts new clients and creates a thread to handle that client
- HelloWorldServerThread's constructor takes the client variable to handle
- HelloWorldServer2's main routine calls the start method
- HelloWorldServerThread's run routine is then started.

© THE UNIVERSITY OF BRISTOL - 10.1017/9781315311111.001 25 July 2022 20

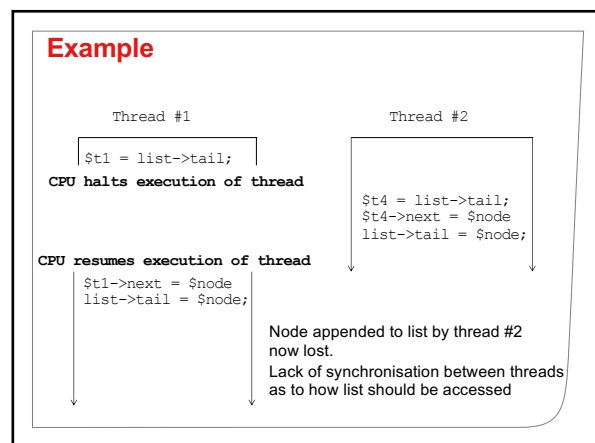
20

Thread problems

- Variables shared between more than one thread must be used carefully
- Real communications systems will deal with multiple connections simultaneously

© THE UNIVERSITY OF BRISTOL - 10.1017/9781315311111.001 25 July 2022 21

21



22

Java Threads

- Need to be careful when accessing a variable shared amongst threads to access the variable one at a time
 - i.e. lock the variable when using it to ensure its integrity
- Easy way to do this is to use the **synchronized** keyword

© THE UNIVERSITY OF BRISTOL - 10.1017/9781315311111.001 25 July 2022 23

23

Java Threads

```

public void sayToAll(ChatServerSession from, String message)
{
    int i, len;
    synchronized(sessions) {
        len = sessions.size();
        for(i=0; i<len; i++) {
            ChatServerSession to = sessions.get(i);
            if(from != to) {
                to.send(message);
            }
        }
    }
}

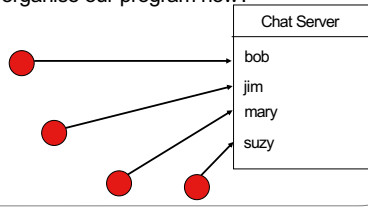
```

© THE UNIVERSITY OF BRISTOL - 10.1017/9781315311111.001 25 July 2022 24

24

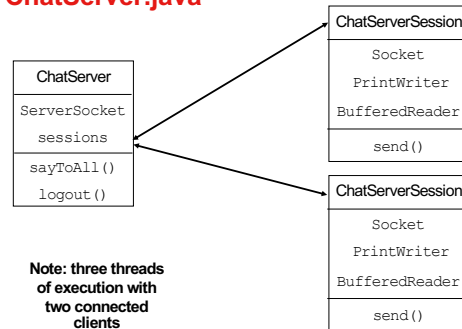
ChatServer.java

- A chat server takes lines of text from each client and relays the text to all other clients
- Required variable: List of connected clients
- How do we organise our program now?



25

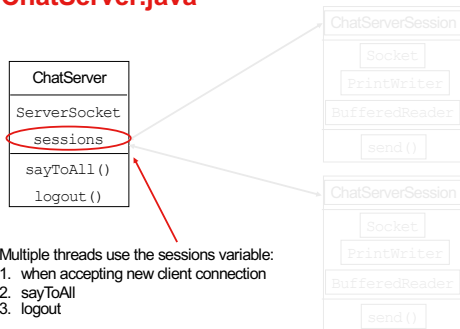
ChatServer.java



**Note: three threads
of execution with
two connected
clients**

26

ChatServer.java



Multiple threads use the sessions variable:

1. when accepting new client connection
2. sayToAll
3. logout

27

Learning Objectives

1. Understand what is meant by "blocked" in a communications software
2. Understand how threads can allow a single process to have multiple points of simultaneous execution
3. Understand the need for synchronized access to variables shared amongst threads

28

Summary

- Methods that block are a problem for communications systems as they prevent other clients getting service
- Threads are one way to avoid blocking on input
- Homework task:
 - Add the ability to ChatServer.java for clients to specify their name and prefix it to their messages
 - Make sure the name is unique!

29

COMPX204-22B Practical Networking and Cyber Security

Lecture 4 – Java Threads

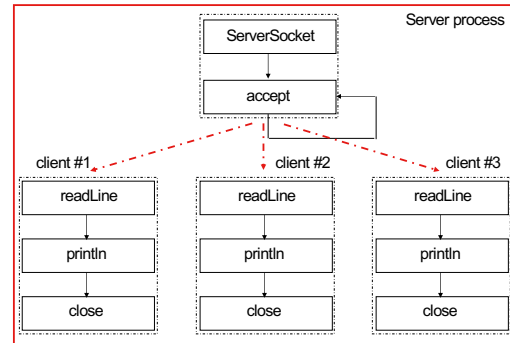
Matthew Luckie
mluckie@waikato.ac.nz



28 July 2022

1

Summary so far



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

28 July 2022

2

2

Learning Objectives

1. Understand how threads can allow a single process to have multiple points of simultaneous execution
2. Understand the need for synchronized access to variables shared amongst threads

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

28 July 2022

3

3

Thread problems

- Variables shared between more than one thread must be used carefully
- Real communications systems will deal with multiple connections simultaneously

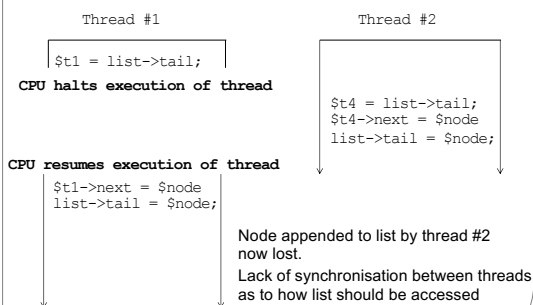
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

28 July 2022

4

4

Example: concatenation to a linked list



5

Example: incrementing a shared int (1/2)

```

class incr {
    int x;
}
class incr_thread extends Thread
{
    private incr i;
    public incr_thread(incr i) {
        this.i = i;
    }
    public void run() {
        for(int j = 0; j < 1000000; j++) {
            i.x++;
        }
    }
}

```

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

28 July 2022

6

6

Example: incrementing a shared int (2/2)

```
class increment {
    public static void main(String args[])
    {
        incr i = new incr();
        incr_thread a = new incr_thread(i);
        incr_thread b = new incr_thread(i);
        a.start(); b.start();
        try {
            Thread.sleep(1000);
        } catch (Exception e) {
            System.err.println("Exception: " + e);
        }
        System.out.println(i.x);
    }
}
```

7

Java Threads: need to use synchronized

- Need to be careful when accessing a variable shared amongst threads to access the variable one at a time
 - i.e, lock the variable when using it to ensure its integrity
- Easy way to do this is to use the **synchronized** keyword

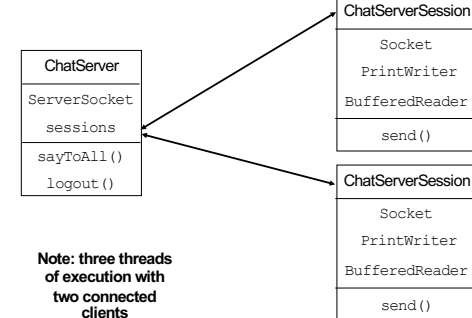
8

Example: synchronized

```
class incr_thread extends Thread    class incr {
{                                     int x;
    private incr i;
    public incr_thread(incr i) {
        this.i = i;
    }
    public void run() {
        for(int j = 0; j < 1000000; j++) {
            synchronized(i) {
                i.x++;
            }
        }
    }
}
```

9

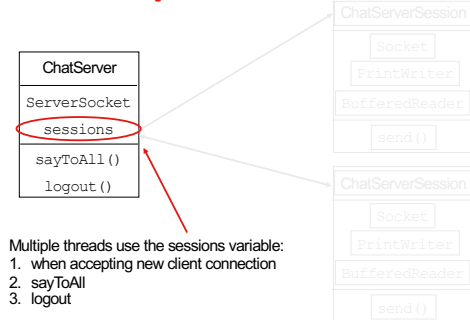
ChatServer.java



Note: three threads of execution with two connected clients

10

ChatServer.java



Multiple threads use the sessions variable:

1. when accepting new client connection
2. sayToAll
3. logout

11

Learning Objectives

1. Understand how threads can allow a single process to have multiple points of simultaneous execution
2. Understand the need for synchronized access to variables shared amongst threads

12

Summary

- Methods that block are a problem for communications systems as they prevent other clients getting service
- Threads are one way to avoid blocking on input

COMPX204-22B Practical Networking and Cyber Security

Lecture 5 – Ethernet #1

Matthew Luckie
mluckie@waikato.ac.nz



© The University of Waikato - Te Whare Wānanga o Waikato 2 August 2022

1

Learning Objectives

1. Appreciate the utility of “Protocol Stacks” – how each additional layer can add useful functionality to a networked system
2. Be familiar with Ethernet technologies
 - full-duplex, half duplex, and simplex;
 - Wired and Wireless Ethernet;
 - twisted pair; coax; fiber optics
3. Understand how data can be *serialised* onto a physical medium as 1s and 0s
 - how to avoid timing issues from independent running clocks
 - how to detect and correct errors in transmission

© The University of Waikato - Te Whare Wānanga o Waikato

2 August 2022

2

2

Network communications

- Complex environment
- **Different network elements** are concerned with **different functions**
- To simplify the development of networks, we develop a network in layers

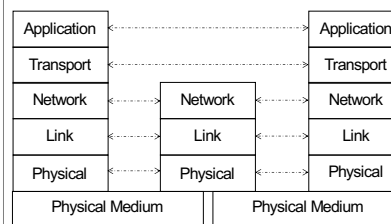
© The University of Waikato - Te Whare Wānanga o Waikato

2 August 2022

3

3

Protocol Layers (the Internet view)



A layered hierarchy: each layer implements a protocol to talk to the same layer at other nodes

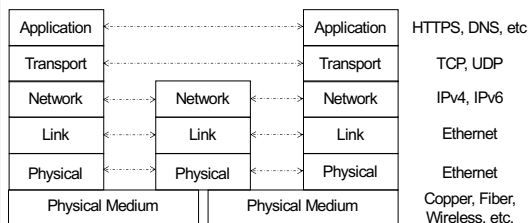
© The University of Waikato - Te Whare Wānanga o Waikato

2 August 2022

4

4

Protocol Layers (the Internet view)



A layered hierarchy: each layer implements a protocol to talk to the same layer at other nodes

© The University of Waikato - Te Whare Wānanga o Waikato

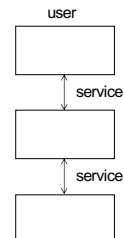
2 August 2022

5

5

A Layered Hierarchy

- Each layer provides a service to the higher layer through a standard interface
- Each layer makes “calls” for services from the layer below
- Important: layering provides **flexibility** to modify and evolve protocols and services
- Important: enables technology to **change** at one layer without affecting the other layers
 - transitioning from IPv4 to IPv6
 - transitioning from copper to fibre to radio waves



© The University of Waikato - Te Whare Wānanga o Waikato

2 August 2022

6

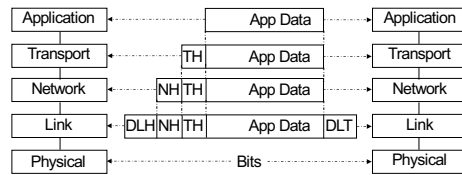
6

Protocols

- Protocols are the rules and procedures that govern the way computers communicate
 - Ensure that devices process communications correctly
- Protocols specify message **formats** and **actions**
- Protocols are piled up to form a **stack**
- Protocols are normally acronyms that end in "P"
 - IP
 - HTTP
 - ...
 - TCP, UDP, ICMP, BGP, ...

7

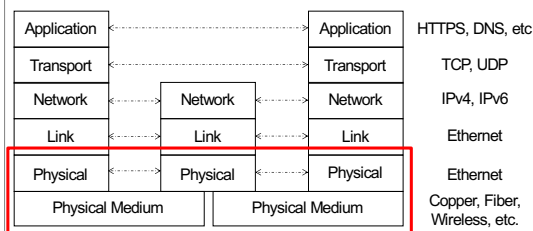
Encapsulation



- Application data is encapsulated inside of **packets** with **headers**
- Kind of like how a sub-class extends features of another class by adding something to the class

8

Protocol Layers (the Internet view)



9

Ethernet Local Area Networks (LANs)

- Goal of a network is to connect computers together
 - Could have many point to point links, but inefficient to do this in terms of resources required

e.g.:
5 computers,
4 links each,
10 links total



- Local Area Network
 - Computers share a set of cables and take turns to exchange information in some organised way

10

Ethernet History

- Xerox 1970s: Metcalfe
- DEC, Intel, Xerox (DIX standard)
- IEEE standard 802.3
 - **Speed (Mbps) + baseband + cabling standard**
 - Thick wire Ethernet (10base5)
 - Thin wire Ethernet (10base2)
 - Twisted pair Ethernet (10baseT)
 - Optical fibre Ethernet (10baseFL)
 - Fast Ethernet (100base-...)
 - Gigabit Ethernet (1000base-...)
 - 10 Gigabit Ethernet (10Gbase-...)
 - 40/100 Gigabit Ethernet (100Gbase-...) Fibre only

11

Ethernet

- Designed in the era of a shared coaxial cable being the medium
- Goal was to efficiently share the bandwidth available amongst connected nodes
- CSMA/CD
 - **Carrier Sense**: if a signal from another station is being broadcast, don't speak over the top
 - **Multiple Access**: multiple nodes want to share the medium
 - **Collision Detect**: if another station begins transmitting at the same time, back off a random amount of time and start again
 - Removed in 10 Gigabit Ethernet

12

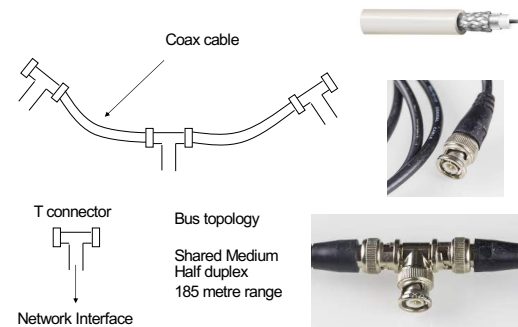
Transmitting and Receiving

- **Half duplex**
 - Station can transmit and receive on the same medium, but can't do both at the same time
- **Full duplex**
 - Station can transmit and receive on the same medium at the same time
- **Simplex**
 - Station can only transmit
 - Station can only receive

Note: 10GE: Only Full Duplex

13

10base2 Ethernet



14

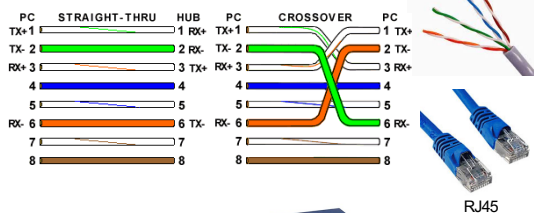
Ethernet Today

- Very few coaxial cable networks
 - Coax is heavy and expensive
- Mostly twisted pair using Category 5 cables
 - Tighter the twist, the more resistant to interference
 - Full duplex: separate pairs for transmitting and receiving
- More Category 6 cabling being deployed
 - Plastic core to separate pairs (reduce cross-talk)
- More fibre-optic based Ethernet
 - Long distance
 - Very high speed
 - Does not conduct electricity!
 - Good when connecting buildings with under-ground cables.



15

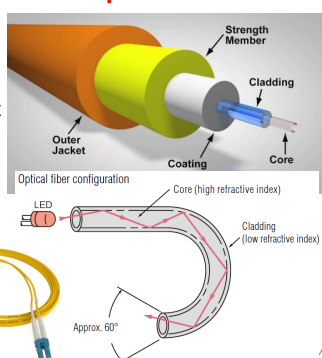
Ethernet Today: Cat 5 / Cat 6



16

Ethernet Today: Fiber Optics

- Glass carries light
- Light source LED or laser
- Cladding keeps light inside core due to refraction
- Coating etc is to keep water away from glass, provide strength



17

Wireless Ethernet

- Other common form of Ethernet is 802.11 wireless
 - 802.11b: up to 11Mbps (1999)
 - 802.11g: up to 54Mbps (2003)
 - 802.11n: up to 600Mbps (2009)
 - 802.11ac: up to 1300Mbps (2013)
 - 802.11ad: up to 7Tbps (2016)
- Uses same frame format as wired Ethernet, but physical layer is substantially different
- Operates in 2.4Ghz / 5Ghz "Industrial, Scientific, and Medical" (ISM) bands.
 - This frequency is an unlicensed band, which means anyone can use it
 - 2.4 is particularly used by cordless phones, microwave ovens, some baby monitors, Bluetooth, and wireless.

18

Wireless Ethernet: Access Points



- An Access Point (AP)
 - advertises the existence of a wireless network
 - acts as a "bridge" between wired and wireless Ethernet devices
- An Omni-directional antenna focuses wireless signal in a single (usually horizontal) plane.

19

Communicating in binary

- Each networked system has an independent clock
 - A separate clock signal is not provided
 - This will become clearer in the lecture
- The fundamental unit in computer communications is a bit having a value of one or zero.
- Data is usually sent in units of 8-bit bytes.

20

Experiment

- Receive a binary number
- Using hand signals to denote 0 or 1
 - A computer often uses voltage on a wire
- In class: what number did I send?

21

Experiment

- What did I send?
 - Was it 10101100 (172)
 - Or was it 01010011 (32)
- Need to define how data bits are sent!
 - We'll define one as raised hand, zero as lowered hand
 - We'll define each bit as occupying one second in time
- Could have also defined the opposite
 - One as lowered hand, zero as raised hand.

22

Communicating in binary

- Need to define when we're going to start transmitting
- Which means we also must define an idle state (i.e., what we should do when we have nothing to transmit)
- Idle state: assert 1 continuously (hand in air)
- To start a message: send a '0' bit

23

Communicating in binary

- Need some way to distinguish one byte from another
- It is not practical to send many bits as a single stream as clocks on different devices run at different rates
 - That is, they *drift*
- Class demo:

24

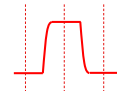
Communicating in binary

- I sent: 10101100 00000001
 - A stretch of nine continuous zeros.
 - Or was it ten continuous zeros and a stop bit?
- Need to define a maximum number of bits to send
 - i.e., when the transmitter should stop sending
 - i.e., the minimum requirement for clock accuracy
 - Computers often deal in 8-bit bytes, so 8 is a natural boundary
- Send 8 bits, then one stop bit (value of 1)
- Transmitter must use start bit to send next sequence

25

Issues

- Noise immunity
 - Can distinguish 0 and 1 fairly easily (most of the time)
 - Advantage of binary is clearer signalling than (say) ternary
- Timing immunity
 - Look at middle of each interval when signal is settled rather than at the start of each bit
 - Restart timing on every character
- Can add extra error checking
 - E.g., checksums, parity, etc.



26

Parity: Error Detection

- Two parity modes: even and odd.
- Typically applied to 8-bits (1 byte) at a time
- The total number of 1-bits in a 9-bit transmission must be either even or odd.
- If parity check fails, transmission discarded

Input	Even	Odd
00000000	00000000 0	00000000 1
01000110	01000110 1	01000110 0
11110000	11110000 0	11110000 1

27

Two-Dimensional Parity

- Simple example of Forward Error Correction (FEC)

00100100 0	• For a block of bytes, add parity bits for horizontal and vertical rows.
10100110 0	
11011100 1	• Allows receiver to correct single-bit errors
10101010 0	
11111000 1	• Cost: a 16-bit (2 byte) overhead for a 64-bit (8 byte) transmission
00000100 1	
00101000 0	
01001100 1	
01101100	

28

Two-Dimensional Parity

- Simple example of Forward Error Correction (FEC)

00100010 0	• Which bit is incorrect, assuming even parity?
10100110 0	
11011101 0	
10101010 0	
11111000 1	
01010100 1	
00101010 1	
01001111 0	
00111000	

29

Two-Dimensional Parity

- Simple example of Forward Error Correction (FEC)

00100010 0	• Which bit is incorrect, assuming even parity?
10100110 0	
11011101 0	
10101010 0	
11111000 1	
01010100 1	
00101010 1	
01001111 0	Row has 5 1-bits, and 0 parity bit
00111000	Column has 5 1-bits, and 0 parity bit

30

Serial communications

- Used on *serial port* of computers
 - For communication with dial-up modem
 - Serial mouse (before PS/2, USB, Bluetooth mice)
 - For REX board used in COMPX203
- High speed serial busses are common:
 - SATA, USB, PCIe lanes
- Standard agreed speeds (baud rate)
 - 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200
 - Use crystal oscillator for precise frequency control

31

Serial communications

- Key questions of serial I/O communications systems:
 - How and when are data bits sent?
 - When do we start and stop?
 - What if there are timing errors?
 - What if there are errors on the line?

32

COMPX204-22B Practical Networking and Cyber Security

Lecture 6 – Ethernet #2

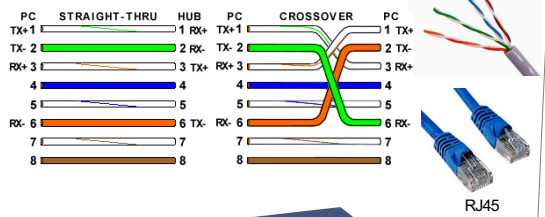
Matthew Luckie
mluckie@waikato.ac.nz



4 August 2022

1

Ethernet Today: Cat 5 / Cat 6



RJ45

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

4 August 2022

2

2

Learning Objectives

1. Understand how data can be *serialised* onto a physical medium as 1s and 0s
 - how to avoid timing issues from independent running clocks
 - how to detect and correct errors in transmission

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

4 August 2022

3

3

Twisted pair: Signals and Timing

- Ethernet systems use an encoding system that combines signal and timing
- Goal is to provide regular transitions in voltage level so receiver can ensure their clock is running at an appropriate rate
- 10baseT: Manchester encoding
- 100baseT: 4B5B
- 1000baseT: more advanced encoding

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

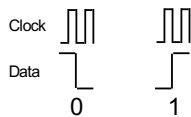
4 August 2022

4

4

10baseT: 802.3 Manchester Encoding

- Goal is to ensure regular voltage transition for clocking
- 0 is conveyed by going from high to low
- 1 is conveyed by going from low to high
- Clock rate is twice the data rate: not terribly efficient



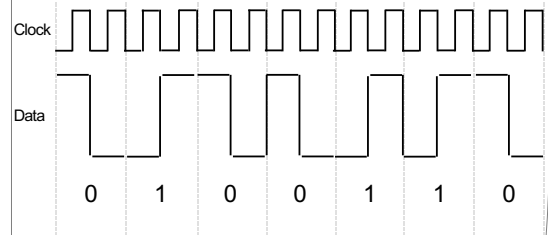
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

4 August 2022

5

5

10baseT: 802.3 Manchester Encoding



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

4 August 2022

6

6

100baseT 4B5B + MLT3

- Cat5 supports 125Mhz. Use 4B5B + MLT3

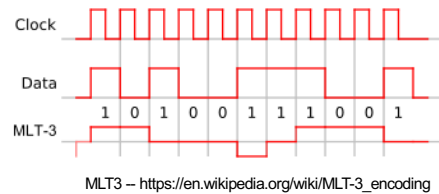
0000 → 11110	1000 → 10010
0001 → 01001	1001 → 10011
0010 → 10100	1010 → 10110
0011 → 10101	1011 → 10111
0100 → 01010	1100 → 11010
0101 → 01011	1101 → 11011
0110 → 01110	1110 → 11100
0111 → 01111	1111 → 11101

4B5B Dictionary

7

100baseT 4B5B + MLT3

- Cat5 supports 125Mhz. Use 4B5B + MLT3
- MLT3: cycle sequentially through -1, 0, 1, 0 each time a "1" is transmitted.
- 4B5B guarantees at least two "1"s per 4-bit sequence



8

1000baseT

- Cat5 supports 125Mhz. How do we get 10x bandwidth of 100baseT from Cat5 cables?
- Approach:
 - Use four twisted pairs in cable, not just two, and transmit / receive on all four twisted pairs
 - Use additional voltage levels to encode multiple bits

9

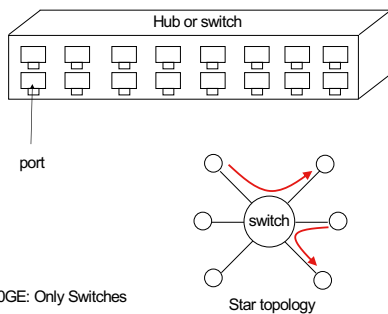
Cat5 (and 6) Ethernet

- Category 5 twisted pair
- Most common scenario
- Hubs
 - Send frames out all ports (broadcast)
 - Cheap devices, inefficient, not seen very much now since switches are also cheap
- Switches
 - Listen to traffic arriving on ports, remember (learn) which nodes transmit through each port
 - Direct frames out appropriate port as learned
 - 10G and up is only switched (or point to point)



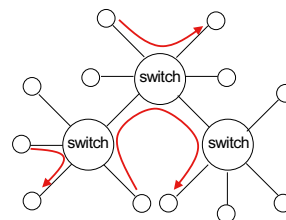
10

Cat5 Ethernet



11

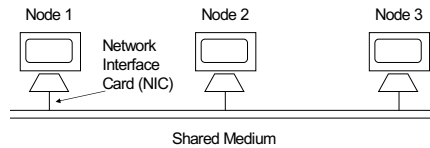
Cat5 Ethernet



Frames can be switched so that the Ether can carry multiple frames simultaneously, since they use different physical paths

12

Ethernet Addressing

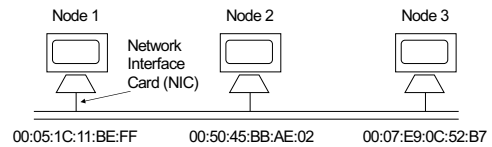


Ethernet systems use a shared medium to communicate.
We require the ability to distinguish between nodes when sending messages.
We do so using addresses.

4 August 2002

13

Ethernet Addresses



- Ethernet addresses

- known as Ethernet MAC addresses (Medium Access Control)
- are 48-bits in size. (6 bytes)
- are unique (barring errors)
- are assigned by the NIC manufacturer
- are usually written as six pairs of hexadecimal characters, each pair separated by a colon

4 August 2015

14

Ethernet Addressing

- Unicast addresses

- The usual case: Only the addressee uses the message
- [bbbbbbb0]:xx:xx:xx:xx

- Multicast addresses

- Sender addresses all machines who want to listen
- [bbbbbbb1]:xx:xx:xx:xx

- Broadcast messages

- Sender addresses all machines on the network
- FF:FF:FF:FF:FF:FF

```
sorcerer$ /sbin/ifconfig
em0          media: (1000baseT <full-duplex>)
             ether 98:90:96:dc:81:10
```

4 August 2002

15

Network Interfaces

- Can receive, by default

- All messages for its own unicast address
- All messages broadcast

- Can be programmed to receive multicast messages

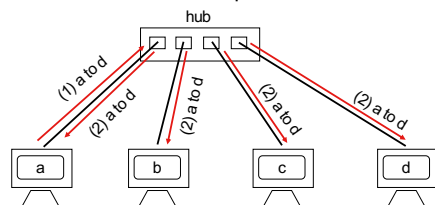
- Can be programmed to receive all messages (*promiscuous mode*)

4 August 2015

16

Ethernet hubs

- Ethernet hubs forward messages out all ports
- Cheap devices, inefficient, not seen very much now since switches are also cheap



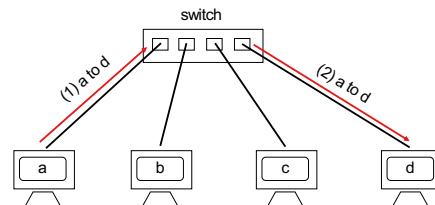
4 December 2007

17

Ethernet switches

- Ethernet switches usually only forward messages out the appropriate port

- Interfaces in promiscuous mode won't see messages that the switch does not forward

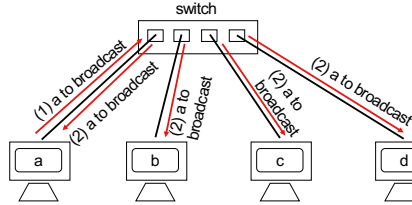


4 August 20

18

Ethernet switches

- Ethernet switches will forward broadcast messages out all ports
 - Interfaces in promiscuous mode won't see messages that the switch does not forward



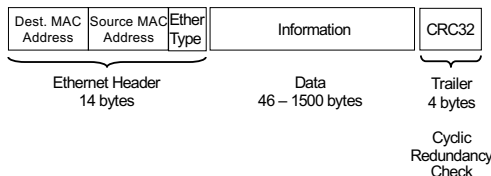
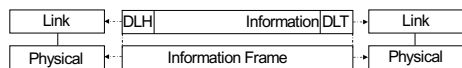
19

Learning Objectives

- Understand the format of an Ethernet frame
- Appreciate the scalability benefits of being able to assign a structure to addresses
- Understand the process of finding the Ethernet MAC address associated with an IPv4 or IPv6 address
- Understand that a host will only perform ARP and NDP for systems in the local IPv4 or IPv6 network

20

Ethernet Frames



21

Ethernet Frames

- Destination MAC Address: destination!
- Source MAC Address: source!
- EtherType: protocol type
 - Allows multiple types of Network protocols to be carried in Ethernet frames
 - IPv4 0x0800
 - IPv6 0x86DD
- CRC32: error checking field
 - Fairly strong technique to detect (but not correct) errors in received message

22

Ethernet Frames on the wire

- Ethernet also specifies how the frame should be transmitted on the physical medium
 - 8 byte preamble before transmission of frame:
 - 7 bytes of 10101010
 - 1 byte of 10101011
 - Used to allow receiver to synchronise its clock
 - 12 byte (96 bit) delay after transmitting frame before node can send again
 - Information must be at least 46 bytes
 - $46 + 14 + 4 = 64$ bytes
 - Required to allow receiver to detect collision

23

So far

- Specified physical characteristics of Ethernet
- Specified Ethernet link layer
- We've now encountered multiple types of addresses
 - DNS names sorcerer.cms.waikato.ac.nz
 - IPv4 addresses 130.217.250.39
 - IPv6 addresses 2001:df0:4:4000:9a90:96ff:fedc:8110
 - Ethernet MAC addresses 98:90:96:dc:81:10
- Why are there multiple addressing schemes?

24

Networks

00:ab:13:43:2a:ff
02:30:4f:a1:b4:11
0c:12:11:ab:fe:ed
06:31:f3:89:40:ff
0a:35:72:19:aa:f1
08:99:22:21:1a:6a

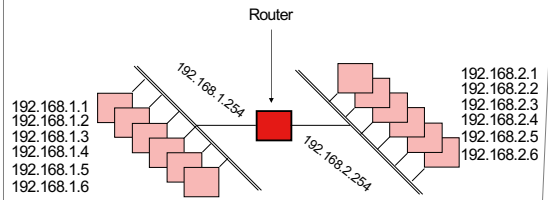
MAC addresses are wired in (flat)

© THE UNIVERSITY OF BRISTOL - TE WAREHOUSE CLASHED

4 August 2022

25

Networks



MAC addresses are wired in (flat)
IPv4 addresses are chosen (structured)

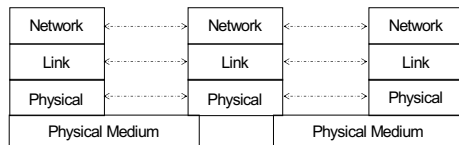
© THE UNIVERSITY OF BRISTOL - TE WAREHOUSE CLASHED

4 August 2022

26

Protocol Stacks

- Physical, Link: Ethernet
- Network: IPv4, IPv6

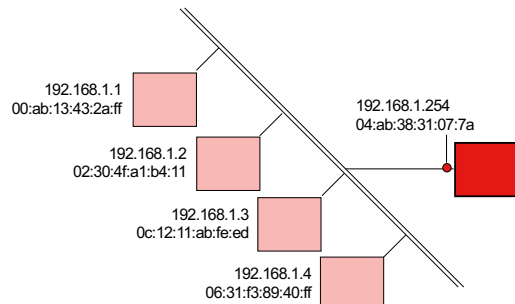


© THE UNIVERSITY OF BRISTOL - TE WAREHOUSE CLASHED

4 August 2022

27

Networks



© THE UNIVERSITY OF BRISTOL - TE WAREHOUSE CLASHED

4 August 2022

28

IP over Ethernet Communications

- From the software writer's perspective, the network protocols used are IP based.
 - Socket and ServerSocket open ports bound to IP addresses
- Application writers do not use Ethernet addresses
- The operating system handles communicating over Ethernet

© THE UNIVERSITY OF BRISTOL - TE WAREHOUSE CLASHED

4 August 2022

29

IPv4 over Ethernet

- The operating system needs to find out what destination Ethernet MAC address should be used when sending an IP packet
- The operating system does this using the Address Resolution Protocol (ARP)

© THE UNIVERSITY OF BRISTOL - TE WAREHOUSE CLASHED

4 August 2022

30

Address Resolution Protocol (ARP)

192.168.1.1
00:ab:13:43:2a:ff

A

B

192.168.1.3
0c:12:11:ab:fe:ed

C

- Computer A wants to talk to C
- Link Layer is Ethernet
- Computer A needs to know how to fill out the Ethernet fields

© THE UNIVERSITY OF WINNEDOT - 10 WARRIOR WARRIOR CLASSMATE

4 August 2022

31

Address Resolution Protocol (ARP)

192.168.1.1
00:ab:13:43:2a:ff

A

B

192.168.1.3
0c:12:11:ab:fe:ed

C

- A broadcasts a packet asking C to identify itself
- Ethernet header:
 - To: ff:ff:ff:ff:ff:ff
 - From: 00:ab:13:43:2a:ff
- Ethernet Payload:
 - Who-has 192.168.1.3

© THE UNIVERSITY OF WINNEDOT - 10 WARRIOR WARRIOR CLASSMATE

4 August 2022

32

Address Resolution Protocol (ARP)

192.168.1.1
00:ab:13:43:2a:ff

A

B

192.168.1.3
0c:12:11:ab:fe:ed

C

- C replies to A, identifying itself
- Ethernet header:
 - To: 00:ab:13:43:2a:ff
 - From: 0c:12:11:ab:fe:ed
- Ethernet Payload:
 - 192.168.1.3 is-at 0c:12:11:ab:fe:ed

© THE UNIVERSITY OF WINNEDOT - 10 WARRIOR WARRIOR CLASSMATE

4 August 2022

33

Address Resolution Protocol (ARP)

- A computer does not do ARP every time it needs to send something
- Instead, it keeps a record of recent replies to arp requests
 - ARP cache
 - Periodically, entries are expired
- You can view the contents of the arp cache using
 - arp -a -n
 - /sbin/arp or /usr/sbin/arp depending on system

© THE UNIVERSITY OF WINNEDOT - 10 WARRIOR WARRIOR CLASSMATE

4 August 2022

34

ARP Cache

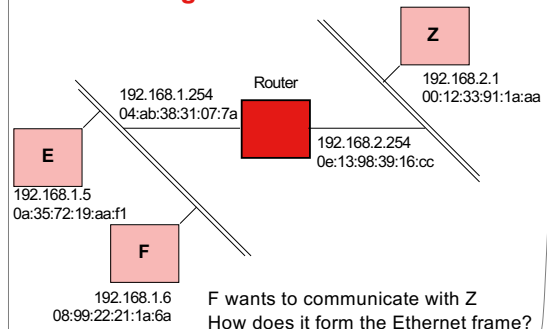
```
[mluckie@sorcerer ~]$ /usr/sbin/arp -an
? (130.217.250.39) at 98:90:96:dc:81:10 on em0
permanent [ethernet]
? (130.217.250.15) at 00:30:48:91:0d:94 on em0
expires in 1199 seconds [ethernet]
? (130.217.250.251) at 00:08:e3:ff:fd:10 on em0
expires in 1200 seconds [ethernet]
```

© THE UNIVERSITY OF WINNEDOT - 10 WARRIOR WARRIOR CLASSMATE

4 August 2022

35

IP Networking



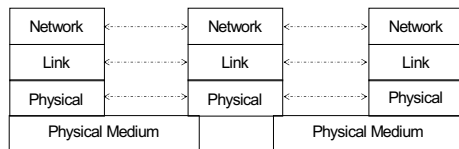
© THE UNIVERSITY OF WINNEDOT - 10 WARRIOR WARRIOR CLASSMATE

4 August 2022

36

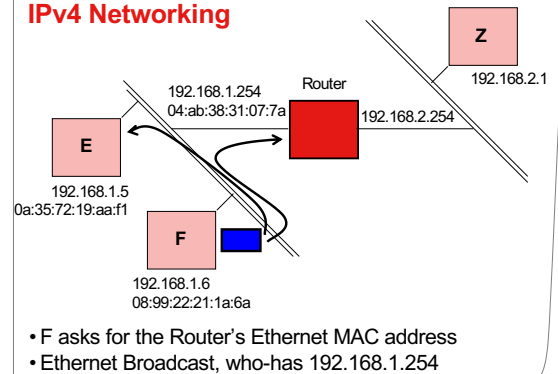
Protocol Stacks: IPv4 / Ethernet

- Physical, Link: Ethernet
- Network: IPv4



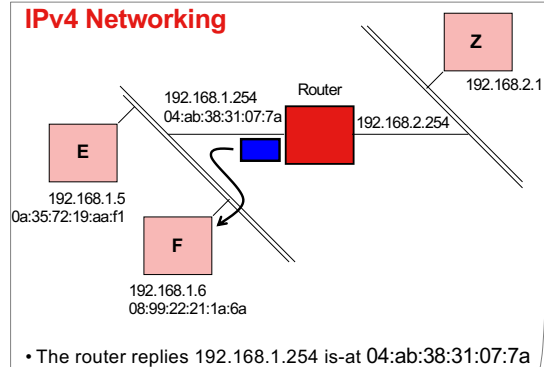
37

IPv4 Networking



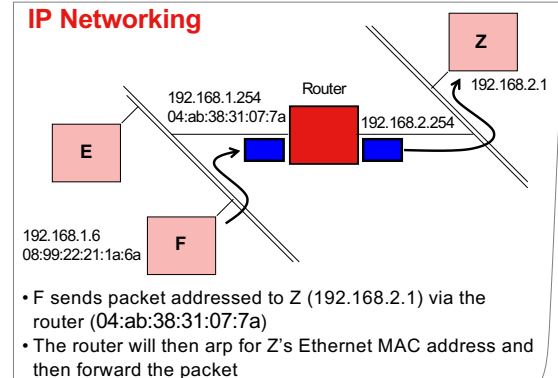
38

IPv4 Networking



39

IP Networking



40

IPv6 Neighbor Discovery Protocol (NDP)

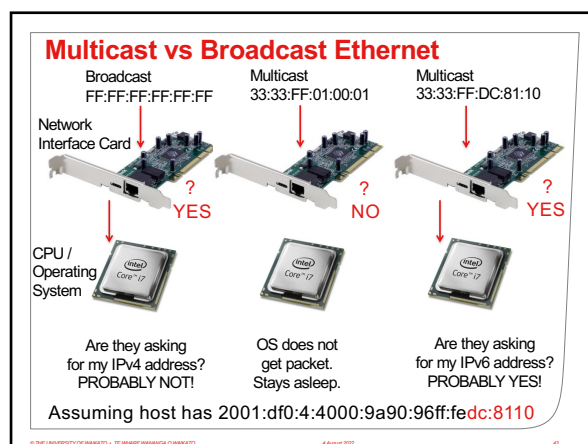
- IPv6 uses a different protocol to obtain Ethernet MAC addresses for IPv6 addresses ("neighbor discovery")
- Host sends NDP frame to multicast Ethernet address
 - 33:33:FF:<lower 24 bits of IPv6 address>
- 2001:df0:4:4000::1:1
 - Expanded: 2001:0df0:0004:4000:0000:0000:0001:0001
 - Dest MAC: 33:33:FF:01:00:01
- 2001:df0:4:4000:9a90:96ff:fedc:8110
 - Dest MAC: 33:33:FF:DC:81:10

41

IPv6 Neighbor Discovery Protocol (NDP)

- Key Difference:
- IPv4 ARP: Broadcast; every host receives and processes the ARP request, even when not for it
- IPv6 NDP: Multicast; all hosts will receive frame but only Ethernet devices configured to process frames with specific multicast address will pass frame to operating system.
 - Only hosts with the same last 24 bits in IPv6 address, in usual case only one host.

42



43

NDP cache

```
[mluckie@sorcererer ~]$ ndp -an
```

Neighbor	Linklayer Address	Netif	Expire	S	Flags
2001:df0:4:4000::250:15	00:30:48:91:0d:94	em0	23b51ml3s	S	
2001:df0:4:4000::250:39	98:90:96:dc:81:10	em0	permanent	R	R
fe80::208:e3ff:feff:fd10%em0	00:08:e3:ff:fd:10	em0	19s	R	R
2001:df0:4:4000:9a90:96ff:fedc:8110	98:90:96:dc:81:10	em0	permanent	R	
fe80::9a90:96ff:fedc:8110%em0	98:90:96:dc:81:10	em0	permanent	R	

Like the ARP cache

S: S = Stale, R = Reachable

Flags: R = Router

44

Summary

- ARP maps IPv4 addresses to Ethernet MAC addresses
- NDP maps IPv6 addresses to Ethernet MAC addresses
- ARP/NDP tables are dynamically managed
- ARP/NDP tables are scoped to a single Ethernet bus
- Standards documents (RFCs):
 - RFC 826: IPv4 ARP
 - RFC 4861: IPv6 Neighbor Discovery

45

Learning Objectives

- Understand the format of an Ethernet frame
- Appreciate the scalability benefits of being able to assign a structure to addresses
- Understand the process of finding the Ethernet MAC address associated with an IPv4 or IPv6 address
- Understand that a host will only perform ARP and NDP for systems in the local IPv4 or IPv6 network

46

COMPX204-22B Practical Networking and Cyber Security

Lecture 7 – Domain Name System, UDP

Matthew Luckie
mluckie@waikato.ac.nz



© August 2022

1

Learning Objectives

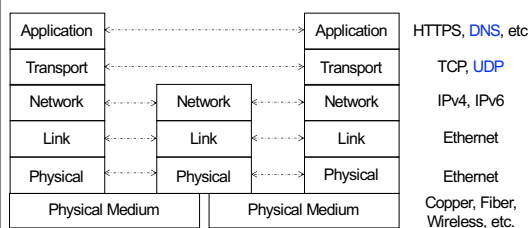
- Understand how the Domain Name System (DNS) is deployed and queried as a distributed hierarchical database
- Be able to write an application that uses the DatagramSocket class to send and receive DatagramPackets using the User Datagram Protocol (UDP)

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

© August 2022

2

Protocol Layers (the Internet view)



- User Datagram Protocol (UDP): Transport Protocol
- Domain Name System (DNS): Application Protocol

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

© August 2022

3

3

DNS

- Domain Name System (DNS)
 - Implemented at the application level
- Provides a mapping from a friendly name to an IP address
 - Among many other services
 - `host` and `dig` commands on Linux, MacOS
 - Java `InetAddress` class
- We have talked about DNS before. This lecture is a little about how it is implemented
 - Hierarchy
 - Distributed network databases
 - Mostly transported over User Datagram Protocol (UDP)

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

© August 2022

4

4

What do we know about DNS?

- Maps a name to an IP address. Names are meaningful to humans and more easily remembered
- Names have a hierarchical form; e.g.:
 - sorcerer.cms.waikato.ac.nz : 130.217.250.39
 - : 2001:df0:4:4000::250:39
 - www.wand.nz : 130.217.250.15
 - www.trademe.co.nz : 151.101.2.137
 - www.caida.org : 192.172.226.122
 - : 2001:48d0:101:501::122
- Asking the DNS to map a name to an IP address does not take very long

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

© August 2022

5

5

DNS hierarchy

- Top Level Domains (TLDs)
 - gTLD: generic top level domains
 - .com, .net, .org, .edu, .gov
 - ccTLD: country-code top level domains
 - .nz, .au, .uk, .us, .ca
- Second level domains (SLDs)
 - Depends on the policy of the TLD as to what goes here
 - .co.nz, .net.nz, .org.nz, .ac.nz, .govt.nz, .wand.nz
 - .com.au, .org.au, .id.au, .gov.au, .edu.au
 - amazon.com, ucsd.edu, whitehouse.gov, endace.com
- Hierarchy tells you who issued the name, not where the address is physically located

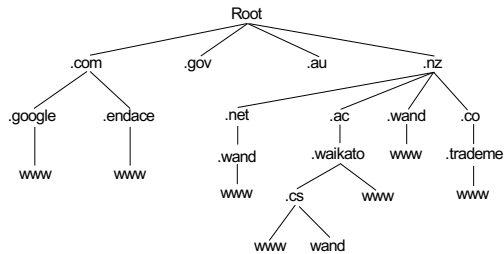
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

© August 2022

6

6

DNS Hierarchy (abbreviated!)



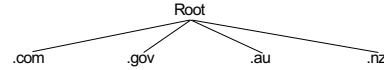
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

8 August 2022

7

DNS Hierarchy

- At each level of the hierarchy exists a set of servers which can either resolve the name, or point the questioner at another server that can help them



- Root servers: list of DNS servers known to exist at the top of the tree

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

8 August 2022

8

Root Servers

- (A through M).root-servers.net
- 13 pre-defined IP addresses
 - These addresses do change periodically, but not all at the same time
- Once upon a time 13 individual points of failure:
 - most of which were in the USA.
 - 13 individual points of failure is not that many for a critical service like DNS
- These days much more robust through being widely distributed

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

8 August 2022

8

Root Servers

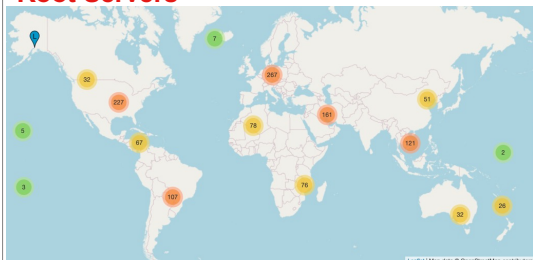
• A: Verisign	198.41.0.4 / 2001:503:ba3e::2:30
• B: University of Southern California	199.9.14.201 / 2001:500:200:b
• C: Cogent	192.33.4.12 / 2001:500:2::c
• D: University of Maryland	199.7.91.13 / 2001:500:2d::d
• E: NASA	192.203.230.10 / 2001:500:a8::e
• F: Internet Systems Consortium	192.5.5.241 / 2001:500:2f::f
• G: US Department of Defense	192.112.36.4 / 2001:500:12::d0d
• H: US Army Research Lab	198.97.190.53 / 2001:500:1::53
• I: Netnod (Sweden)	192.36.148.17 / 2001:7fe::53
• J: Verisign	192.58.128.30 / 2001:503:c27::2:30
• K: RIPE NCC (NL)	193.0.14.129 / 2001:7fd::1
• L: ICANN	199.7.83.42 / 2001:500:9f::42
• M: WIDE Project (Japan)	202.12.27.33 / 2001:dc3::35
• IP addresses of individual letters change over time, but not at the same time	

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

8 August 2022

9

Root Servers



- Currently 1536 instances operated by 12 entities
- 1 entity (Verisign) operates two letters (A, J)
- NZ: 8 Auckland, 3 Wellington, 2 Christchurch

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

8 August 2022

11

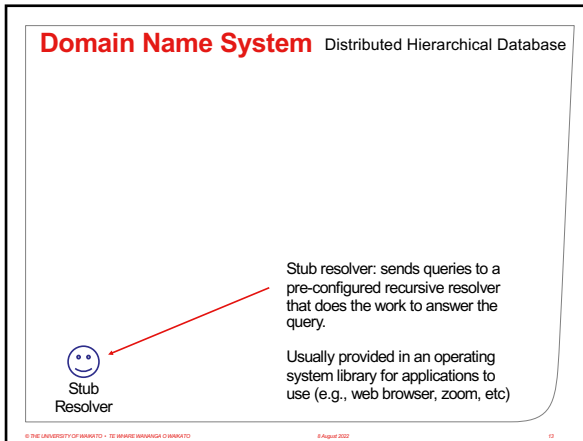
Domain Name System

The Classic
Distributed Hierarchical Database

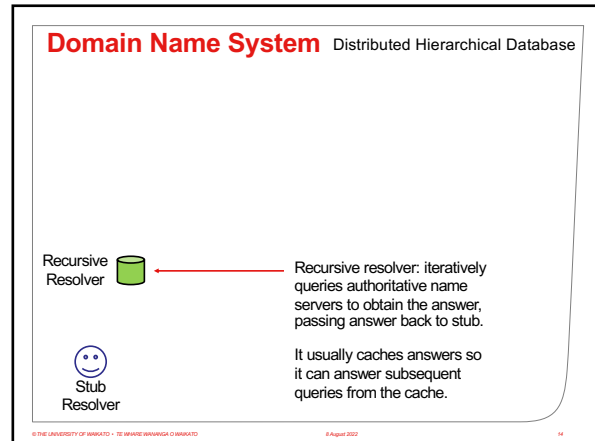
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

8 August 2022

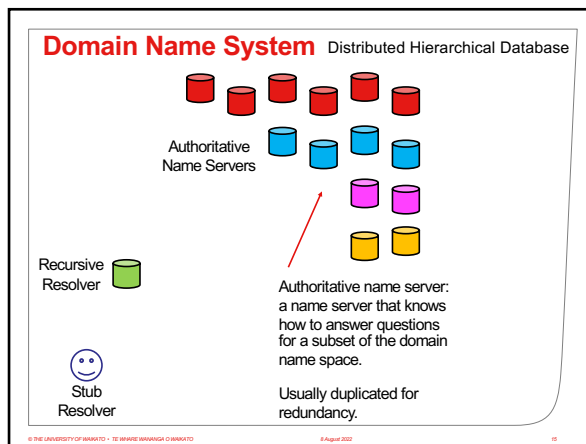
12



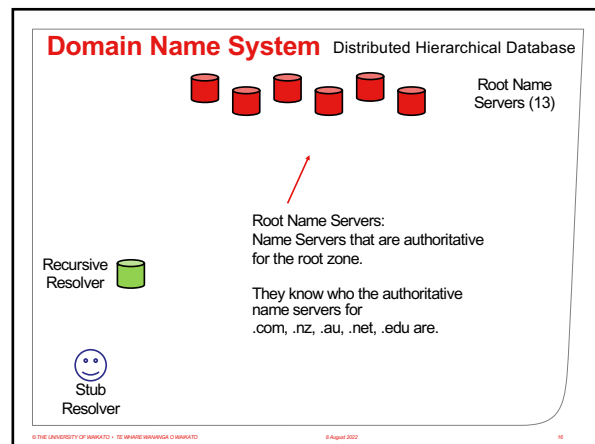
13



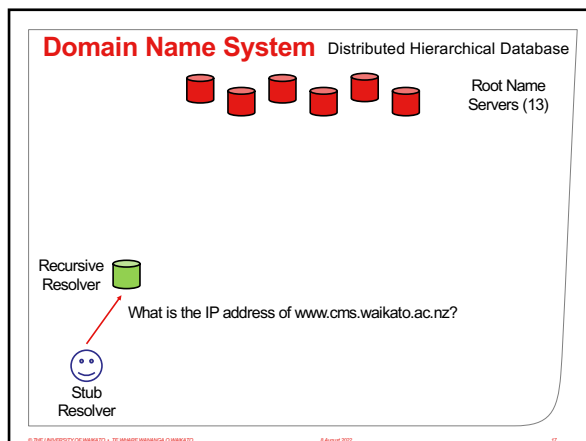
14



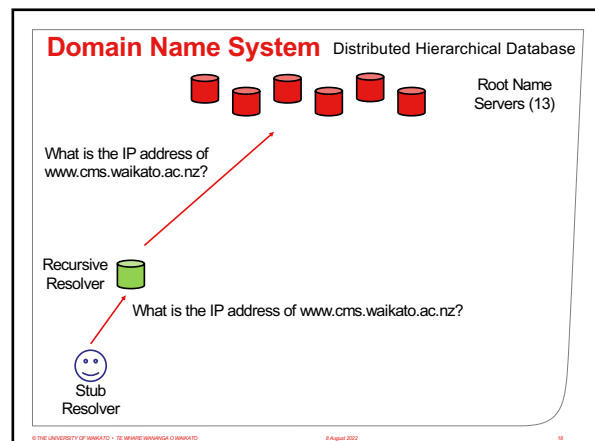
15



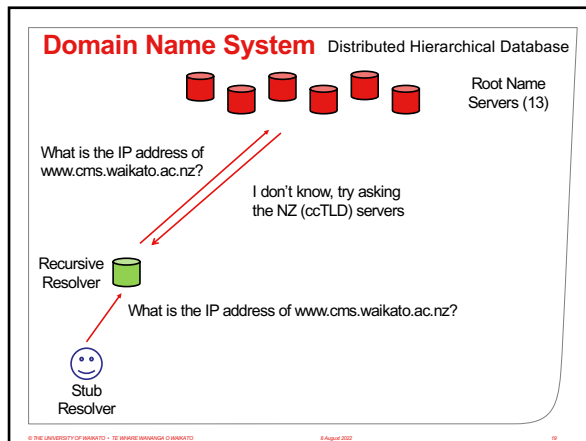
16



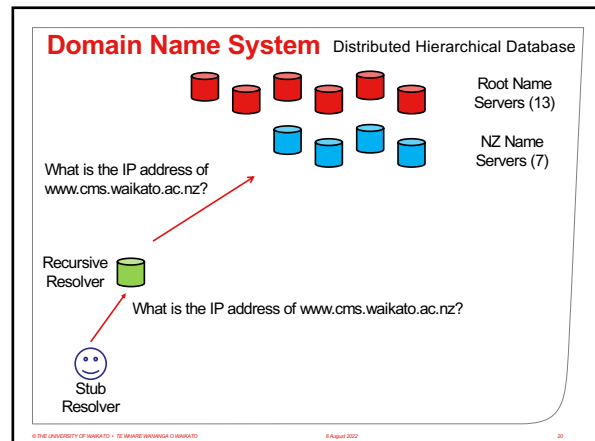
17



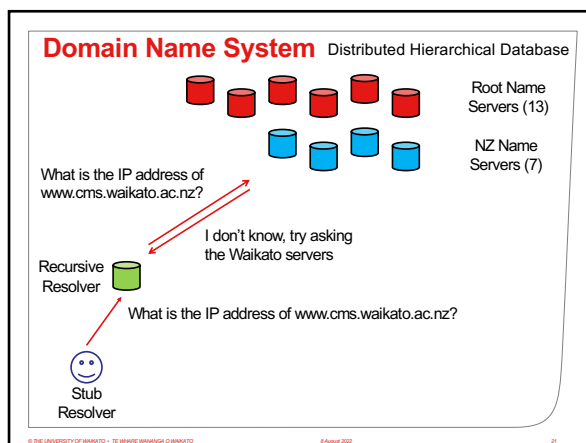
18



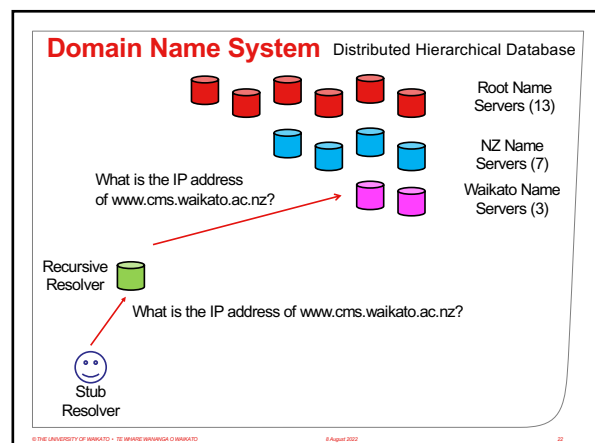
19



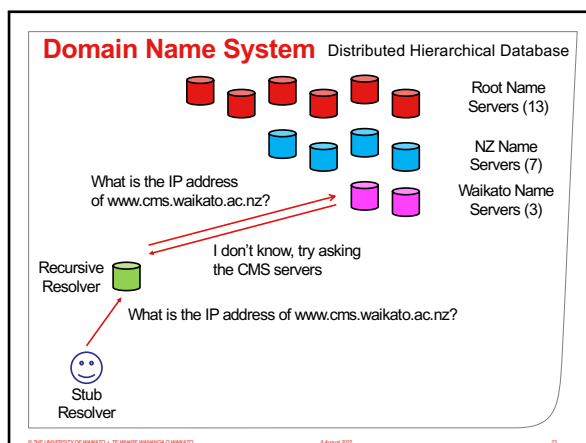
20



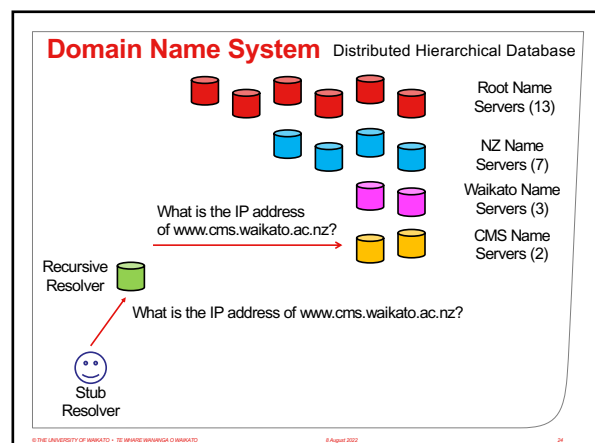
21



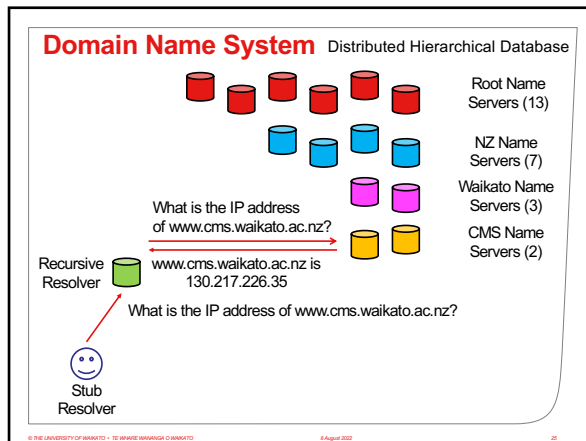
22



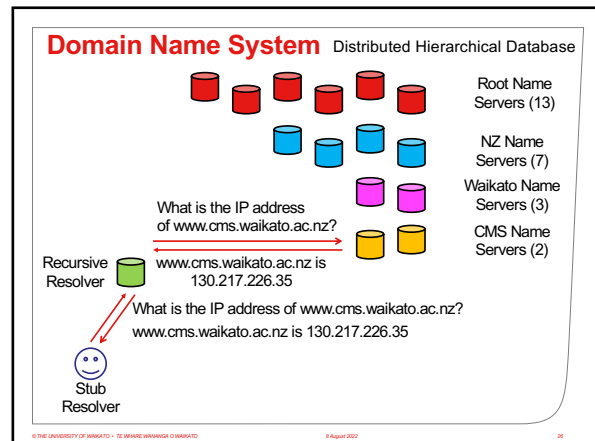
23



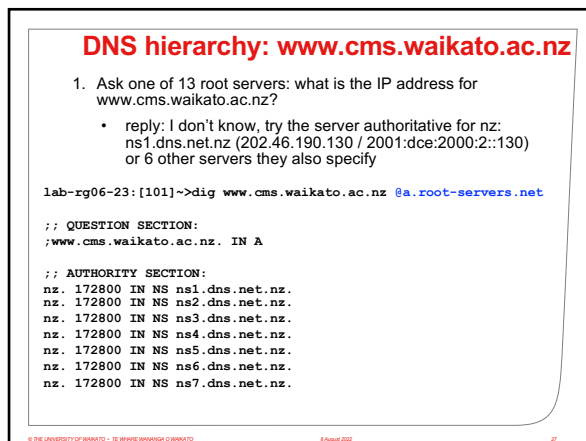
24



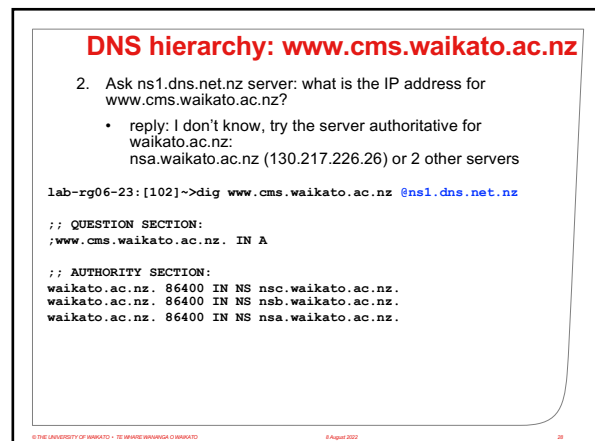
25



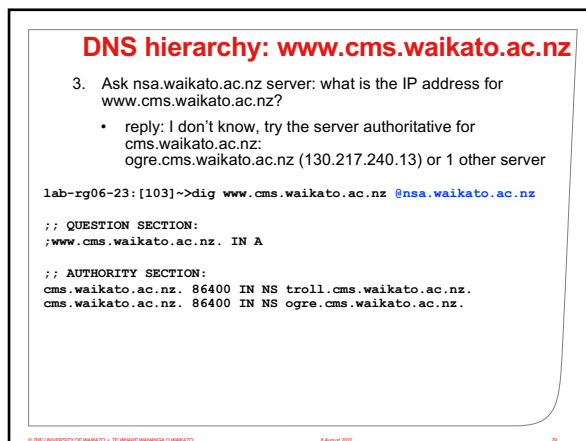
26



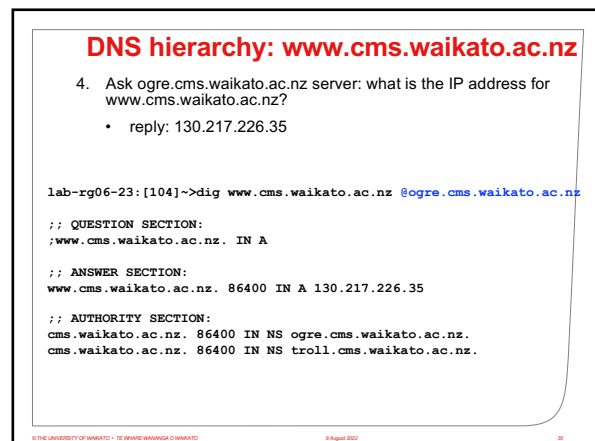
27



28



29



30

DNS hierarchy

- `dig www.cms.waikato.ac.nz @a.root-servers.net`
- `dig www.cms.waikato.ac.nz @ns1.dns.net.nz`
- `dig www.cms.waikato.ac.nz @nsa.waikato.ac.nz`
- `dig www.cms.waikato.ac.nz @ogre.cms.waikato.ac.nz`
- answer: `www.waikato.ac.nz 130.217.226.35`
- Recursive resolvers use an iterative look-up and cache the answer

31

DNS hierarchy

- A note: the DNS hierarchy is nothing like what the underlying network topology looks like.
 - For nz zone
 - ns1 - ns4, and ns7.dns.net.nz all seem to be in NZ
 - ns5 appears to be on the US West Coast (Los Angeles?)
 - ns6 appears to be in Australia (Sydney?)
- It is a distributed database, each server having been delegated some authority to answer queries.

32

DNS caches

- As with ARP, DNS resolvers keep a cache of recently looked up records
 - So each time an IP address is required, the resolver does not have to repeat the process
 - The length of time they keep each entry is defined by a person, and varies from minutes to days.
 - TTL: time to live
- ```
www.cms.waikato.ac.nz. 86400 IN A 130.217.226.35
www.google.co.nz. 300 IN A 142.250.66.163
```
- 300 seconds = 5 minutes
  - 86400 seconds = 24 hours

33

## DNS and UDP

- DNS has short transactions that usually fit in one packet
  - Typically, just a name and address
  - No need for ordering (single packet)
  - Must be very fast as DNS lookups precede most interactions
- User Datagram Protocol (UDP)
  - Connectionless unreliable packet delivery
- UDP does not guarantee delivery, like TCP does
  - We know DNS is usually reliable
  - Recall that when looking up a name, the reply often will contain multiple other name servers to try
  - The resolver can try alternate name servers if the first does not respond in a short time.

34

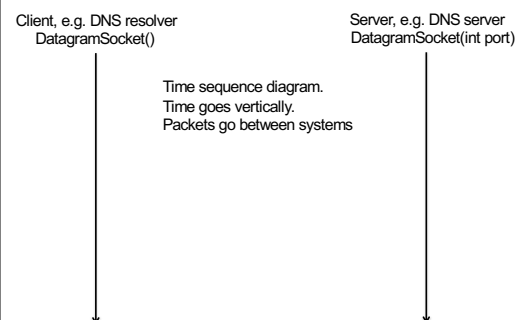
## UDP header

|             |                  |
|-------------|------------------|
| Source Port | Destination Port |
| UDP Length  | Checksum         |

- User datagram protocol (UDP)
  - UDP provides port numbers, a checksum, and a length field.
    - DNS: well known port 53
  - It does not provide much else.
    - Not reliable
    - Not "connection oriented"
    - Not a byte-stream service
- These features are provided by Socket (TCP), not DatagramSocket (UDP)

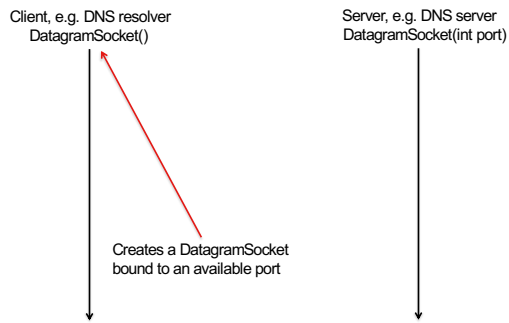
35

## Java: DatagramSocket



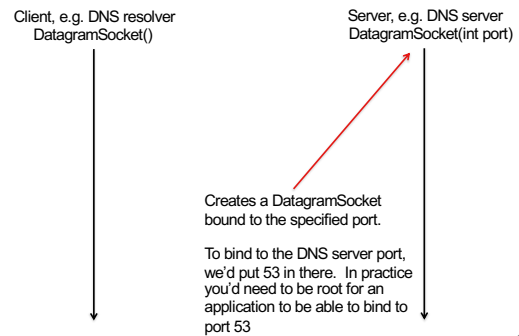
36

## Java: DatagramSocket



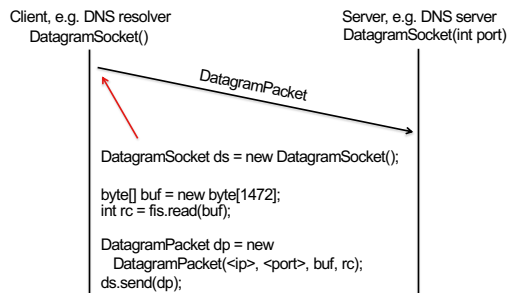
37

## Java: DatagramSocket



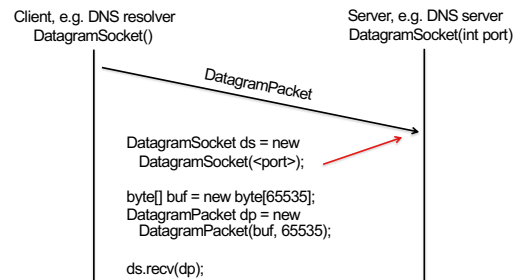
38

## Java: DatagramSocket transmitting



39

## Java: DatagramSocket receiving



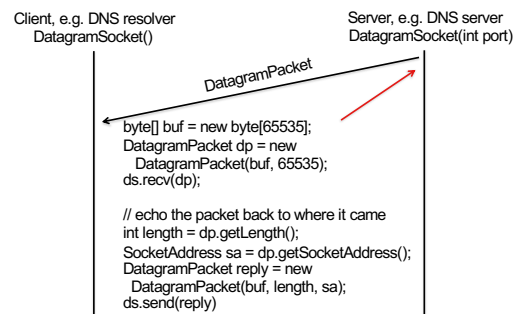
40

## Java: DatagramPackets

- Use `dp.getLength()` to determine how many bytes are valid in the buffer after a `recv()` call
- A single `DatagramSocket` can receive datagrams from many different IP addresses, as the `DatagramSocket` is not connected to anyone
- To determine where a received packet came from:
  - `DatagramPacket::getAddress` returns the `InetAddress`
  - `DatagramPacket::getPort` returns the sender's port
  - `DatagramPacket::getSocketAddress` returns both values in a single object

41

## Java: DatagramSocket replying



42

## Summary

- DNS is an application-level service that provides the ability to obtain an IP address from a more easily remembered name
- DNS is implemented as a distributed hierarchical database
- DNS has built-in redundancy, which the DNS application service takes advantage of
- UDP is an unreliable transport protocol

© THE UNIVERSITY OF MANITOBA - 10 MANITOBA WISDOMS COURSEWORK

8 August 2022

43

43

## Learning Objectives

- Understand how the Domain Name System (DNS) is deployed and queried as a distributed hierarchical database
- Understand the differences between TCP and the User Datagram Protocol (UDP) and when you would use the UDP protocol instead of TCP

© THE UNIVERSITY OF MANITOBA - 10 MANITOBA WISDOMS COURSEWORK

8 August 2022

44

44

# COMPX204-22B Practical Networking and Cyber Security

## Lecture 8 – HTTP

Matthew Luckie  
mluckie@waikato.ac.nz



11 August 2022

1

## Learning Objectives

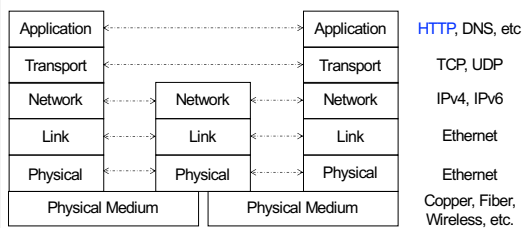
- Understand the basics of how HTTP works at an Application layer
- Appreciate how to construct Unit Tests for an HTTP server

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

11 August 2022

2

## Protocol Layers (the Internet view)



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

11 August 2022

3

3

## HTTP

- Hyper Text Transfer Protocol
- Protocol spoken by web servers
  - Most web servers prefer to speak HTTPS ("Secure HTTP")
  - HTTP: port 80 (communications are in the clear)
  - HTTPS: port 443 (communications are encrypted with TLS)
- Client-server protocol
  - HTTP servers wait online for clients to connect to them
  - Clients can come and go as they please
  - Clients obtain *resources* from the server

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

11 August 2022

4

4

## URL

- Uniform Resource Locator



- No need to supply port (80 is the well-known http port)
- You can supply IP addresses instead of server names, but that is not the normal approach

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

11 August 2022

5

5

## URL

- Uniform Resource Locator



- Speaking https to port 443

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

11 August 2022

6

6

## HTTP and HTML

### HTTP

- Hyper Text Transfer Protocol
- Used by clients to obtain resources
  - HTML documents
  - Images
  - Video

### HTML

- Hyper Text Markup Language
- Describes the structure of web pages using markup
- Contains links to resources

© THE UNIVERSITY OF Waikato - TE WHARE WAIKATO O Waikato

11 August 2022

7

## HTTP as a protocol

- Text-based (protocol messages)
  - Easy to read (for people) messages
- Request-response transactions
  - Client sends a request, Server sends a response

- Both request and response messages follow a similar pattern

- ASCII Text Header
- Empty Line
- Binary Data (optional)

ASCII Text Header  
Empty Line  
Binary Data

© THE UNIVERSITY OF Waikato - TE WHARE WAIKATO O Waikato

11 August 2022

8

## HTTP Requests

```
GET /study/ HTTP/1.1
Host: www.waikato.ac.nz
User-Agent: Mozilla/5.0 (X11; FreeBSD amd64; rv:78.0) Firefox/78.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Pragma: no-cache
Cache-Control: no-cache
```

Requested resource

- Header describes the request and requester
- End of header signaled by an empty line

© THE UNIVERSITY OF Waikato - TE WHARE WAIKATO O Waikato

11 August 2022

9

## HTTP Response: OK

```
HTTP/1.1 200 OK
Content-Encoding: gzip
Accept-Ranges: bytes
Age: 541190
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Wed, 21 Jul 2021 22:21:38 GMT
Etag: "3147526947"
Expires: Wed, 28 Jul 2021 22:21:38 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECS (sec/974D)
Vary: Accept-Encoding
X-Cache: HIT
Content-Length: 648
```

Protocol + Status Code

- HTTP responses contain an ASCII text header followed by an empty line, then binary data

© THE UNIVERSITY OF Waikato - TE WHARE WAIKATO O Waikato

11 August 2022

10

## HTTP File Not Found

### Request:

```
GET /study/fdsfd.html HTTP/1.1
Host: www.waikato.ac.nz
```

Requested resource

### Response:

```
HTTP/1.1 404 File Not Found
File not found
```

Protocol + Status Code

Rendered in browser

© THE UNIVERSITY OF Waikato - TE WHARE WAIKATO O Waikato

11 August 2022

11

## End of line sequence

- The sequence of bytes used to get to the next line depends on the operating system

- Windows: \r\n CR LF
- UNIX: \n LF
- MacOS < 10: \r CR

- CR byte value 13
- LF byte value 10

- HTTP defines end-of-line sequence as \r\n

© THE UNIVERSITY OF Waikato - TE WHARE WAIKATO O Waikato

11 August 2022

12

## readLine

- Java's readLine function will handle any combination of these symbols when reading lines of Strings from an InputStream

|         |      |       |
|---------|------|-------|
| • CR    | \r   | 13    |
| • LF    | \n   | 10    |
| • CR LF | \r\n | 13 10 |

- Java's println function will not necessarily emit the same sequence of bytes

- You cannot use Java's built-in println to write HTTP headers
- I will provide you a println for the HTTP assignment

13

## Handling binary data: writing header strings

```
String message = "HTTP/1.1 200 OK\r\n\r\n";
byte[] array = message.getBytes();

BufferedOutputStream bos = new
 BufferedOutputStream(client.getOutputStream());

bos.write(array, 0, array.length);
```

↑                      ↑  
start index      number of bytes

- Note: array.length works with this example because message.getBytes() creates a byte array exactly the needed size

14

## Binary Files

- Binary files will contain bytes, the exact interpretation of which depends on the type of file

- Image file
- Program file

- There will be bytes with values 13 (CR) and 10 (LF) sprinkled amongst the file

- They do not mean \r or \n in this situation

- We cannot use readLine to read binary files**

15

## Handling binary data: reading input files

Read binary files into a byte array

```
byte[] buf = new byte[1024];
FileInputStream fis = new FileInputStream(/* something */);

int rc = fis.read(buf);
while(rc != -1) /* read until end of file */
{
 /* do something with buf */

 /* read next part of file */
 rc = fis.read(buf);
}
fis.close();
```

16

## Handling binary data: writing binary data

```
byte[] buf = new byte[1024];
FileInputStream fis = new FileInputStream(/*something*/);

BufferedOutputStream bos = new
 BufferedOutputStream(socket.getOutputStream());

int rc = fis.read(buf);
while(rc != -1) /* read until end of file */
{
 /* do something with buf */
 bos.write(buf, 0, rc);

 /* read next part of file */
 rc = fis.read(buf);
}
bos.flush();
bos.close();
fis.close();
```

↑                      ↑  
start index      number of bytes

17

## Virtual Hosting

- HTTP supports hosting multiple websites on a single webserver
- Accomplished using the Host: header in the request

```
GET / HTTP/1.1
Host: www.facebook.com

GET / HTTP/1.1
Host: m.facebook.com
```



18

## What file to return when no file specified?

- If a URL has / at the end, what should the webserver return?
- Convention is to return the "index.html" file

GET / HTTP/1.1  
Host: www.waikato.ac.nz      /web/www.waikato.ac.nz/index.html

GET /study/ HTTP/1.1  
Host: m.facebook.com      /web/www.waikato.ac.nz/study/index.html

19

## Unit Testing and HTTP

- In your second assignment, you will implement a basic HTTP server
- An HTTP server receives interactions from untrustworthy clients
- You should write your server defensively
  - i.e., write tests that simulate an attacker, and ensure that your server does not crash or misbehave

20

## Unit Testing

- Given that software is designed and developed to do a particular job, it is generally desirable that tests be formally defined as part of the specification.
- Any time code is changed, it must be confirmed that the acceptance tests still pass.
- Thus, the tests would ideally be defined in a way that they are unambiguous and can be automatically carried out .... like a kind of computer program for testing.
- If the number of tests is very large, it may be useful to define and execute tests only on units of the system that have been changed.
- These notions combine in the discipline of *automated unit testing*.

21

## Unit Testing and Networking

- Difficult to test a networking application end-to-end
- Typical approach is to simulate packet sequences (or text sequences)
- HTTP header parsing is well suited to testing through simulation

22

## Suggested Program Structure

- **HttpServer**
  - Listen on a port for incoming requests, spawn a session for each
- **HttpServerSession** (in `HttpServer.java`)
  - Extends thread, reads and writes across the Socket
- **HttpRequest**
  - Carefully parses HTTP request line by line
- **HttpRequestTest**
  - Contains a set of unit tests for the `HttpRequest`

23

## HttpRequest

```
class HttpRequest
{
 private String file = null;
 private String host = null;
 private boolean done = false;
 private int line = 0;

 public boolean isDone() { return done; }
 public String getFile() { return file; }
 public String getHost() { return host; }

 public void process(String in)
 {
 /*
 * process the line, setting 'done' when
 * HttpServerSession should examine the contents
 * of the request using getFile and getHost
 */
 }
}
```

24

## HttpServerRequestTest

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class HttpServerRequestTest
{
 @Test
 @DisplayName("Test well-formed HTTP request for /")
 void wellformedSlash() {
 HttpServerRequest hsr = new HttpServerRequest();
 hsr.process("GET / HTTP/1.1");
 hsr.process("");

 assertTrue(hsr.isDone(), "isDone");
 assertEquals("index.html", hsr.getFile(), "getFile");
 assertNull(hsr.getHost(), "getHost");
 }
}
```

<https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>

25

## Downloading JUnit5 Console Launcher

- <https://junit.org/junit5/docs/current/user-guide/#running-tests-console-launcher>
- wget <https://repo1.maven.org/maven2/org/junit/platform/junit-platform-console-standalone/1.9.0/junit-platform-console-standalone-1.9.0.jar>

26

## Running JUnit5 in console

```
java -jar junit-platform-console-standalone-1.9.0.jar
-cp . -c HttpServerRequestTest

javac -cp junit-platform-console-standalone-1.9.0.jar
HttpServerRequestTest.java
HttpServerRequest.java
```

27

## Learning Objectives

- Understand the basics of how HTTP works at an Application layer
- Appreciate how to construct Unit Tests for an HTTP server

28

# COMPX204-22B Practical Networking and Cyber Security

## Lecture 9 – TCP Part 1

Matthew Luckie  
mluckie@waikato.ac.nz



15 August 2022

1

## TCP

- Transmission control protocol (TCP)
- TCP is known as a *transport protocol* that is:
  - reliable,
  - connection-oriented,
  - stream-based
- When you use Socket / ServerSocket in Java, you use TCP as your transport protocol.
- TCP is implemented in the operating system kernel, not the application software.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

15 August 2022

2

2

## Learning Objectives

- Be able to describe the key features of TCP
- Understand the relationship between the port number you provide a ServerSocket and how that is used in a TCP header
- Describe why a TCP packet has two port fields in a header
- Describe how the Sequence and Acknowledgement fields are used

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

15 August 2022

3

3

## Transport protocols in the Internet

- There are other transport protocols in the Internet
  - UDP – User Datagram Protocol
  - SCTP – Stream Control Transmission Protocol
  - DCCP – Datagram Congestion Control Protocol
- They have different features
  - We covered UDP in the last lecture. UDP provides unreliable packet transmission and application multiplexing (ports)

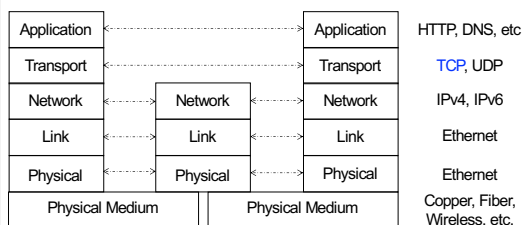
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

15 August 2022

4

4

## Protocol Layers (the Internet view)



- TCP is a *Transport Protocol*

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

15 August 2022

5

5

## TCP features

- Connection-oriented
  - Applications establish connections between computers using TCP (port numbers)
  - No broadcast or multicast mechanism
- Full duplex
  - Both client and server can send information to each other
- Reliable
  - Lost messages get retransmitted
  - Messages are protected using a (weak) checksum
- Network friendly / Fair
  - Avoids overwhelming the network's ability to forward information
  - Tries to be fair to others using the network at the same time

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

15 August 2022

6

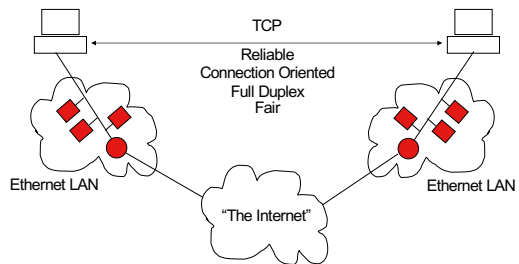
6

## TCP features

- Technical protocol specification provides for:
  - Connection oriented
  - Full duplex
  - Reliability
- Logic implemented in your operating system
  - Network friendliness / fairness
  - Relies on every implementation being fair

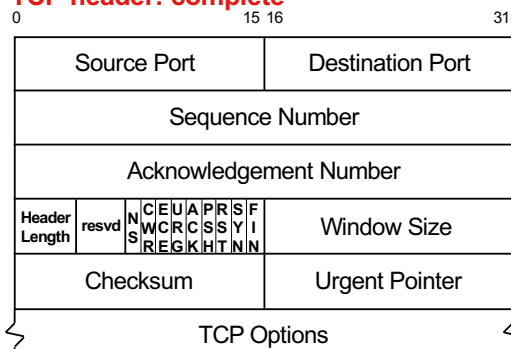
7

## Summary so far



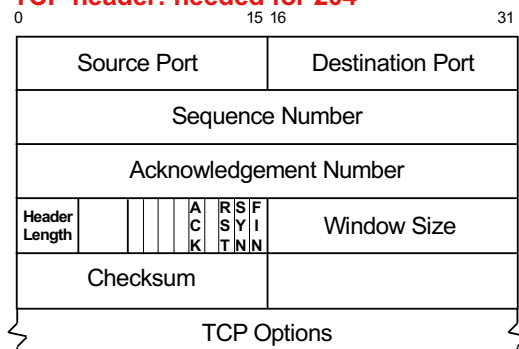
8

**TCP header: complete**



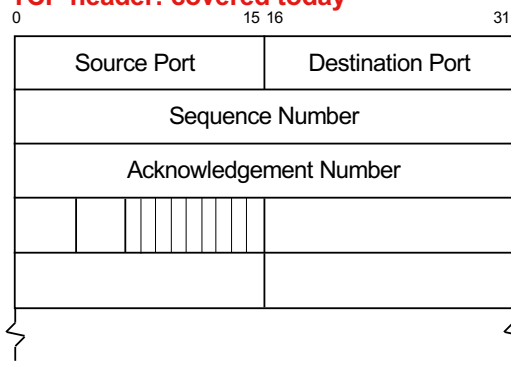
9

**TCP header: needed for 204**



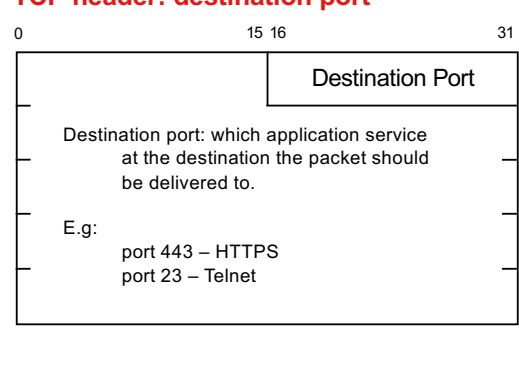
10

**TCP header: covered today**

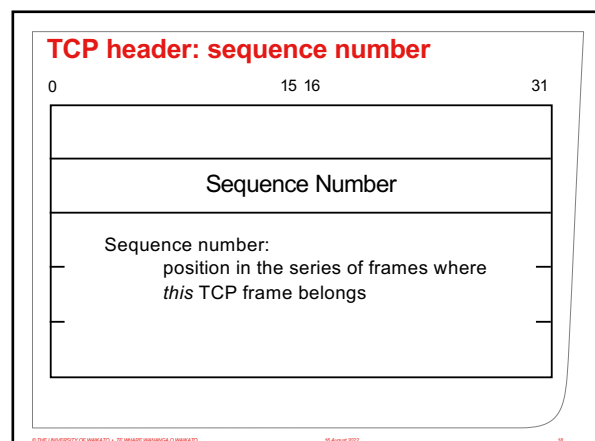
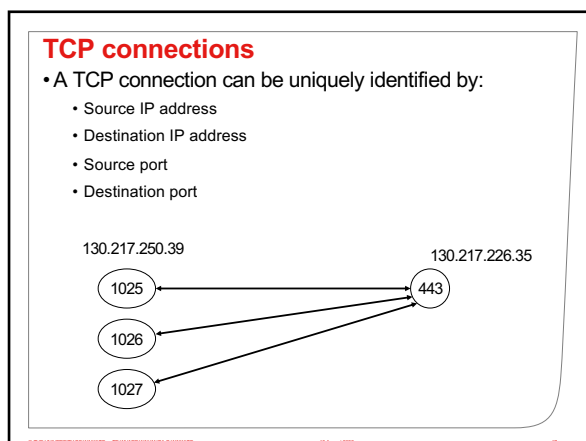
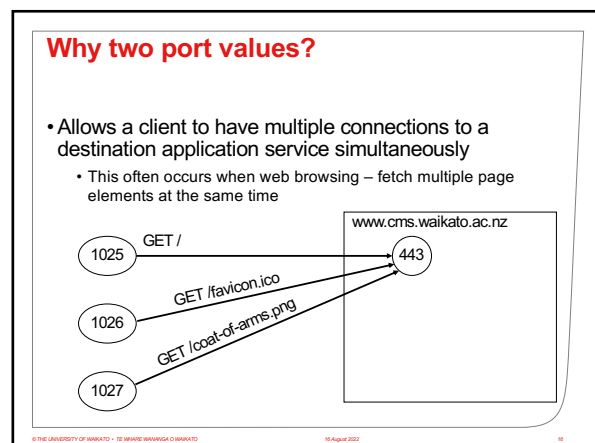
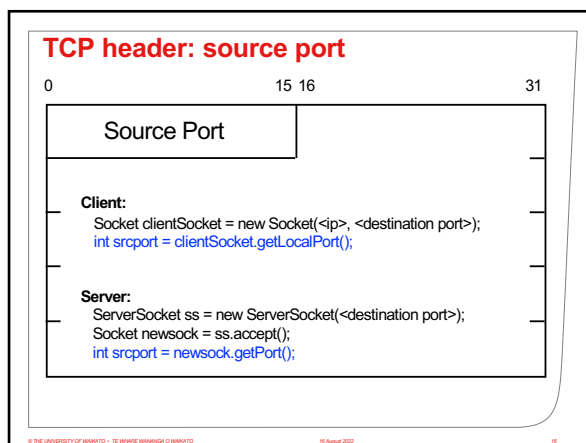
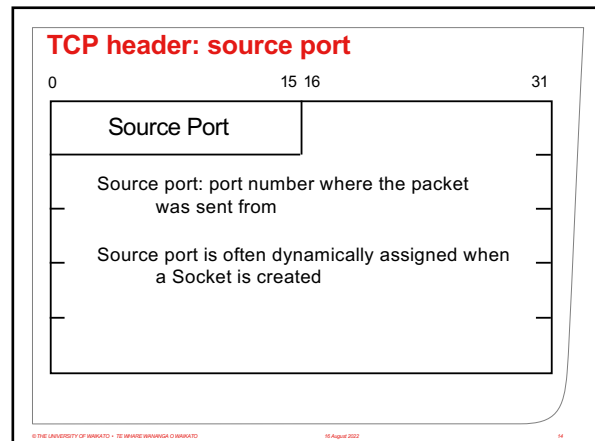
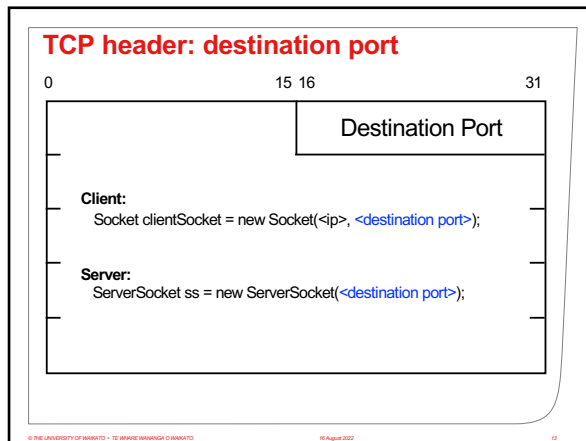


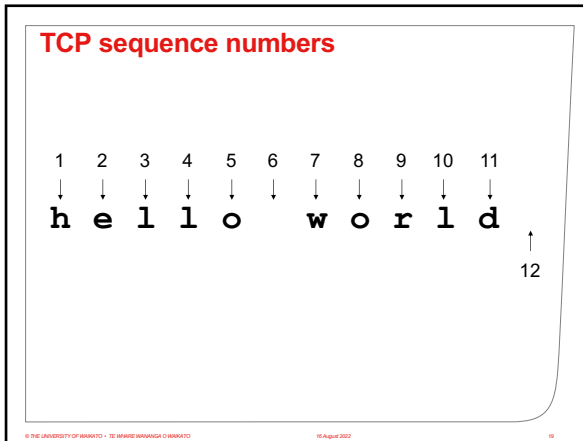
11

**TCP header: destination port**

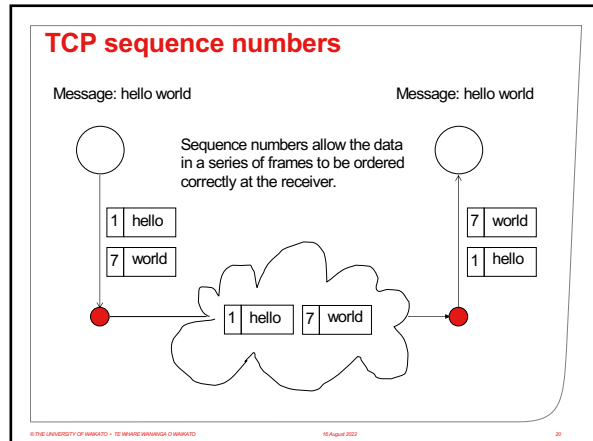


12

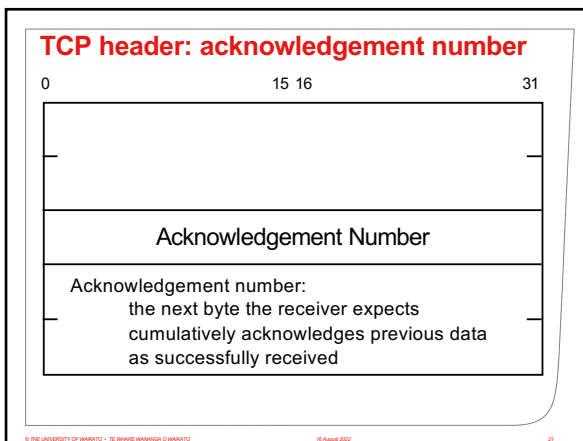




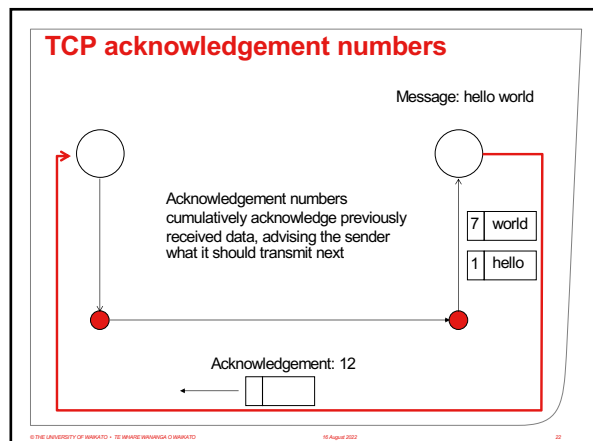
19



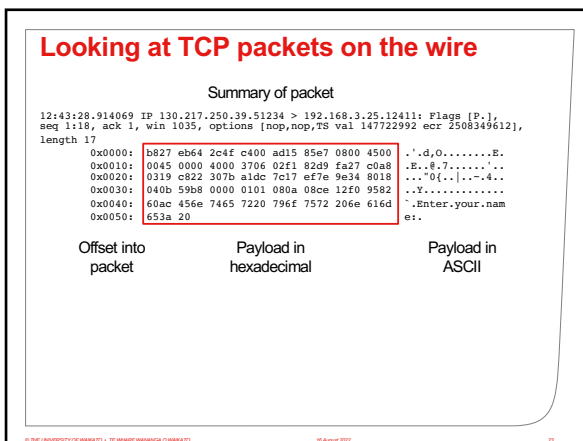
20



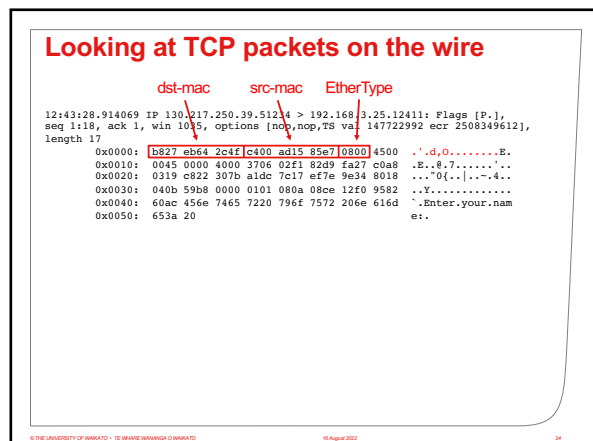
21



22



23



24

## Looking at TCP packets on the wire

```

12:43:28.914069 IP 130.217.250.39.51234 > 192.168.3.25.12411: Flags [P.],
seq 1:18, ack 1, win 1035, options [nop,nop,TS val 147722992 ecr 2508349612],
length 17
0x0000: b827 eb64 2c4f c400 ad15 85e7 0800 4500 .'.d.O.....E.
0x0010: 0045 0000 4000 3706 02f1 82d9 fa27 c0a8 ..E..8.7.....'.
0x0020: 0319 c822 307b a1dc 7c17 ef7e 9e34 8018 ...0{...-4...
0x0030: 040b 59b8 0000 0101 080a 08ce 12f0 9582 ..Y.....
0x0040: 60ac 456e 7465 7220 796f 7572 206e 616d ..Enter.your.nam
0x0050: 653a 20 e:.

```

25

## Looking at TCP packets on the wire

```

12:43:28.914069 IP 130.217.250.39.51234 > 192.168.3.25.12411: Flags [P.],
seq 1:18, ack 1, win 1035, options [nop,nop,TS val 147722992 ecr 2508349612],
length 17
0x0000: b827 eb64 2c4f c400 ad15 85e7 0800 4500 .'.d.O.....E.
0x0010: 0045 0000 4000 3706 02f1 82d9 fa27 c0a8 ..E..8.7.....'.
0x0020: 0319 c822 307b a1dc 7c17 ef7e 9e34 8018 ...0{...-4...
0x0030: 040b 59b8 0000 0101 080a 08ce 12f0 9582 ..Y.....
0x0040: 60ac 456e 7465 7220 796f 7572 206e 616d ..Enter.your.nam
0x0050: 653a 20 e:.

```

26

## Learning Objectives

- Be able to describe the key features of TCP
- Understand the relationship between the port number you provide a ServerSocket and how that is used in a TCP header
- Describe why a TCP packet has two port fields in a header
- Describe how the Sequence and Acknowledgement fields are used

27

## TCP part 1 summary

- TCP is a connection-oriented, stream-based, reliable transport protocol
- This lecture:
  - TCP packet format
  - Definition and use-of each field
- After that:
  - TCP: Establishing connections, congestion control, ...

28

# COMPX204-22B Practical Networking and Cyber Security

## Lecture 10 – TCP Part 2

Matthew Luckie  
mluckie@waikato.ac.nz



10 August 2022

1

## TCP

- Transmission control protocol (TCP)
- TCP is known as a *transport protocol* that is:
  - reliable,
  - connection-oriented,
  - stream-based
- When you use Socket / ServerSocket in Java, you use TCP as your transport protocol.
- TCP is implemented in the operating system kernel, not the application software.

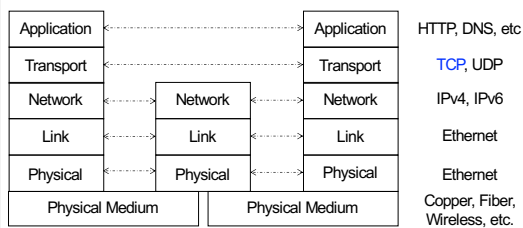
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

1 September 2015

2

2

## Protocol Layers (the Internet view)



- TCP is a *Transport Protocol*

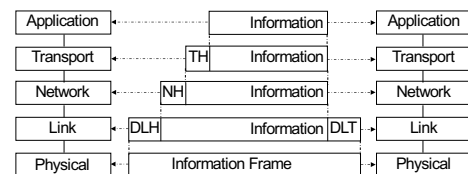
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

10 August 2022

3

3

## Encapsulation: TCP + Internet model



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

1 September 2015

4

4

## TCP features

- Connection-oriented
  - Applications establish connections between systems using TCP (port numbers)
  - No broadcast or multicast mechanism
- Full duplex
  - Both client and server can send information to each other
- Reliable
  - A sender will retransmit packets it believes are lost
  - Messages are protected using a (weak) checksum
- Network friendly
  - Sender avoids overwhelming the network's ability to forward
  - Sender tries to be fair to others using the network at the same time

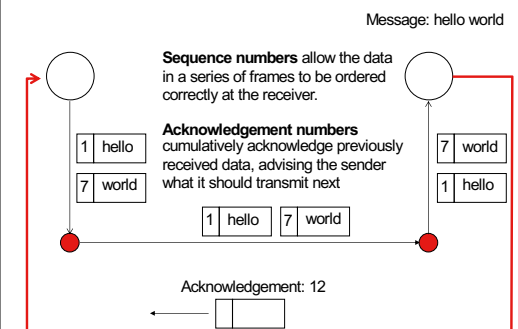
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

10 August 2022

5

5

## TCP sequence + acknowledgment nums

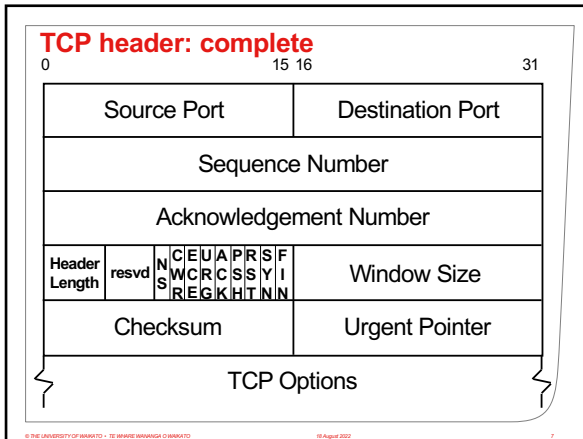


© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

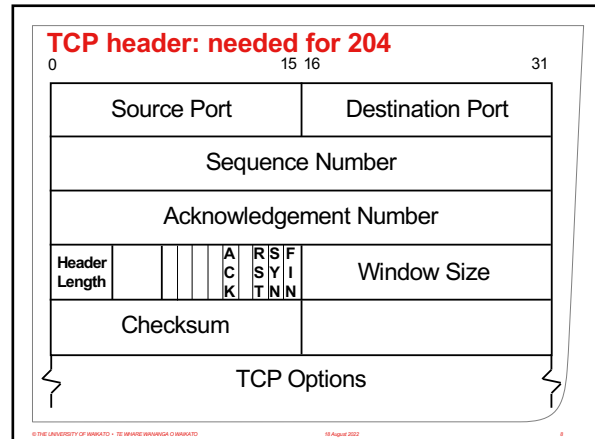
10 August 2022

6

6



7



8

### Learning Objectives

- Understand the TCP connection establishment process
  - Three-packet handshake: SYN, SYN/ACK, ACK
  - Sequence number synchronization
- Understand how TCP connections are completed
  - Three/Four packet FIN exchange
- Understand how TCP checksums are computed
  - General algorithm, common across IP-based protocols
- Understand how the TCP acknowledgment process causes a sender to decide to retransmit a lost or corrupted packet

© THE UNIVERSITY OF WARWICK - TE WARE WARWICK © WARWICK 1 September 2016 9

9

### TCP header: flags

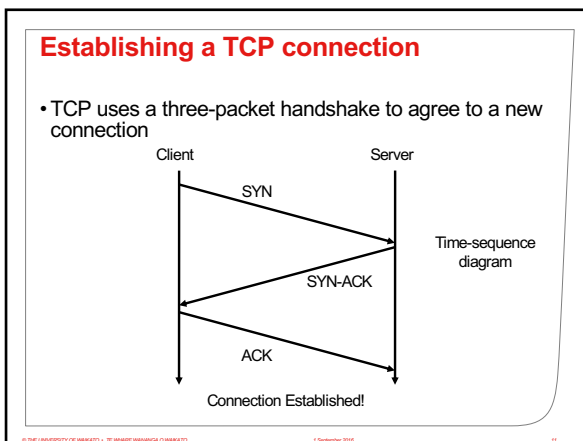
0 15 16 31

ACK: value in acknowledgement field is valid  
RST: reset connection  
SYN: synchronise sequence numbers  
used to establish new connection  
FIN: no more data from sender to transmit

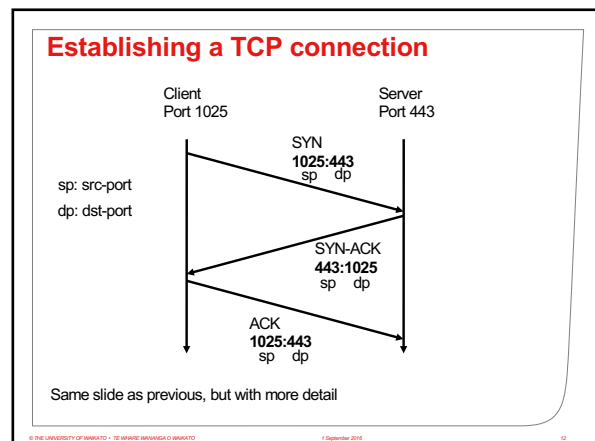
ACK RST SYN FIN

© THE UNIVERSITY OF WARWICK - TE WARE WARWICK © WARWICK 1 September 2016 10

10



11



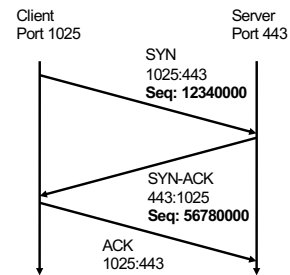
12

## Establishing a TCP connection

- Both ends of the TCP connection need to select an initial sequence number to use
- A random initial sequence number is best
  - Reduces scope for a third-party to hijack a connection by impersonating one of the two parties involved
  - If a system reboots and then happens to choose the same ports to communicate with another system, reduces chance old packets will interfere with new connection

13

## Establishing a TCP connection



Same slide as previous, but with more detail

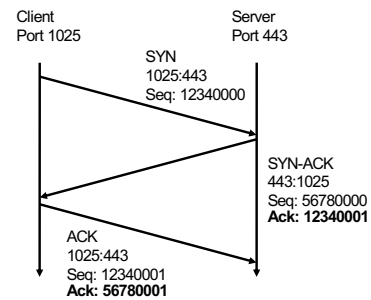
14

## Establishing a TCP connection

- TCP SYN packets are acknowledged as being received!
- The SYN packet is defined to take up one byte in the sequence number space

15

## Establishing a TCP connection



Same slide as previous, but with more detail

16

## Establishing a TCP connection (SYN)

### syn (from client)

```

10:46:40.296951 IP 192.168.3.25.58288 > 130.217.250.39.51234:
Flags [S], seq 3719351571, win 65535,
options [mss 1460,nop,wscale 6,sackOK,TS val 754452689 ecr 0],
length 0
0x0000: c400 ad15 85e7 b827 eb64 2c4f 0800 4510'.d.O..E.
0x0010: 003c 0000 4000 4006 f9e3 c0a8 0319 82d9 <...@.....
0x0020: fa27 e3b0 c822 ddb0 cd13 0000 0000 a002 '..".....
0x0030: ffff 7add 0000 0204 05b4 0103 0306 0402 ..z.....
0x0040: 080a 2cf8 08d1 0000 0000 \sequence

```

### syn-ack (from server)

```

10:46:40.312701 IP 130.217.250.39.51234 > 192.168.3.25.58288:
Flags [S.], seq 2996965380, ack 3719351572, win 65535,
options [mss 1440,nop,wscale 6,sackOK,TS val 786419391 ecr 754452689],
length 0
0x0000: b827 eb64 2c4f c400 ad15 85e7 0800 4500'.d.O.....E.
0x0010: 003c 0000 4000 3706 02fa 82d9 fa27 c0a8 <...@.7.....'..
0x0020: 0319 c822 e3b0 b2a2 1004 ddb0 cd14 a012 '..".....
0x0030: ffff ba9a 0000 0204 05a0 0103 0306 0402 ..z.....
0x0040: 080a 2edf cebf 2cf8 08d1 \acknowledgement

```

Note: acknowledgement of SYN is sequence + 1

17

## Establishing a TCP connection (SYN)

### syn-ack (from server)

```

10:46:40.312701 IP 130.217.250.39.51234 > 192.168.3.25.58288: Flags
[S.], seq 2996965380, ack 3719351572, win 65535,
options [mss 1440,nop,wscale 6,sackOK,TS val 786419391 ecr 754452689],
length 0
0x0000: b827 eb64 2c4f c400 ad15 85e7 0800 4500'.d.O.....E.
0x0010: 003c 0000 4000 3706 02fa 82d9 fa27 c0a8 <...@.7.....'..
0x0020: 0319 c822 e3b0 b2a2 1004 ddb0 cd14 a012 '..".....
0x0030: ffff ba9a 0000 0204 05a0 0103 0306 0402 ..z.....
0x0040: 080a 2edf cebf 2cf8 08d1 \sequence

```

### ack (from client)

```

10:46:40.312820 IP 192.168.3.25.58288 > 130.217.250.39.51234: Flags
[.], ack 1, win 1035,
options [nop,nop,TS val 754452705 ecr 786419391],
length 0
0x0000: c400 ad15 85e7 b827 eb64 2c4f 0800 4510'.d.O.....E.
0x0010: 003c 0000 4000 4006 f9f1 c0a8 0319 82d9 <...@.7.....'..
0x0020: fa27 e3b0 c822 ddb0 cd14 b2a2 1005 8010 '..".....
0x0030: 040b e536 0000 0101 080a 2cf8 08e1 2edf ..6.....
0x0040: cebf \acknowledgement

```

Note: acknowledgement of SYN is sequence + 1  
 $0xb2a21004 + 1 = 0xb2a21005$

18

## Acknowledgement of data

### data (from server)

```
10:46:40.332237 IP 130.217.250.39.51234 > 192.168.3.25.58288: Flags [P.],
seq 1118, ack 1, win 1035, options [nop,nop,TS val 786419410 ecr
754452705], length 17
0x0000: b827 eb64 2c4f c400 ad15 85e7 0800 4500 .'.d.O.....E.
0x0010: 0045 0000 4000 3706 02f1 82d9 fa27 c0a8 .E..ê.7.....'..
0x0020: 0319 c822 e3b0 b2a2 1005 ddb0 cd14 8018 ..."......
0x0030: 040b c31e 0000 0101 080a 2edf ced2 2cf8
0x0040: 08e1 456e 7465 7220 796f 7572 206e 616d ..Enter.your.nam
0x0050: 653a 20 e:..
sequence
```

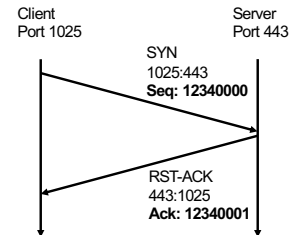
### ack (from client)

```
10:46:40.371773 IP 192.168.3.25.58288 > 130.217.250.39.51234: Flags [.],
ack 18, win 1035, options [nop,nop,TS val 754452764 ecr 786419410],
length 0
0x0000: c400 ad15 85e7 b827 eb64 2c4f 0800 4510 '.d.O..E.
0x0010: 0034 0000 4000 4006 f9f1 cba8 0319 82d9 .4..ê.....
0x0020: fa27 e3b0 c822 ddb0 cd14 b2a2 1016 8010
0x0030: 040b e4d7 0000 0101 080a 2cf8 091c 2edf
0x0040: ced2
acknowledgement
```

Note: acknowledgement of this data is sequence + 17  
 $0xb2a21005 + 17 (0x11) = 0xb2a21016$

19

## TCP "connection refused"



- The port is not open on the destination system
- The port is open, but backlog of pending connections
- A firewall between the client + server is blocking access

20

## TCP is a reliable transport protocol

- If a packet is lost or corrupted, TCP will try sending it again
  - Detecting corruption:
    - checksum computation did not match
  - Detecting loss: multiple methods
    - Simplest: no acknowledgement received before timeout
    - More common: multiple duplicate acknowledgements
- Note: a TCP receiver does not explicitly tell the sender it is missing a packet, or that one was corrupted
  - The sender infers this when an acknowledgement is not received before a time-out occurs.

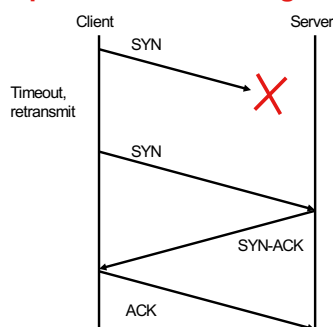
21

## Why not tell the sender when a packet is lost?

- TCP assumes if a packet is lost, the network was unable to forward it, due to congestion
- Congestion occurs when packets cannot be forwarded at the rate they are arriving at.
- Loss implying congestion is not entirely true, some mediums are more likely to have packets corrupted
  - e.g., wireless.
- Theory goes that sending a new packet into the network to report a lost packet only adds to the congestion problem
  - This is not entirely true

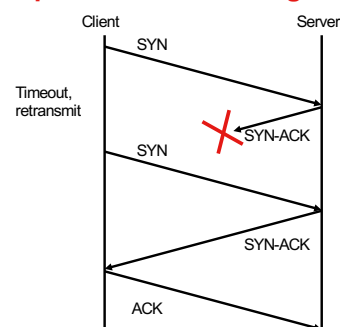
22

## Example loss: Establishing a connection



23

## Example loss: Establishing a connection



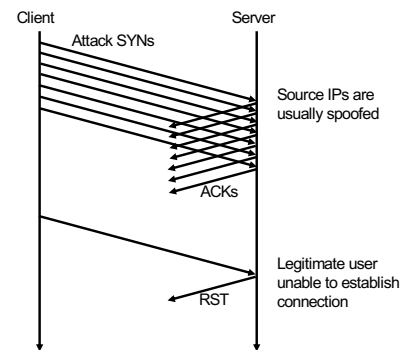
24

### Denial of Service: SYN flood

- Each time a SYN probe is received, the receiver must
  - Check if there is a socket listening for new connections
  - If there is, it must search a table of existing connections to see if there is already a connection established
    - 4-tuple: source-IP, destination-IP, source-port, destination-port
  - If receiver does not find a record of an existing connection, it adds a new connection record to the table and sends a SYN-ACK in reply
- Simple TCP denial of service attack:
  - Send many SYN packets with randomised source port values and spoofed (randomised) source addresses.
  - Receiver must
    - Search a table of existing connections (consumes CPU time)
    - allocate a record (consumes memory)

25

### Denial of Service: SYN flood



26

### Denial of Service: SYN flood

- Note: this technique not so useful these days, most operating systems have some defence mechanism
- <https://www.cloudflare.com/learning/ddos/syn-flood-ddos-attack/>

27

### Detecting Corruption: Internet checksum

- Checksums in most Internet protocols are simple to compute but relatively weak.
- Core computation: sum of 16-bit words.

|   |   |   |   |  |        |        |
|---|---|---|---|--|--------|--------|
| h | e | l | l |  | 0x6865 | 0x6c6c |
| o |   | w | o |  | 0x6f20 | 0x776f |
| r | l | d |   |  | 0x726c | 0x64   |

$$\begin{aligned}
 \text{Sum of 16-bit words} &= 0x6865 + 0x6c6c + 0x6f20 + 0x776f + 0x726c + 0x6400 \\
 &= 0x00022e30
 \end{aligned}$$

28

### Internet checksum calculation

$$\begin{aligned}
 \text{Sum of 16-bit words} &= 0x6865 + 0x6c6c + 0x6f20 + 0x776f + 0x726c + 0x6400 \\
 &= 0x00022e30
 \end{aligned}$$

To obtain 16-bit Internet checksum, we fold the 32-bit value into a 16-bit one as follows:

$$0x0002 + 0x2e30 = 0x2e32$$

Then negate:

$$\begin{aligned}
 0x2e32 &= 0010\ 1110\ 0011\ 0010 \\
 \text{Negated} &= 1101\ 0001\ 1100\ 1101 \Rightarrow 0xd1cd
 \end{aligned}$$

29

### Internet checksum: word swapping

|   |   |   |   |  |        |        |
|---|---|---|---|--|--------|--------|
| h | e | l | l |  | 0x6865 | 0x6c6c |
| o |   | w | o |  | 0x6f20 | 0x776f |
| r | l | d |   |  | 0x726c | 0x64   |

Checksum: 0xd1cd

First 4 characters swapped in transit:

|   |   |   |   |  |        |        |
|---|---|---|---|--|--------|--------|
| l | l | h | e |  | 0x6c6c | 0x6865 |
| o |   | w | o |  | 0x6f20 | 0x776f |
| r | l | d |   |  | 0x726c | 0x64   |

Corruption not detected!

Checksum: 0xd1cd

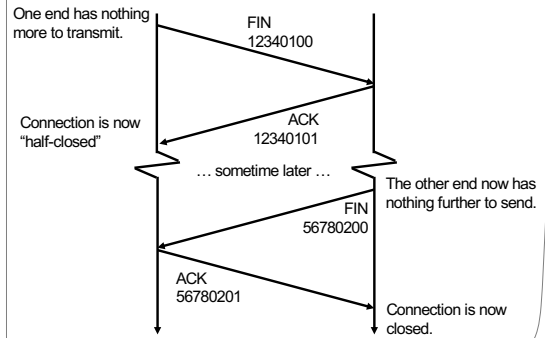
30

## Closing a connection

- Closing a connection in TCP is done in two steps
  - One host A tells the other host B it has nothing more to send
    - (is **finished**)
  - Connection is now in *half-closed* state
- Eventually, the host B tells host A it too has finished.
- Connection is now finished.

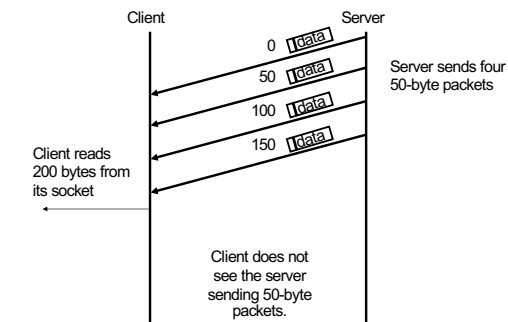
31

## Closing a connection



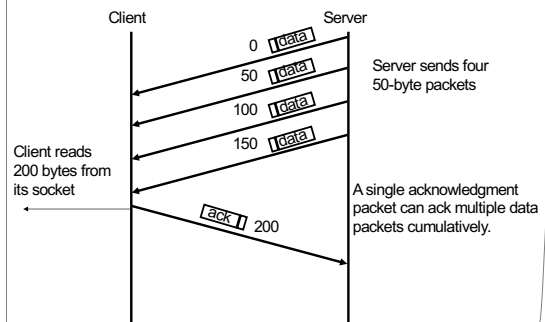
32

## TCP is a stream-based protocol



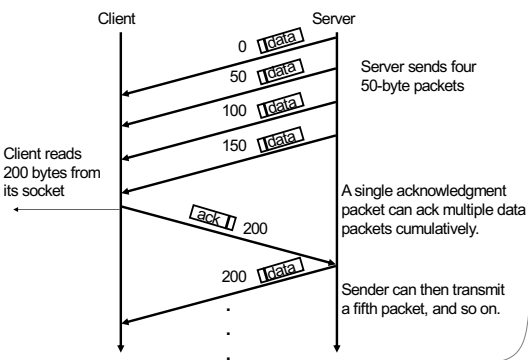
33

## TCP is a reliable, stream-based protocol



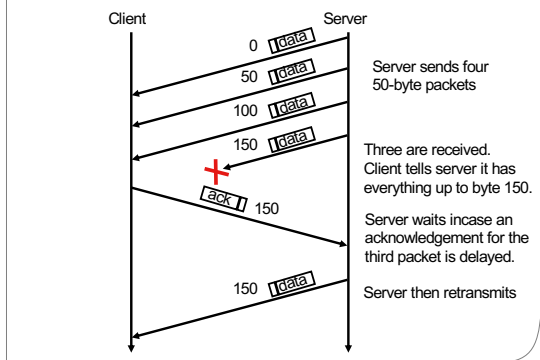
34

## TCP is a reliable, stream-based protocol



35

## TCP is a reliable, stream-based protocol



36

## Implementation of a reliable transport

- Requires sender to keep a copy of the data that the receiver has not yet acknowledged.
  - Operating system keeps data in a buffer in the kernel
- Requires sender to decide when data should be declared lost and retransmitted
  - Operating system keeps track of how long it took previous data to be acknowledged

37

## Learning Objectives

- Understand the TCP connection establishment process
  - Three-packet handshake: SYN, SYN/ACK, ACK
  - Sequence number synchronization
- Understand how TCP connections are completed
  - Three/Four packet FIN exchange
- Understand how TCP checksums are computed
  - General algorithm, common across IP-based protocols
- Understand how the TCP acknowledgment process causes a sender to decide to retransmit a lost or corrupted packet

38

## Summary

- TCP is a *transport protocol* that is:
- Reliable
  - Unacknowledged packets are retransmitted after they are lost and/or time out
- connection-oriented,
  - Hosts establish connection with 3-packet handshake
  - tear-down connections with 4-packet exchange
- stream-based
  - Client socket sees a stream of bytes, like reading or writing a file.
  - Does not see individual packets, or retransmitted packets, or connection establishment/teardown packets.

39

## Next lecture

- TCP fairness
  - Slow-start
  - Retransmission timer
  - Aspects of congestion control

40

# COMPX204-22B Practical Networking and Cyber Security

## Lecture 11 – TCP Part 3

Matthew Luckie  
mluckie@waikato.ac.nz



23 August 2022

1

## Learning Objectives

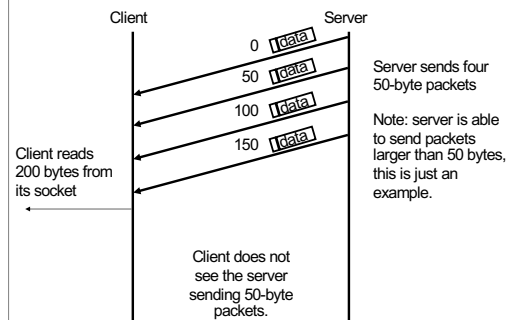
- Understand how TCP decides how long to wait before retransmitting a packet it thinks has been lost
- Understand how TCP decides what the appropriate rate to transmit is
- Understand why duplicate acknowledgements can suggest a packet has been lost
- Be able to describe the difference between fast retransmit and fast recovery

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

23 August 2022

2

## TCP is a reliable, stream-based protocol



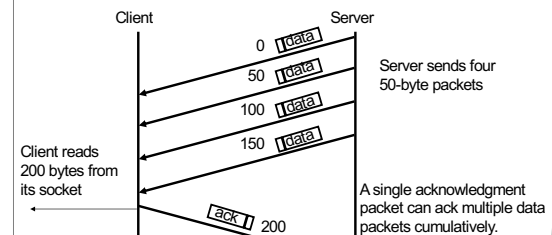
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

1 September 2018

3

3

## TCP is a reliable, stream-based protocol



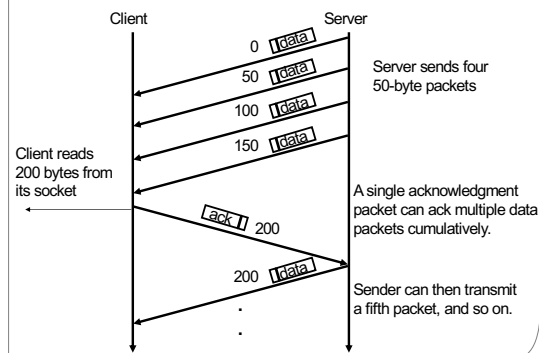
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

1 September 2018

4

4

## TCP is a reliable, stream-based protocol



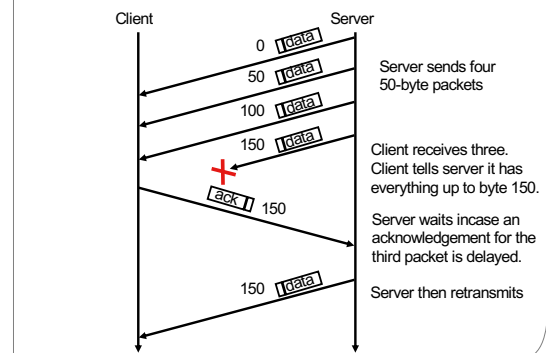
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

1 September 2018

5

5

## TCP is a reliable, stream-based protocol



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

1 September 2018

6

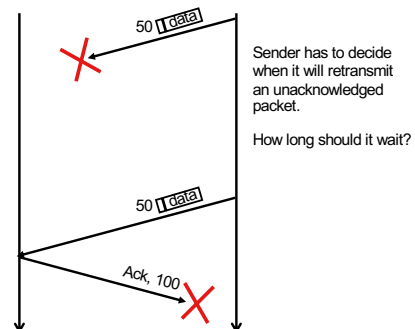
6

## Implementation of a reliable transport

- Requires sender to keep a copy of the data that the receiver has not yet acknowledged.
  - Operating system keeps data in a buffer in the kernel
- Requires sender to decide when data should be declared lost and retransmitted
  - Operating system keeps track of how long it took previous data to be acknowledged

7

## Dealing with lost packets



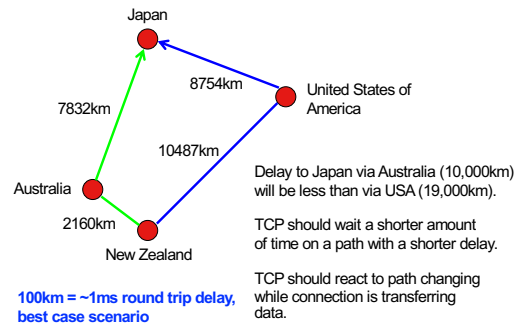
8

## How long should a sender wait?

- Intuitively, it should wait roughly the amount of time earlier packets required to be successfully acknowledged
- Challenge: network delays are variable
  - Delay varies based on how busy (congested) the network is
  - Delay varies if the path through the network changes
- Goals:
  - React to delay characteristics changing
  - Avoid retransmitting a packet that has been delayed more than usual but will be acknowledged in due course
    - I.e., avoid requiring the network to do useless work

9

## Dealing with different delays



10

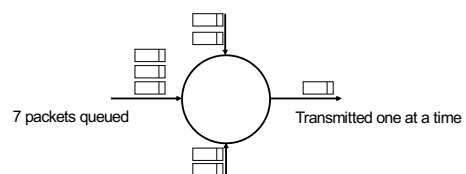
## Estimating delay (Round Trip Time)

- Round Trip Time: RTT
- Estimated by averaging a sequence of measured RTTs with exponential smoothing --- Smoothed RTT (SRTT)
- $$SRTT(k) = \underbrace{g * \text{MeasuredRTT}(k)}_{\text{Most recent measurement.}} + \underbrace{(1-g) * SRTT(k-1)}_{\text{Prior estimations.}}$$
- Weighting is used as most recent measurements are considered most important
  - Want to react to path changes, but not too quickly
  - Normally,  $g = 0.125$

11

## Delay variation

- Recall that the delay experienced will change over time depending on congestion levels
  - Congestion: how much time a packet has to wait at a router to be forwarded
  - Similar in a sense to traffic congestion: how much time you have to wait at the lights.



12

## Impact of Delay Variation

- A path between two hosts may have significant variation
  - even if the path does not change
  - A smoothed RTT is an estimation of the recent RTT and does not take account of variation
- Need to wait long enough to ensure packet was lost rather than delayed a long time

13

## Estimating Delay Variation

- $RTT_{var}(k) = RTT(k) - SRTT(k-1)$ 
  - RTT variation is the difference between the current RTT measured and the previous estimate of the average
- $SRTT_{var}(k) = h * RTT_{var}(k) + (1-h) * SRTT_{var}(k-1)$ 
  - Most recent measurement.
  - Prior estimations.
- Estimate of the variation is smoothed over time like the SRTT calculation so the most recent variance figure is the most important
  - Normally  $h = 0.25$

14

## Putting it all together

- So far, we've produced equations to estimate the RTT (SRTT) and RTT variance ( $SRTT_{var}$ ) of a TCP connection
- Put them together to get a TCP retransmit timer (RTO)
- $RTO(k) = SRTT(k) + f * SRTT_{var}(k)$
- Normally  $f=4$ 
  - High probability that the packet will be acknowledged before the RTO expires

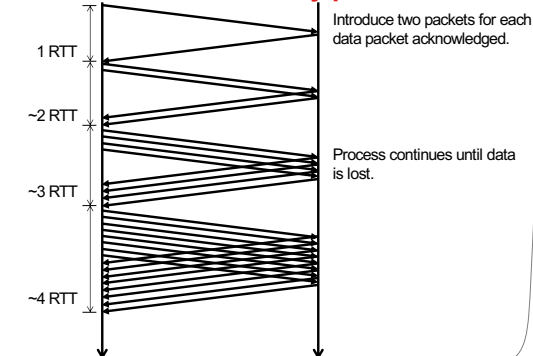
15

## TCP slow start

- When a TCP connection is established, the hosts have no reliable measure of available capacity
- The pair of hosts need to figure out the rate at which they can each send data
- Solution: TCP slow-start

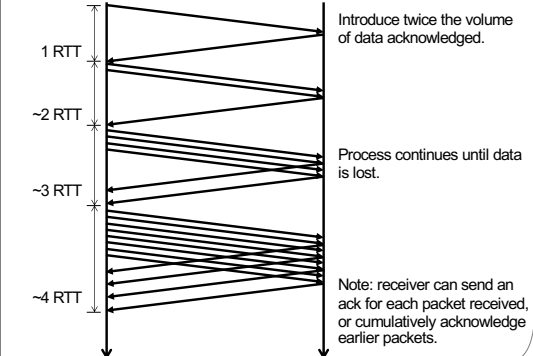
16

## TCP slow start: ack every packet



17

## TCP slow start: cumulative ack

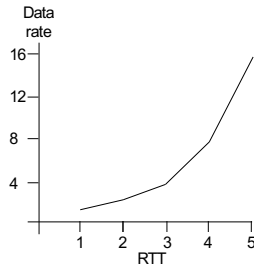


18

## TCP slow start

- Despite its name, TCP slow-start results in an exponential increase in transmission rate

- 1 RTT: 1 data unit
- 2 RTT: 2 data units
- 3 RTT: 4 data units
- 4 RTT: 8 data units
- 5 RTT: 16 data units
- and so on



Note: modern TCP implementations start with 10 packets

19

## TCP congestion avoidance

- Increasing transmission rate at an exponential rate will mean you should find the available capacity fairly rapidly
- When you reach the available capacity, data will start to be lost
- In order to be fair, TCP implementations reduce the rate at which new data is introduced into the network when congestion is encountered

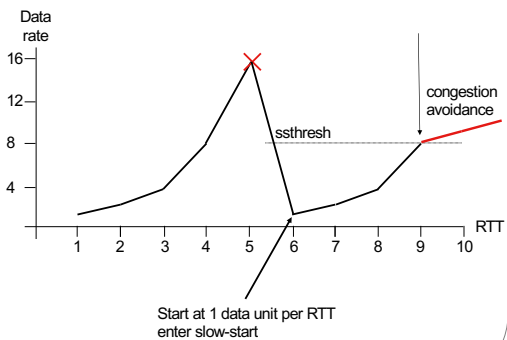
20

## TCP congestion avoidance

- Congestion window (*cwnd*): the amount of data a sender can transmit without waiting for an acknowledgment
- TCP congestion avoidance: increase rate of transmission by one data unit per RTT.
- When congestion is encountered, halve the current value of *cwnd* and store it in a variable named "slow start threshold" (*ssthresh*)
- ssthresh*: rate at which congestion avoidance begins

21

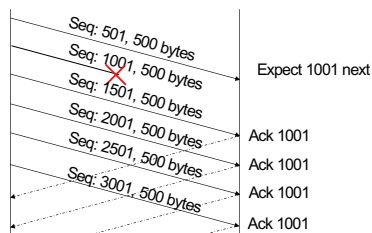
## TCP congestion avoidance: timeout



22

## Duplicate acks imply packet loss

- A receiver has a fairly good idea that a segment has been lost by the receipt of the third duplicate acknowledgement, asking for data beginning with byte 1001.

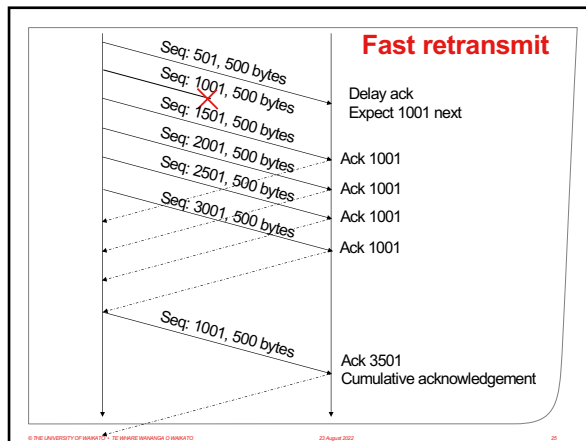


23

## Retransmission schemes

- TCP retransmits when the RTO expires
  - i.e., the absence of an acknowledgement is used to infer packet loss
- Fast retransmit:** on the third duplicate ack, retransmit the assumed missing segment
  - i.e., infer packet loss by an apparent hole in the receiver's window
  - the fact that we are receiving duplicate acknowledgements means data is still being received, so **the congestion event was not severe**
    - Don't want to go into slow-start again, just want to avoid congestion
    - Have more confidence that packet was lost, rather than just reordered

24

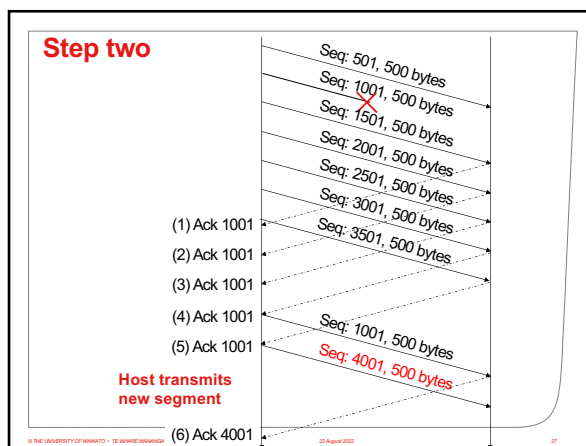


25

### Fast Retransmit + Fast Recovery

- **Step One:**
  - When third dup-ack is received, set *ssthresh* to half of the current value of *cwnd* or half of the receiver's window, whichever is smaller
  - Retransmit the segment assumed to be missing
  - Set *cwnd* to *ssthresh* plus 3 times the segment size
- **Step Two:**
  - Each time another dup-ack is received, increment *cwnd* by the segment size and transmit a new segment, if allowed by the value of *cwnd*
  - This part is known as TCP NewReno Fast Recovery

26

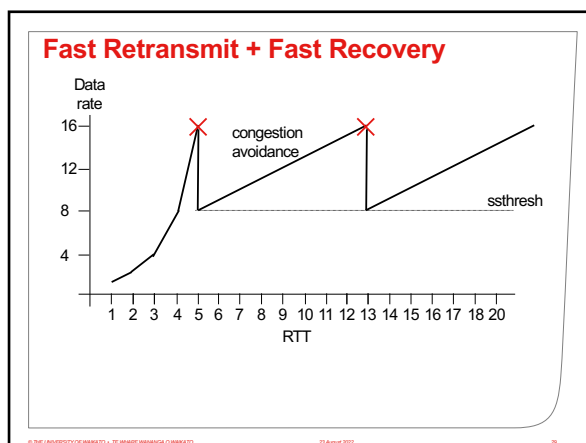


27

### Fast Retransmit + Fast Recovery

- **Step three**
  - When an acknowledgement that is asking for new data arrives, set *cwnd* to *ssthresh* (the value stored when we entered fast-retransmit).
  - Enter congestion avoidance

28



29

### Summary

- TCP is the most widely used transport protocol in the Internet: reliable, byte-stream, connection oriented, and FAIR!
- TCP has to work on paths that vary with length (delay) and available capacity
- TCP retransmit timer: decide when to declare a packet lost
- Slow-start: quickly infer the rate at which TCP should be transmitting at
- Congestion avoidance: slowly increase the rate at which new data is sent to avoid causing congestion
- Fast retransmit / fast recovery: recover quickly from single packet loss

30

## Learning Objectives

- Understand how TCP decides how long to wait before retransmitting a packet it thinks has been lost
- Understand how TCP decides what the appropriate rate to transmit is
- Understand why duplicate acknowledgements can suggest a packet has been lost
- Be able to describe the difference between fast retransmit and fast recovery

# COMPX204-22B Practical Networking and Cybersecurity

## Lecture 12 Network Security – Part 1

Matthew Luckie  
mluckie@waikato.ac.nz



20 September 2022

1

## Network Security

- Computer networks and network protocols are designed to make sharing information and resources easy
  - Sharing printers, sending email, fetching files, making phone calls
  - Simple protocols succeed / complex protocols fail
- Vectors for an adversary to
  - break into a computer system
  - access information they should not be able to
  - impersonate third parties
  - tamper with a communication

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

2

2

## This lecture

- Encryption
- Digital Signatures
- Authentication
- Firewalls

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

3

3

## Learning Objectives

- Understand the role of public and private keys in asymmetric encryption
- Understand what a "hash" is
- Understand how a private key and a hash can be used to produce a digital signature, and how the signature can be verified with the public key
- Understand how two parties can verify a shared secret without disclosing that secret
- Understand the role of a network firewall and the types of decisions it might make

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

4

4

## Everyday Encryption



All of these are methods to arrange encryption on a WiFi network (except none)

HTTPS provides authentication and encryption

<https://learn.waikato.ac.nz/course/view.php?id=60145>



FileVault secures the data on your disk by encrypting its contents automatically.

Turn Off FileVault...

**WARNING:** You will need your login password or a recovery key to access your data. A recovery key is automatically generated as part of this setup. If you forget both your password and recovery key, the data will be lost.

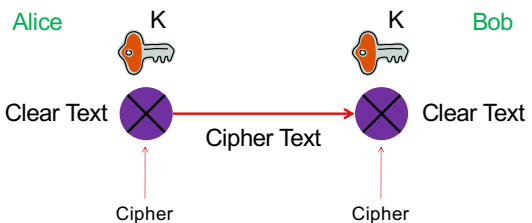
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

5

5

## Symmetric Encryption



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

6

6

## Symmetric Encryption

- Both parties have a secret shared key – say 128 bits in size
- Fast!
- Ciphers and Cipher Text are public information
- Cipher algorithm might be different for encryption than for decryption
- If key is revealed, then all information sent is compromised
  - Need to have a secure method to distribute keys

© THE UNIVERSITY OF BRISTOL - 10 BRISTOL BRUNNEN CHARTERED

20 September 2022

7

## Symmetric Encryption: Caesar Cipher

- Earliest known *Substitution Cipher*
- Encryption Cipher: Right Shift
- Decryption Cipher: Left Shift
- Secret Key: number of letters to shift alphabet

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z |
| ↓ |   |   |   |   |   |   |   |   |   |   |   |   | ↓ |   |   |   |   |   |   |   |   |   |   |   |   |
| C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B |

Encrypted Text: JGNNQYQTNF  
What is the plain text?

© THE UNIVERSITY OF BRISTOL - 10 BRISTOL BRUNNEN CHARTERED

20 September 2022

8

## Symmetric Encryption: Rail Fence Cipher

- *Transposition Cipher*
- Cipher: rearrange letters without altering their values
- Secret Key: number of rows

HELLO WORLD  
↓  
H        W  
E        O  
L        R  
O        D  
L        L  
↓  
HWE OLORDLL

© THE UNIVERSITY OF BRISTOL - 10 BRISTOL BRUNNEN CHARTERED

20 September 2022

9

## Product Ciphers

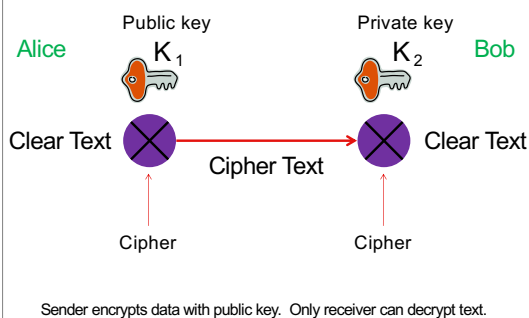
- Combinations of Transposition and Substitution Ciphers and more
- E.g. Advanced Encryption Standard (AES)

© THE UNIVERSITY OF BRISTOL - 10 BRISTOL BRUNNEN CHARTERED

20 September 2022

10

## Asymmetric Encryption



© THE UNIVERSITY OF BRISTOL - 10 BRISTOL BRUNNEN CHARTERED

20 September 2022

11

## Asymmetric Encryption

- Sender has key pair:
  - private key: never revealed
  - public key: given freely, very large – say 2048 bits in size
- Slower than symmetric encryption
- Public key is related to private key but difficult to reverse engineer because of its size
- RSA: 1978 algorithm in wide use today. Very strong.

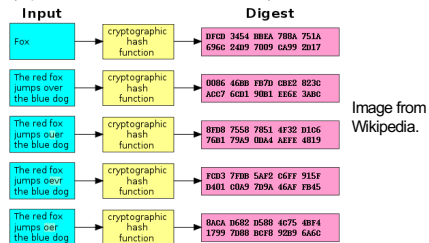
© THE UNIVERSITY OF BRISTOL - 10 BRISTOL BRUNNEN CHARTERED

20 September 2022

12

## One-way Hashes

- Generate fixed length “checksum” over data
- Impossible to derive data from hash
- Should be difficult, short of a brute-force search, to find a collision (a piece of data with the same hash)



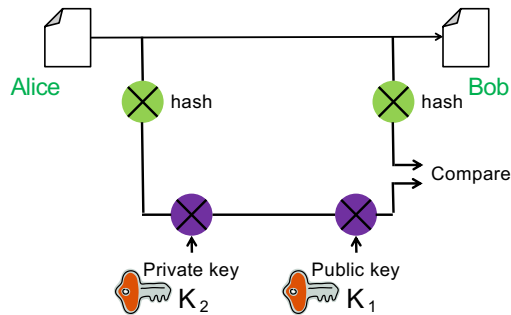
13

## One-way Hashes

- MD5 (scamper-cvs-20211212b.tar.gz) =  
ba7e5e2cd63b7531e6b4d6a3e0579427
  - 128 bits. Collisions in  $2^{21}$ . Broken. Seconds on a modern PC.
- SHA1 (scamper-cvs-20211212b.tar.gz) =  
60614ca2ffd248ee4f880312fa94fff9fc22415c
  - SHA-1 160 bits. Collisions in  $2^{51}$ . Weak
- SHA256 (scamper-cvs-20211212b.tar.gz) =  
fee949ff7e0312ba672944fe2084aed75a8979b778c33adba26ffc522a1346d8
  - SHA-2 (224/256/384/512) bits. Recommended for now.

14

## Digital Signatures



15

## Digital Signatures: Applications

- Signing email (PGP / GPG)
  - Prove that the email's sender sent the email as claimed
- DNS responses (DNSSEC)
  - Prove that the answer received in DNS is authentic and comes from the authoritative source rather than an intermediary
- BGP route advertisements
  - Prove that the network announcing a prefix is allowed to do so (is not hijacking address space)
  - Prove that the path taken by the prefix from announcer to receiver is authentic

16

## Authentication

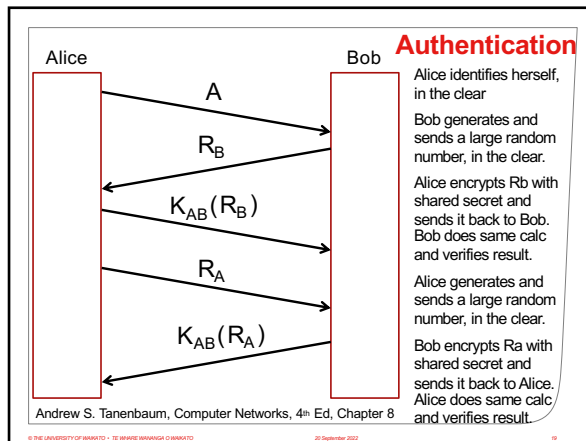
- Most computer security is derived from authorised parties having a shared secret
  - Passwords to login to computers
  - Passphrases to unlock sensitive data
  - WPA keys to use on wireless networks

17

## Authentication

- How do we authenticate two parties across an untrusted network, where someone could be watching?
  - Don't want to reveal our password!
- Most authentication relies on a shared secret
  - Challenge-response authentication
  - Both parties know the key but do not reveal it to each other
- There are many authentication protocols built around shared secret keys.

18



19

### Authentication

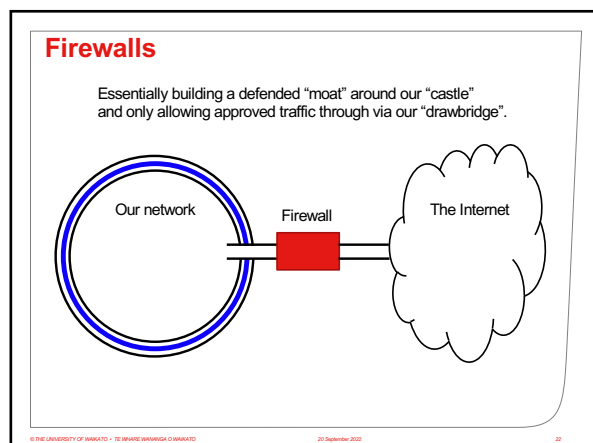
- At end of process, Alice and Bob have verified each other's identity by checking each have the secret key
- Usually what happens next is a session key is encrypted using the shared key and passed to the other party.

20

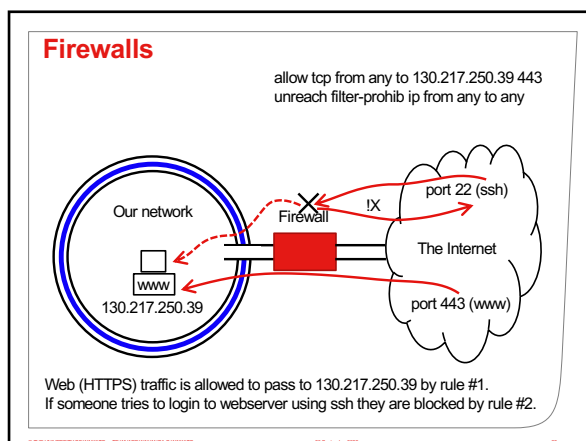
### Firewalls

- A "firewall" is a type of router that passes select traffic through
- Placement of firewall is critical: should be in path where all traffic in and out of a network is forced
- Each packet is compared against a set of rules that decide if it is allowed or not
- Typical rules examine:
  - Interface where packet arrived.
  - IP header: addresses, protocol.
  - TCP, UDP headers: ports, TCP flags
  - Stateful firewalls: is this packet part of a known connection/flow?
- Default policy is often to deny entry. Conservative.

21



22



23

### Summary

- Encryption allows us to protect sensitive data traversing untrusted networks
- One-way hashes and public key encryption allow us to build digital signatures
- Authentication systems allow us to provide our identity across untrusted networks
- Firewalls allow us to police traffic entering and exiting a network

24

## Learning Objectives

- Understand the role of public and private keys in asymmetric encryption
- Understand what a “hash” is
- Understand how a private key and a hash can be used to produce a digital signature, and how the signature can be verified with the public key
- Understand how two parties can verify a shared secret without disclosing that secret
- Understand the role of a network firewall and the types of decisions it might make

© THE UNIVERSITY OF MANKATO • 10 WISCONSIN WISCONSIN • WISCONSIN

20 September 2022

25

# COMPX204-22B Practical Networking and Cybersecurity

Lecture 13  
Network Security – Part 2

Matthew Luckie  
mluckie@waikato.ac.nz



22 September 2022

1

## Learning Objectives

- Understand how a private key and a hash can be used to produce a digital signature, and how the signature can be verified with the public key
- Understand how two parties can verify a shared secret without disclosing that secret
- Understand the role of a network firewall and the types of decisions it might make

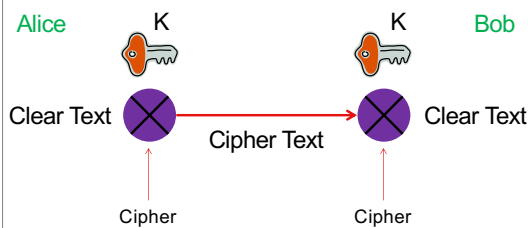
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

2

2

## Symmetric Encryption



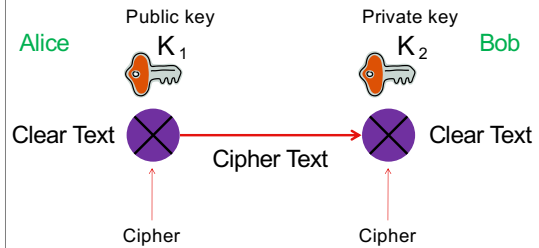
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

3

3

## Asymmetric Encryption



Sender encrypts data with public key. Only receiver can decrypt text.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

4

4

## One-way Hashes

- Generate fixed length "checksum" over data
  - Impossible to derive data from hash
  - Should be difficult, short of a brute-force search, to find a collision (a piece of data with the same hash)

| Input                               |                             | Digest                                               |
|-------------------------------------|-----------------------------|------------------------------------------------------|
| Fox                                 | cryptographic hash function | 0FED 3454 0BBA 78BA 751A<br>676C 24D7 70D9 6A59 2B17 |
| The red fox jumps over the blue dog | cryptographic hash function | 0006 4636 1B7D 0B25 023C<br>AC27 60B1 9B81 1B26 3A8C |
| The red fox jumps over the blue dog | cryptographic hash function | 0700 7558 7851 4F30 0106<br>76B1 79A9 0BA4 AEF5 4B19 |
| The red fox jumps over the blue dog | cryptographic hash function | F0B3 770B 5A82 06FF 915F<br>D401 6DA7 7D9A 46AF 7B45 |
| The red fox jumps over the blue dog | cryptographic hash function | 8ACA 0682 0588 4075 4BF4<br>1799 7088 BCF8 92B9 6A6C |

Image from Wikipedia.

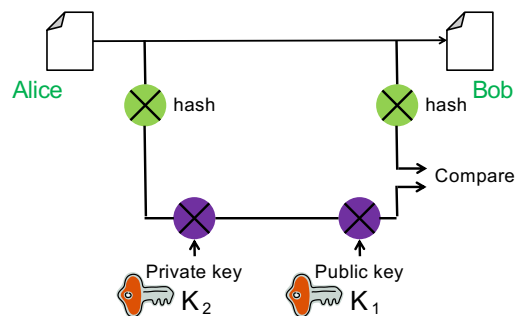
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

5

5

## Digital Signatures



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

6

6

## Digital Signatures: Applications

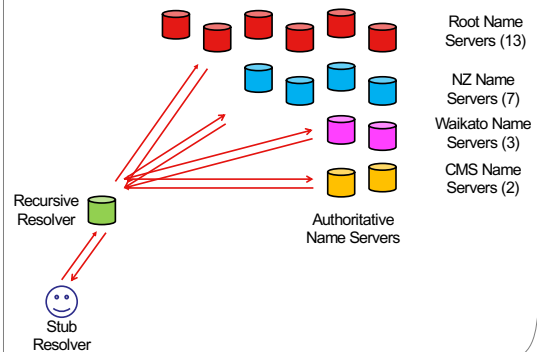
- Signing email (PGP / GPG)
  - Prove that the email's sender sent the email as claimed
- DNS responses (DNSSEC)
  - Prove that the answer received in DNS is authentic and comes from the authoritative source rather than an intermediary
- BGP route advertisements
  - Prove that the network announcing a prefix is allowed to do so (is not hijacking address space)
  - Prove that the path taken by the prefix from announcer to receiver is authentic

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

7

## Domain Name System Distributed Hierarchical Database

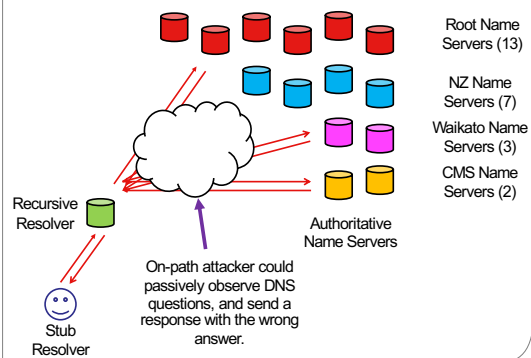


© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

8

## Domain Name System Distributed Hierarchical Database

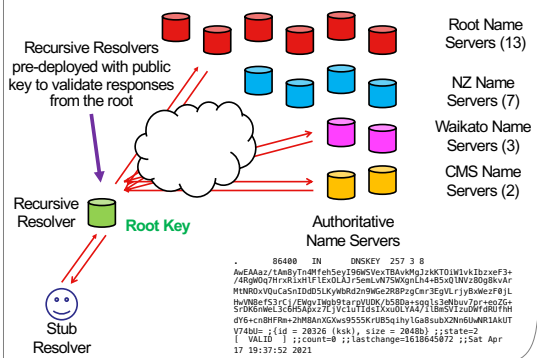


© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

9

## DNSSEC Chain of Trust

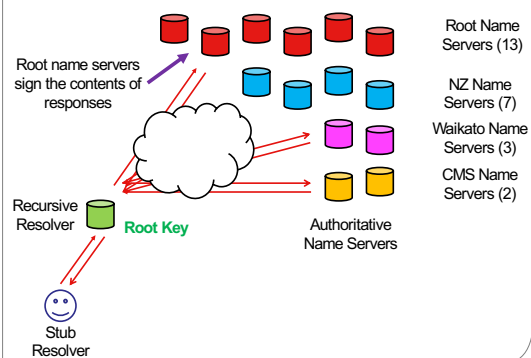


© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

10

## DNSSEC Chain of Trust

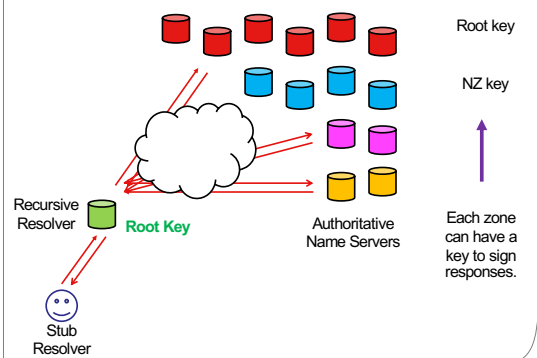


© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

11

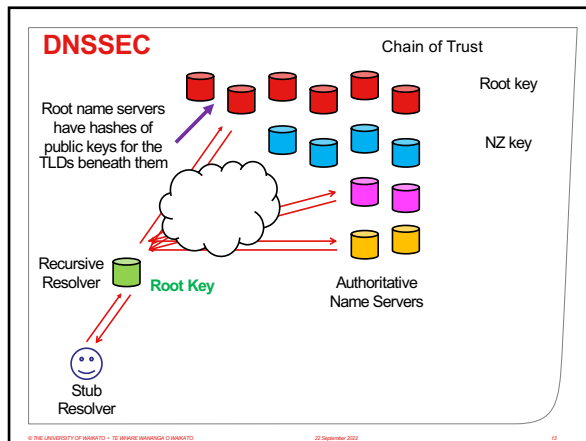
## DNSSEC Chain of Trust



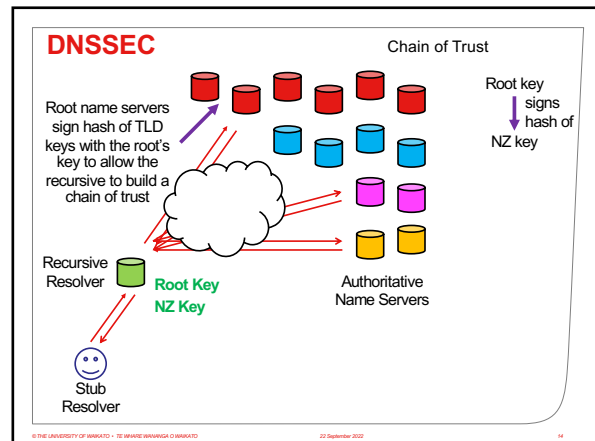
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

22 September 2022

12



13



14

### Digital Signatures: Applications

- Signing email (PGP / GPG)
  - Prove that the email's sender sent the email as claimed
- DNS responses (DNSSEC)
  - Prove that the answer received in DNS is authentic and comes from the authoritative source rather than an intermediary
- BGP route advertisements
  - Prove that the network announcing a prefix is allowed to do so (is not hijacking address space)
  - Prove that the path taken by the prefix from announcer to receiver is authentic

15

### Authentication

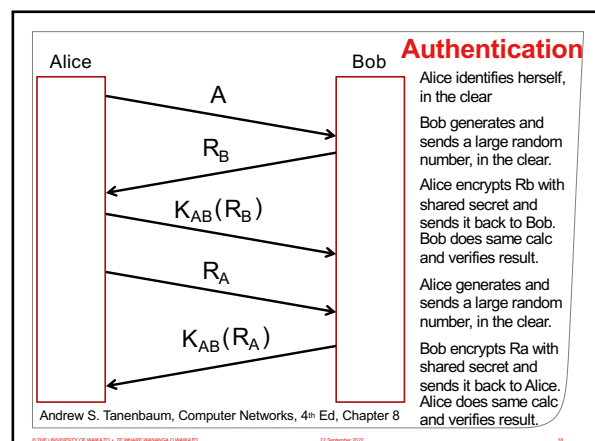
- Most computer security is derived from authorised parties having a shared secret
  - Passwords to login to computers
  - Passphrases to unlock sensitive data
  - WPA keys to use on wireless networks

16

### Authentication

- How do we authenticate two parties across an untrusted network, where someone could be watching?
  - Don't want to reveal our password!
- Most authentication relies on a shared secret
  - Challenge-response authentication
  - Both parties know the key but do not reveal it to each other
- There are many authentication protocols built around shared secret keys.

17



18

## Authentication

- At end of process, Alice and Bob have verified each other's identity by checking each have the secret key
- Usually what happens next is a session key is encrypted using the shared key and passed to the other party.

19

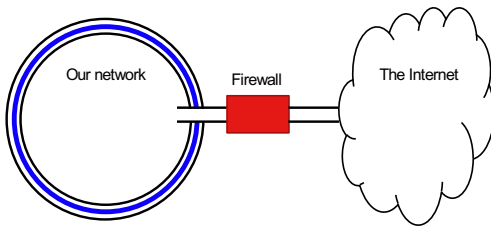
## Firewalls

- A "firewall" is a type of router that passes select traffic through
- Placement of firewall is critical: should be in path where all traffic in and out of a network is forced
- Each packet is compared against a set of rules that decide if it is allowed or not
- Typical rules examine:
  - Interface where packet arrived.
  - IP header: addresses, protocol.
  - TCP, UDP headers: ports, TCP flags
  - Stateful firewalls: is this packet part of a known connection/flow?
- Default policy is often to deny entry. Conservative.

20

## Firewalls

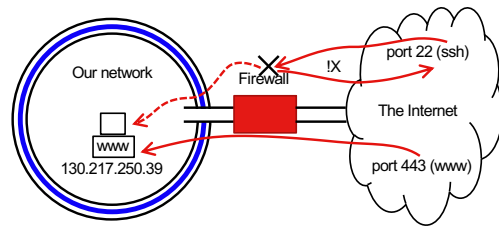
Essentially building a defended "moat" around our "castle" and only allowing approved traffic through via our "drawbridge".



21

## Firewalls

allow tcp from any to 130.217.250.39 443  
unreach filter-prohib ip from any to any



Web (HTTPS) traffic is allowed to pass to 130.217.250.39 by rule #1.  
If someone tries to login to webserver using ssh they are blocked by rule #2.

22

## Summary

- Encryption allows us to protect sensitive data traversing untrusted networks
- One-way hashes and public key encryption allow us to build digital signatures
- Authentication systems allow us to provide our identity across untrusted networks
- Firewalls allow us to police traffic entering and exiting a network

23

## Learning Objectives

- Understand the role of public and private keys in asymmetric encryption
- Understand what a "hash" is
- Understand how a private key and a hash can be used to produce a digital signature, and how the signature can be verified with the public key
- Understand how two parties can verify a shared secret without disclosing that secret
- Understand the role of a network firewall and the types of decisions it might make

24

# COMPX204-22B Practical Networking and Cybersecurity

## Lecture 14 Network Security – Part 3

Matthew Luckie  
mluckie@waikato.ac.nz



© THE UNIVERSITY OF WAIKATO • 27 September 2022

1

## This lecture

- Transport Layer Security (TLS)
  - Previously known as Secure Sockets Layer (SSL)
- Certificate Path Verification
- Hostname Validation

© THE UNIVERSITY OF WAIKATO • 27 September 2022

2

2

## Learning Objectives

- Understand (conceptually) how TLS provides authentication and encryption, and what its weaknesses are
- Understand how certificate hierarchies can allow validation of arbitrary certificates if they are signed by a trusted authority
- Understand how certificates can be validated, and what the responsibilities of the programmer are

© THE UNIVERSITY OF WAIKATO • 27 September 2022

3

3

## Transport Layer Security (TLS)

- History: Initially developed as SSL by Netscape
  - (original GUI web-browser)
  - First use: HTTPS (secure HTTP)
  - 1993-1995
  - Quickly applied to other protocols
- Goal of TLS:
  - Framework to authenticate socket end-point
  - Provide privacy through encryption

© THE UNIVERSITY OF WAIKATO • 27 September 2022

4

4

## Transport Layer Security (TLS)

- Goal was to provide an application programming interface (API) like Sockets, to allow for retro-fitting of security
- But security is complex, and it isn't that easy
- Most (all?) TLS libraries are awful
- TLS has evolved

© THE UNIVERSITY OF WAIKATO • 27 September 2022

5

5

## TLS Evolution

- SSLv1: 1993, never released
- SSLv2: 1995, first public protocol, flawed, absolutely do not use
- SSLv3: 1996, last SSL protocol from Netscape. Also broken
- TLSv1.0: 1999, IETF standards version of protocol. Can be downgraded to SSLv3, so do not use this either
- TLSv1.1: 2006, broken, deprecated
- TLSv1.2: 2008.
- TLSv1.3: 2018. } Current Recommended Protocols

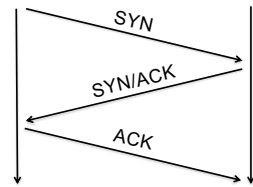
© THE UNIVERSITY OF WAIKATO • 27 September 2022

6

6

## TLS Overview

- TLS is generally used in conjunction with TCP
- TLS may be used at any time in the TCP connection, but typically is used immediately upon connection establishment



© THE UNIVERSITY OF WARRICK - TE WARRICK WARRICKA C WARRICKO

27 September 2022

7

## TLS Handshake

ClientHello

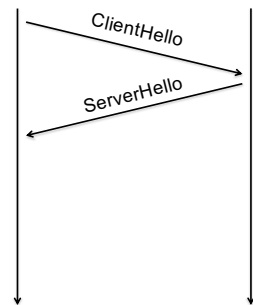
Client tells server supported Cipher functions, highest TLS version supported, and supplies random value

© THE UNIVERSITY OF WARRICK - TE WARRICK WARRICKA C WARRICKO

27 September 2022

8

## TLS Handshake



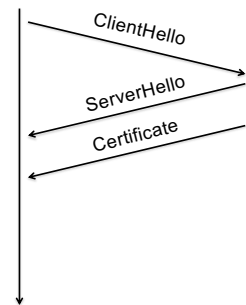
Server selects Cipher and Hash functions, TLS version, a random value. Reports these values to the client

© THE UNIVERSITY OF WARRICK - TE WARRICK WARRICKA C WARRICKO

27 September 2022

9

## TLS Handshake



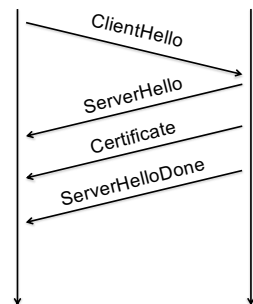
Server sends the client a certificate describing their public key, server name, and who has signed their certificate

© THE UNIVERSITY OF WARRICK - TE WARRICK WARRICKA C WARRICKO

27 September 2022

10

## TLS Handshake



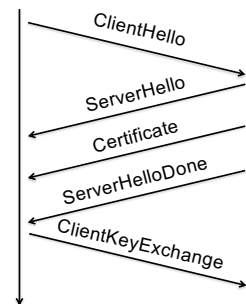
Server tells Client it is done with sending TLS parameters

© THE UNIVERSITY OF WARRICK - TE WARRICK WARRICKA C WARRICKO

27 September 2022

11

## TLS Handshake

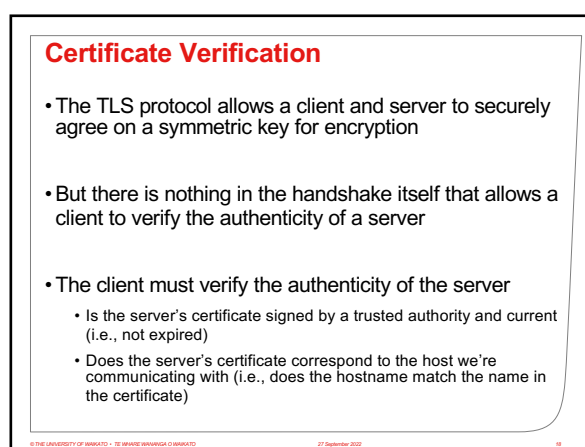
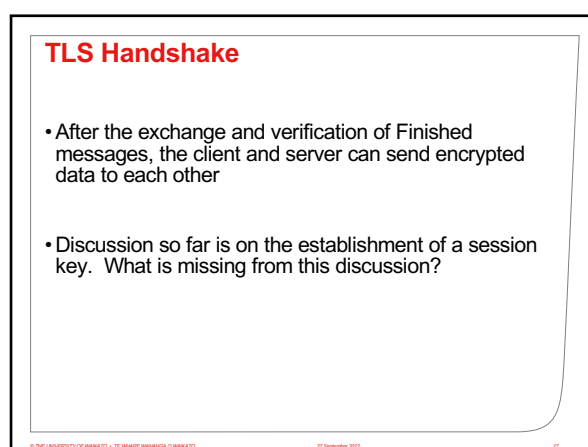
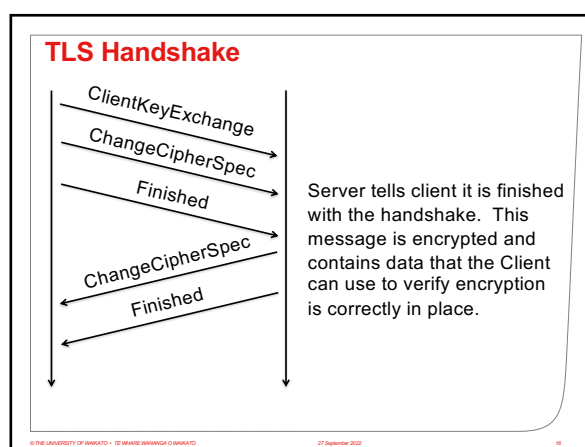
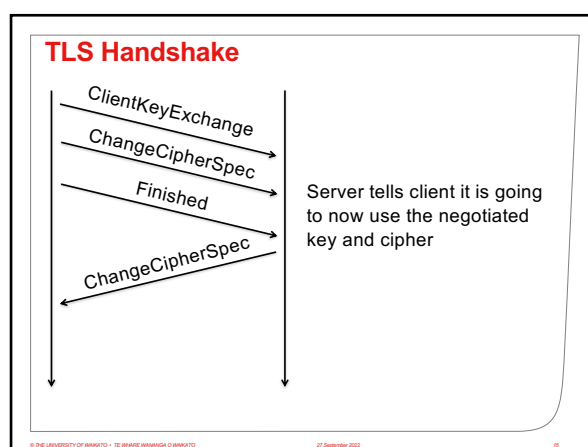
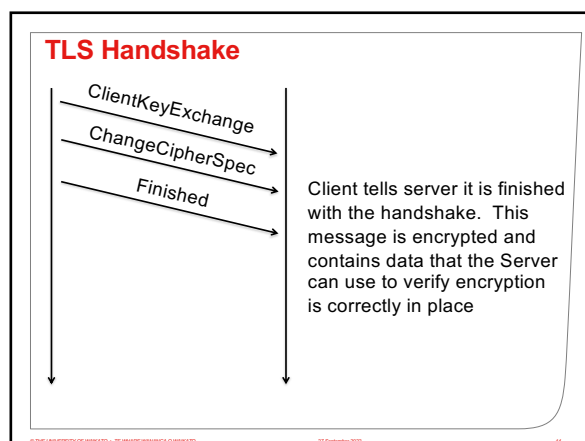
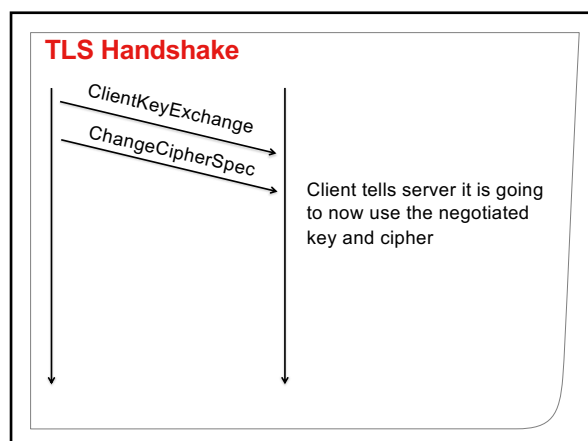


Client sends information to allow the server to derive the same symmetric encryption key that the client will use.

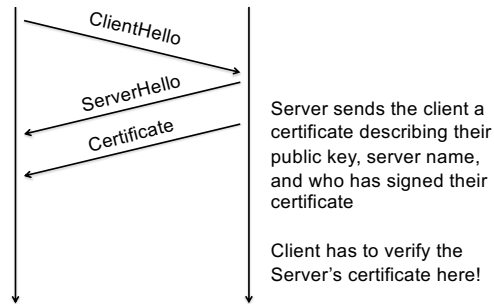
© THE UNIVERSITY OF WARRICK - TE WARRICK WARRICKA C WARRICKO

27 September 2022

12



## Certificate Verification



19

## The most dangerous code in the world

- Title of a research paper presented at ACM Computer and Communications Security (CCS) 2012
- Many TLS libraries (OpenSSL, JSSE, etc)
  1. Verify the certificate is current and signed by a recognized authority
  2. **Do not provide a convenient way to perform hostname verification**
- Affected (2012) many popular apps:
  - Payment applications
  - Chase online banking app
  - Lots of Android applications

20

## Certificate Path Verification

- A certificate path verification algorithm checks that the certificate presented has been signed by a trusted authority
- The signature is included in the certificate presented
  - a digital signature
- Need a set of trusted public keys of certificate authorities (CAs)
- Operating systems and web browsers ship with a file that contains public keys of ~200 CAs.
- TLS libraries check that the digital signature is correct by verifying it with the public key of the CA.

21

## Certificate Authorities (CAs)

- Hundreds exist, not all trustworthy
- Server operators create:
  - Public Certificate and Private Key pair
  - Certificate Signing Request (CSR) to pass to a CA
- CA verifies the CSR is authentic
- CA signs the certificate with their private key and returns the signed certificate to the server operator
- CA typically requires payment for this service
  - Alternative: Let's Encrypt.

22

## Certificate Signing Requests (Abridged)

```

$ openssl req -new -keyout sorcerer-private.pem -out sorcerer-csr.pem
Generating a 2048 bit RSA private key
.....+++
.....+++
writing new private key to 'sorcerer-private.pem'
Enter PEM pass phrase: <passphrase>
Verifying - Enter PEM pass phrase: <passphrase>

Country Name (2 letter code) [AU]:NZ
State or Province Name (full name) [Some-State]:Waikato
Locality Name (eg, city) []:Hamilton
Organization Name (eg, company) [Internet Widgits Pty Ltd]:University of Waikato
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:sorcerer.cms.waikato.ac.nz
Email Address []:mluckie@waikato.ac.nz

```

Bold font: what I typed.

23

## Certificate Signing

sorcerer-csr.pem → CA

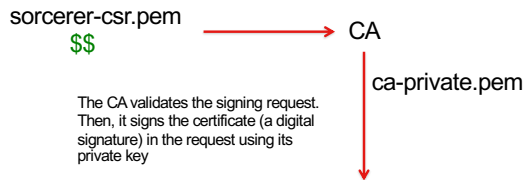
CSR: Certificate Signing Request.  
This file contains a public key, and certificate meta data regarding who is making the signing request and for how long.

CA: Certificate Authority.  
One of ~200 that could sign the certificate, allowing the certificate to be trusted by most OSes

Note: sorcerer-private.pem (the private key) is never disclosed to the CA (or any third party)

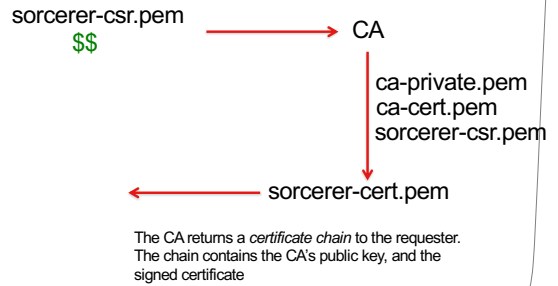
24

## Certificate Signing



25

## Certificate Signing



26

## Using Certificates in Server Applications

- Need to provide the server application both
  - The certificate to present to the client
  - The private key to decrypt the ClientKeyExchange process

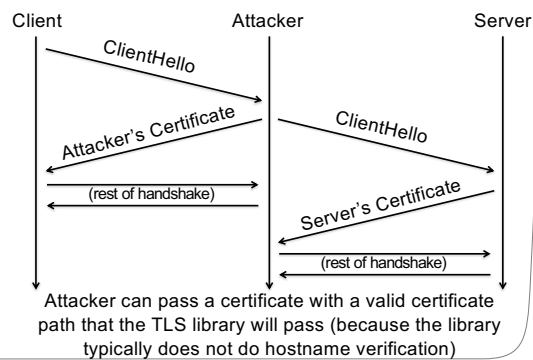
27

## The most dangerous code in the world

- Title of a research paper presented at ACM Computer and Communications Security (CCS) 2012
- Many TLS libraries (OpenSSL, JSSE, etc)
  1. Verify the certificate is current and signed by a recognized authority
  2. **Do not provide a convenient way to perform hostname verification**
- Affected (2012) many popular apps:
  - Payment applications
  - Chase online banking app
  - Lots of Android applications

28

## Man in the middle attacks



29

## Hostname Verification

- Users of TLS libraries are usually required to
  1. Ensure that the server presented a certificate
  2. Ensure that the certificate corresponds to the end-point
- Usually accomplished by comparing the hostname with the name specified in the certificate
  - If connecting to `www.google.com`, does the name in the certificate match `www.google.com`?
  - If they do not match, the connection should be immediately aborted
- This too can be complicated, will talk about this more next lecture.

30

## Learning Objectives

- Understand (conceptually) how TLS provides authentication and encryption, and what its weaknesses are
- Understand how certificate hierarchies can allow validation of arbitrary certificates if they are signed by a trusted authority
- Understand how certificates can be validated, and what the responsibilities of the programmer are

31

## Summary

- The TLS protocol provides a secure way to establish a symmetric key to encrypt communications
- Protocol relies on certificate verification:
  - Is there a valid certificate path to a CA?
  - Does the hostname match that found in the certificate presented?
- Next lecture: using a TLS library

32

# COMPX204-22B Practical Networking and Cybersecurity

## Lecture 15 Network Security – Part 4

Matthew Luckie  
mluckie@waikato.ac.nz



20 September 2022

1

## Previous Lecture

- Transport Layer Security (TLS)
  - Protocol supporting authentication and encryption
- Certificate Path Verification
  - Is there a signed path from a certificate sent during the TLS handshake to a certificate we trust?
- Hostname Validation
  - Does the Hostname in the certificate presented by the server correspond to the system we believe we are communicating with?

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

2

2

## This Lecture

- Java APIs for working with TLS Sockets
  - How to use them
  - Pitfalls
  - Tightly bound to Assignment 4, which should walk you through this lecture in a more practical way.
- An approach to building a trivial communications system that uses TLS

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

3

3

## Learning Objectives

- Become familiar with various Java classes that form the Java Secure Socket Extension (JSSE) library
- In conjunction with Assignment 4, put together a simple secure file transfer application that uses TLS

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

4

4

## Java Secure Socket Extension (JSSE)

- Package built into Java to provide useful classes for security – particularly TLS.
- `import javax.net.ssl.*;`
  - Classes for using SSL sockets
- `import java.security.KeyStore`
  - Classes for loading keys for authentication and encryption
- `import java.security.cert.X509Certificate;`
  - Holds a Certificate in X.509 format
- `import javax.naming.Ldap.*;`
  - Classes for parsing X.509 certificate
- `import javax.net.*;`
  - Classes for “Socket Factories”

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

5

5

## TLS Sockets

- Java provides `SSLSocket` and `SSLServerSocket` classes, which are roughly equivalent to `Socket` and `ServerSocket`
  - `SSLServerSocket` listens on a port for incoming connections, like `ServerSocket`
  - `SSLSocket` connects to an `SSLServerSocket`, like `Socket`, and represents an individual connection accepted from an `SSLServerSocket`.
- To create a `SSLSocket` or `SSLServerSocket`, we must use factories

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 September 2022

6

6

## TLS Socket Factories

- Convenient way to set TLS parameters that will apply to Sockets created from the factory, e.g:
  - Which TLS versions to support
  - Which Ciphers and Hashes to use
  - Which Keys to use and which Certificates to trust
- SSLServerSocketFactory creates SSLServerSocket objects
- SSLSocketFactory creates SSLSocket objects

7

## KeyStore, KeyManagerFactory

- Not going to talk about these aspects of the API.
- These are something best learned through experience
  - Assignment Four!
- For this lecture, it is enough to know a KeyStore object contains a set of Keys and Certificates to use when encrypting and authenticating
- The files that contain Keys and Certificates are password protected

8

## SSLServerSocketFactory

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
private static ServerSocketFactory getSSF()
{
 /*
 * Get an SSL Context that speaks some version
 * of TLS, a KeyManager that can hold certs in
 * X.509 format, and a JavaKeyStore (JKS)
 * instance
 */
 SSLContext ctx =
 SSLContext.getInstance("TLS");

 KeyManagerFactory kmf =
 KeyManagerFactory.getInstance("SunX509");

 KeyStore ks =
 KeyStore.getInstance("JKS");
```

9

## SSLServerSocketFactory

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
private static ServerSocketFactory getSSF()
{
 /*
 * store the passphrase to unlock the JKS file.
 * completely insecure, there are better ways.
 * DON'T DO THIS!
 */
 char[] passphrase = "<whatever>".toCharArray();

 /*
 * load the keystore file. The passphrase is
 * an optional parameter to allow for integrity
 * checking of the keystore. Could be null
 */
 ks.load(new FileInputStream("server.jks"),
 passphrase);
```

10

## SSLServerSocketFactory

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
private static ServerSocketFactory getSSF()
{
 /*
 * init the KeyManagerFactory with a source
 * of key material. The passphrase is necessary
 * to unlock the private key contained.
 */
 kmf.init(ks, passphrase);

 /*
 * initialise the SSL context with the keys.
 */
 ctx.init(kmf.getKeyManagers(), null, null);
```

11

## SSLServerSocketFactory

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
private static ServerSocketFactory getSSF()
{
 /*
 * get the factory we will use to create
 * our SSLServerSocket
 */
 SSLServerSocketFactory ssf =
 ctx.getServerSocketFactory();

 return ssf;
}
```

12

## SSLServerSocket

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
public static void main(String args[])
{
 /*
 * use the getSSF method to get a
 * SSLServerSocketFactory and create our
 * SSLServerSocket, bound to specified port
 */
 ServerSocketFactory ssf = getSSF();
 SSLServerSocket ss =
 (SSLServerSocket) ssf.createServerSocket(port);

 String EnabledProtocols[] =
 {"TLSv1.2", "TLSv1.3"};
 ss.setEnabledProtocols(EnabledProtocols);

 SSLSocket s = (SSLSocket)ss.accept();
}
```

13

## So far

- Have created a SSLServerSocketFactory
- Initialised its Key Material
- Created an SSLServerSocket
- Bound that SSLServerSocket to a specified port
- Accepted an SSLSocket (a client socket)
  - Note: to force a TLS handshake, read from the accepted SSLSocket. (e.g., `BufferedReader::readLine()`) or use `SSLSocket::startHandshake()`.
- Next: the client side of this puzzle

14

## Client Side

- Usually much more straight forward
- Defaults will load Java's set of Trusted Certificates
- Java will validate there is a path to a trusted CA
- By default, Java will not do hostname validation

15

## SSLSocketClient

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
public static void main(String args[])
{
 SSLSocketFactory factory =
 (SSLSocketFactory) SSLSocketFactory.getDefault();

 SSLSocket socket =
 (SSLSocket) factory.createSocket(<host>, <port>);

 socket.startHandshake();

 /*
 * at this point, can getInputStream and
 * getOutputStream as you would a regular Socket
 */
}
```

16

## SSLSocketClient

- Missing code: setting the EnabledProtocols as with the SSLServerSocket
- Still need to do HostName validation.
- Since Java 7, you can tell the SSLSocket to include HostName validation in its checks.
  - 2011.

17

## SSLSocketClient

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
public static void main(String args[])
{
 /*
 * set HTTPS-style checking of HostName
 * before the handshake commences
 */

 SSLParameters params = new SSLParameters();

 params.setEndpointIdentificationAlgorithm("HTTPS");
 socket.setSSLParameters(params);

 socket.startHandshake();
}
```

18

## Before there was setEndpointIdentificationAlgorithm

- Needed to know how an “X509 Certificate” was laid out
- Where to find the HostName?
  - Could be in the Subject field, under the “CN” row, or in subjectAltNames.
- Need to carefully parse these strings if wildcards accepted.
- Subject: C=NZ, ST=Waikato, L=Hamilton, O=University of Waikato, OU=Unknown, CN=sorcerer.cms.waikato.ac.nz
- CN = CommonName: the HostName we want to verify

19

## SSLSocketClient

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
public static void main(String args[])
{
 socket.startHandshake();

 /* get the X509Certificate for this session */
 SSLSession sesh = socket.getSession();
 X509Certificate cert = (X509Certificate)
 sesh.getPeerCertificates()[0];

 /* extract the CommonName, and then compare */
 getCommonName(cert);
}
```

20

## SSLSocketClient

Note: THIS CODE IS MISSING NECESSARY EXCEPTION HANDLING

```
String getCommonName(X509Certificate cert)
{
 String name =
 cert.getSubjectX500Principal().getName();
 LdapName ln = new LdapName(name);
 String cn = null;
 for(Rdn rdn : ln.getRdns())
 if("CN".equalsIgnoreCase(rdn.getType()))
 cn = rdn.getValue().toString();
 return cn;
}
```

Rdn: Relative Distinguished Name

21

## Summary

- Goal of today's lecture was to discuss how to use the Java JSSE APIs to do TLS
- Other APIs in other languages are similar. Use TLS libraries with care. Know what your responsibilities are.
- JSSE will become more concrete in assignment four

22

## Learning Objectives

- Become familiar with various Java classes that form the Java Secure Socket Extension (JSSE) library
- In conjunction with Assignment 4, put together a simple application that uses TLS

23

# COMPX204-22B Practical Networking and Cybersecurity

## Lecture 16 IP Routing

Matthew Luckie  
mluckie@waikato.ac.nz



© The University of Waikato - Te Whare Wānanga o Waikato 2022

1

## So far

- Internet Protocol v4 / v6
  - 32/128 bit IP addresses
  - Structured addressing scheme
  - Internet-wide addressing
- This lecture
  - A structured addressing scheme lends itself to reasonably efficient routing
  - Flat addressing schemes are not scalable

© The University of Waikato - Te Whare Wānanga o Waikato

2 October 2022

2

2

## Learning Objectives

- Understand that an IP address consists of “network ID” and “host ID” portions
- Understand the features that distinguish a “router” from an “end host”
- Understand what information is stored in a “routing table”
- Given a routing table, be able to logically visualize how the system is connected

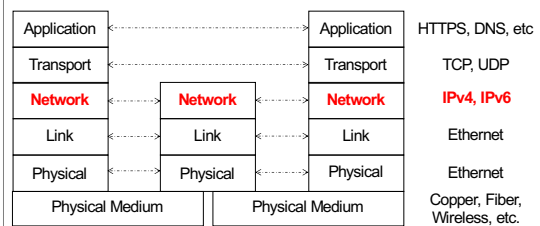
© The University of Waikato - Te Whare Wānanga o Waikato

3 October 2022

3

3

## Protocol Layers (the Internet view)



A layered hierarchy: each layer implements a protocol to talk to the same layer at other nodes

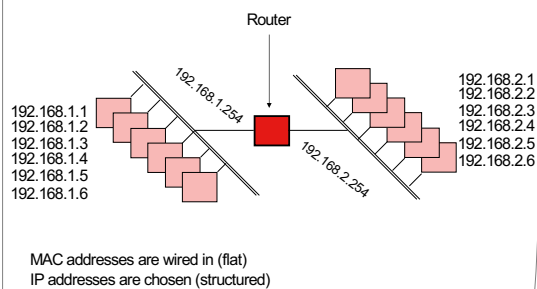
© The University of Waikato - Te Whare Wānanga o Waikato

4 October 2022

4

4

## Networks



© The University of Waikato - Te Whare Wānanga o Waikato

5 October 2022

5

5

## IP addresses

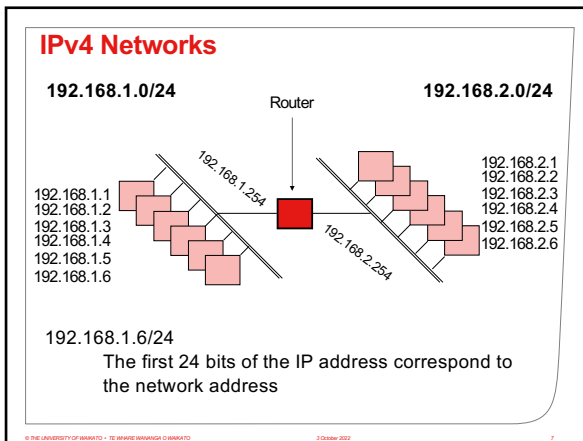
- An IP address consists of
  - A network id (prefix)
  - A host id (suffix)
- In IPv4, the network address usually occupies anywhere between 8 and 24 bits of the address
  - Though it can go all the way to 31.
- In IPv6, the network address typically occupies anywhere between 19 and 64 bits of the address
  - Though it can go all the way to 127

© The University of Waikato - Te Whare Wānanga o Waikato

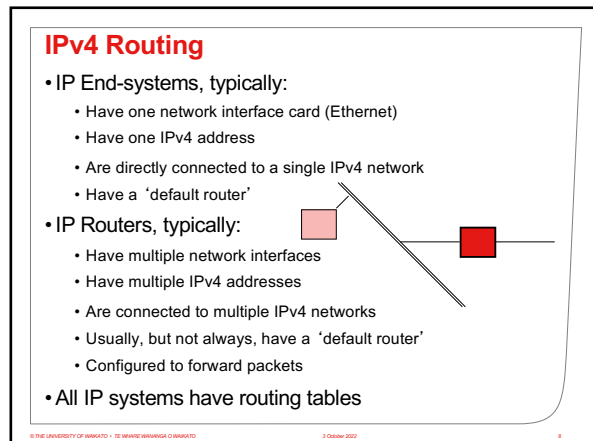
6 October 2022

6

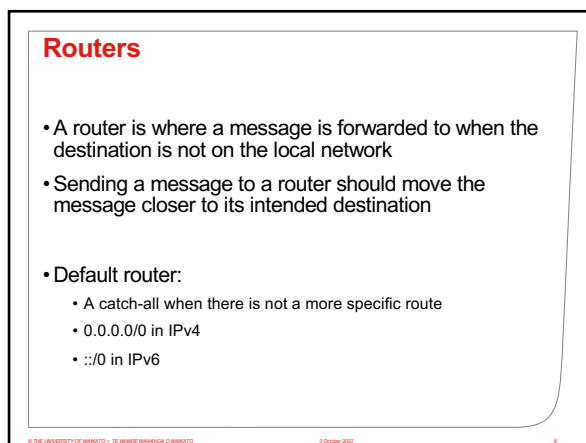
6



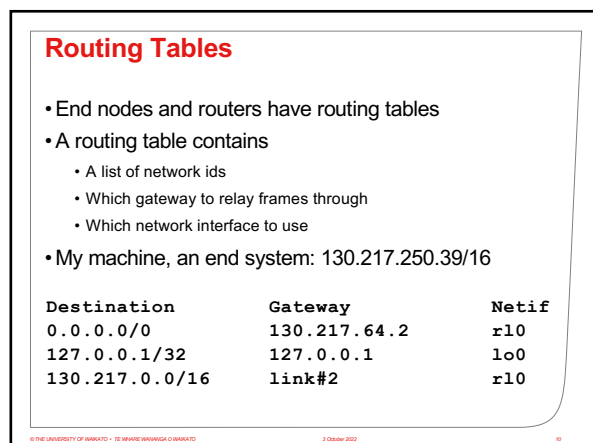
7



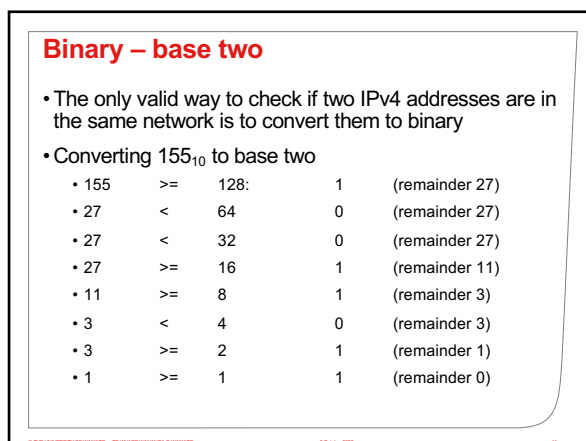
8



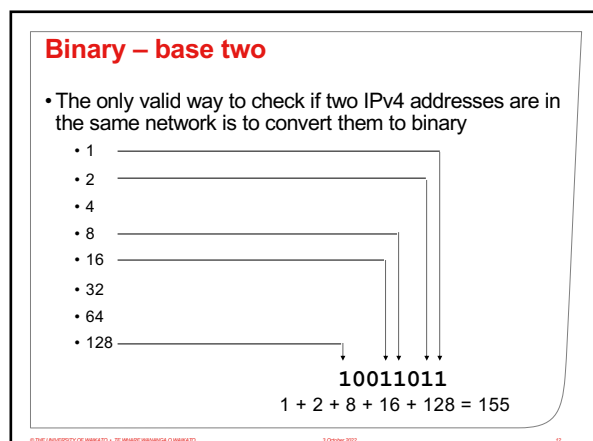
9



10



11



12

**192.168.1.6/24**

```

192 . 168 . 1 . 6
1100 0000 1010 1000 0000 0001 0000 0110

```

**192.168.2.3/24**

```

192 . 168 . 2 . 3
1100 0000 1010 1000 0000 0010 0000 0011

```

These two IP addresses are on different networks because their network addresses are not identical.

13

**192.168.238.45/20**

```

192 . 168 . 238 . 45
1100 0000 1010 1000 1110 1110 0010 1101

```

**192.168.241.55/20**

```

192 . 168 . 241 . 55
1100 0000 1010 1000 1111 0001 0011 0111

```

These two IP addresses are on different networks because their network addresses are not identical.

14

**What about these two?**

- 10.55.1.30/12
- 10.56.44.99/12

15

**Subnet masks**

- Sometimes you will see a subnet mask written in dotted-quad (like an IP address) when looking at routing tables
- A subnet mask is a value that is used with the logical and operation to extract the network id
- A subnet mask when written in binary contains all-ones, then all-zeros
- 255.255.0.0 => masks out first 16 bits
  - 1111 1111 1111 1111 0000 0000 0000 0000
- 255.255.240.0 => masks out first 20 bits
  - 1111 1111 1111 1111 1111 0000 0000 0000

16

**netstat -rn (Linux)**

| Destination | Gateway      | Genmask       | Iface |
|-------------|--------------|---------------|-------|
| 192.168.0.0 | 0.0.0.0      | 255.255.255.0 | eth1  |
| 130.217.0.0 | 0.0.0.0      | 255.255.0.0   | eth0  |
| 0.0.0.0     | 130.217.64.2 | 0.0.0.0       | eth0  |

no gateway systems are directly connected do not need a router

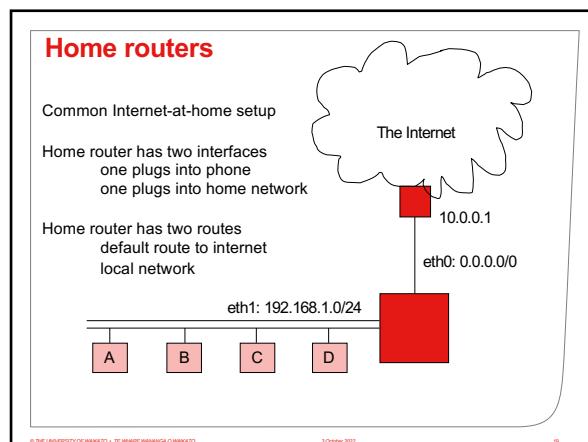
Default route Catch-all

17

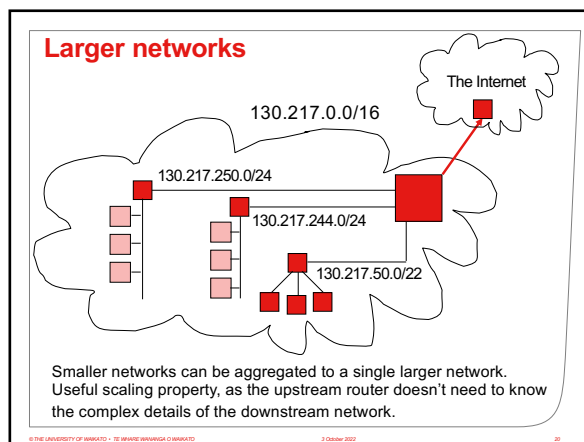
**Network Topology**

| Destination | Gateway      | Genmask       | Iface |
|-------------|--------------|---------------|-------|
| 192.168.0.0 | 0.0.0.0      | 255.255.255.0 | eth1  |
| 130.217.0.0 | 0.0.0.0      | 255.255.0.0   | eth0  |
| 0.0.0.0     | 130.217.64.2 | 0.0.0.0       | eth0  |

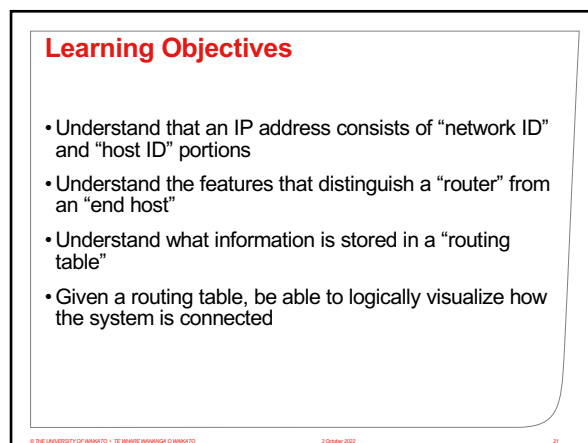
18



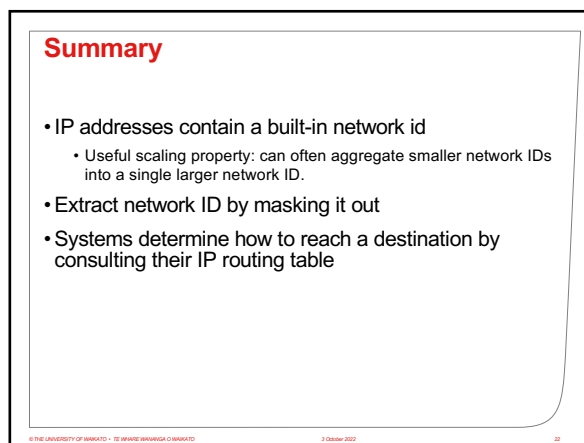
19



20



21



22

# COMPX204-22B Practical Networking and Cybersecurity

## Lecture 17 IP Routing - BGP

Matthew Luckie  
mluckie@waikato.ac.nz



© October 2022

1

## So far

- Internet Protocol v4, v6 (IPv4, IPv6)
  - 32 bit / 128 bit IP addresses
  - Structured addressing scheme, Internet-wide addressing
- Prefix of the address corresponds to the network where the machine is to be found
  - 130.217.250.39 is to be found in the 130.217.0.0/16 network
  - Important scaling property
    - One entry in the routing table for  $2^{16}$  host addresses

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

© October 2022

2

## Learning Objectives

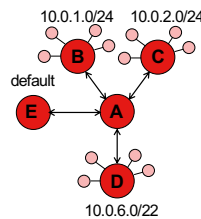
- Be able to reason about which route is chosen, given a routing table and destination address
- Appreciate how Internet-scale routing is achieved using the Border Gateway Protocol
  - What an Autonomous System (AS) is
  - Difference between Provider-Customer and Peer-Peer relationships
  - How routing relationships are established
  - How an AS path is formed
  - How dynamic routing takes place

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

© October 2022

3

## IP forwarding



A's routing table

| Destination | Gateway | Interface |
|-------------|---------|-----------|
| 10.0.1.0/24 | B       | eth0      |
| 10.0.2.0/24 | C       | eth1      |
| 10.0.6.0/23 | D       | eth2      |
| 0.0.0.0/0   | E       | eth3      |

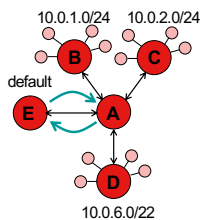
- A has a message to send to 10.0.2.15
- There are two matching routes
  - 10.0.2.0/24, via C
  - Default, via E

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

© October 2022

4

## IP forwarding



A's routing table

| Destination | Gateway | Interface |
|-------------|---------|-----------|
| 10.0.1.0/24 | B       | eth0      |
| 10.0.2.0/24 | C       | eth1      |
| 10.0.6.0/23 | D       | eth2      |
| 0.0.0.0/0   | E       | eth3      |

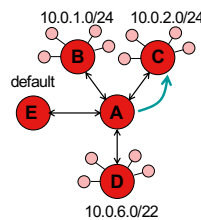
- If it sends it via E
  - E might send it back to A, since E may know to reach C via A.
  - If it does, the message will get stuck in a loop, which is not good.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

© October 2022

5

## IP forwarding



A's routing table

| Destination | Gateway | Interface |
|-------------|---------|-----------|
| 10.0.1.0/24 | B       | eth0      |
| 10.0.2.0/24 | C       | eth1      |
| 10.0.6.0/23 | D       | eth2      |
| 0.0.0.0/0   | E       | eth3      |

- Routing tables can be thought of as ordered from longest network prefix to shortest network prefix
  - /24 is a longer prefix than /22
- The first matching entry in the table is selected
- That is, the most specific destination prefix is chosen

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O HIAWAKATO

© October 2022

6

## IP forwarding

- Order table from most specific to least specific network prefix. Until a match is made:
  - AND the destination address with the subnet mask for the route
  - Compare the result with the network. If they match, return this route
- Example: Select route for 10.0.7.8

| Destination | Gateway | Interface |
|-------------|---------|-----------|
| 10.0.1.0/24 | B       | eth0      |
| 10.0.2.0/24 | C       | eth1      |
| 10.0.6.0/23 | D       | eth2      |
| 0.0.0.0/0   | E       | eth3      |

7

## IP forwarding

- Example: Select route for 10.0.7.8
- Step 1:

|    | Destination | Gateway | Interface |
|----|-------------|---------|-----------|
| 1. | 10.0.1.0/24 | B       | eth0      |
| 2. | 10.0.2.0/24 | C       | eth1      |
| 3. | 10.0.6.0/23 | D       | eth2      |
| 4. | 0.0.0.0/0   | E       | eth3      |

|                 |          |          |          |          |
|-----------------|----------|----------|----------|----------|
| 10.0.1.0/24     | 00001010 | 00000000 | 00000001 | 00000000 |
| 10.0.7.8        | 00001010 | 00000000 | 00000111 | 00001000 |
| /24 Subnet mask | 11111111 | 11111111 | 11111111 | 00000000 |
|                 | 00001010 | 00000000 | 00000111 | 00000000 |

Do not match

8

## IP forwarding

- Example: Select route for 10.0.7.8
- Step 2:

|    | Destination | Gateway | Interface |
|----|-------------|---------|-----------|
| 1. | 10.0.1.0/24 | B       | eth0      |
| 2. | 10.0.2.0/24 | C       | eth1      |
| 3. | 10.0.6.0/23 | D       | eth2      |
| 4. | 0.0.0.0/0   | E       | eth3      |

|                 |          |          |          |          |
|-----------------|----------|----------|----------|----------|
| 10.0.2.0/24     | 00001010 | 00000000 | 00000010 | 00000000 |
| 10.0.7.8        | 00001010 | 00000000 | 00000111 | 00001000 |
| /24 Subnet mask | 11111111 | 11111111 | 11111111 | 00000000 |
|                 | 00001010 | 00000000 | 00000111 | 00000000 |

No Match.

9

## IP forwarding

- Example: Select route for 10.0.7.8
- Step 3:

|    | Destination | Gateway | Interface |
|----|-------------|---------|-----------|
| 1. | 10.0.1.0/24 | B       | eth0      |
| 2. | 10.0.2.0/24 | C       | eth1      |
| 3. | 10.0.6.0/23 | D       | eth2      |
| 4. | 0.0.0.0/0   | E       | eth3      |

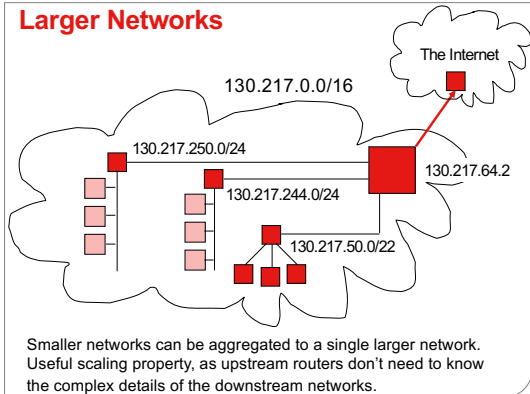
  

|                 |          |          |          |          |
|-----------------|----------|----------|----------|----------|
| 10.0.6.0/23     | 00001010 | 00000000 | 00000110 | 00000000 |
| 10.0.7.8        | 00001010 | 00000000 | 00000111 | 00001000 |
| /23 Subnet mask | 11111111 | 11111111 | 11111110 | 00000000 |
|                 | 00001010 | 00000000 | 00000110 | 00000000 |

Match!

10

## Larger Networks

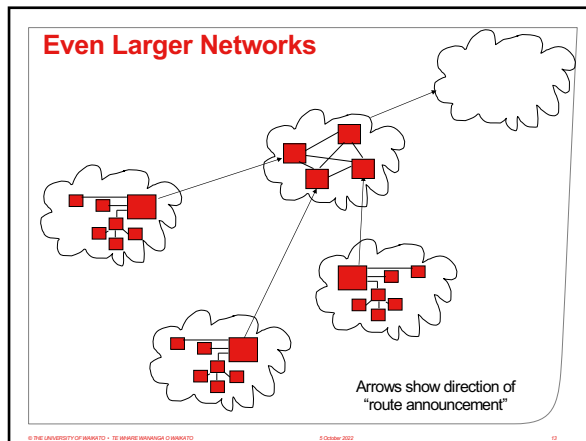


11

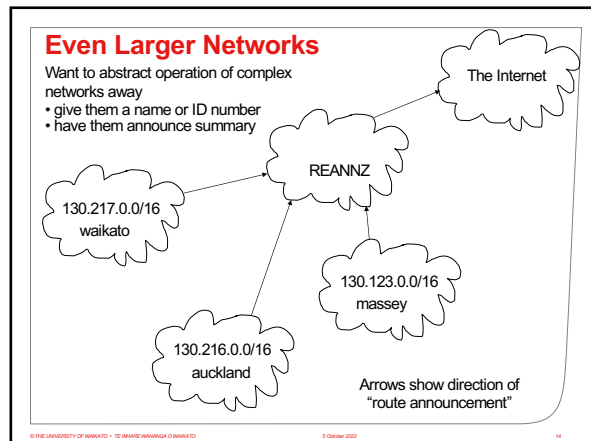
## Larger Networks

- Still possible with this example to manage routing manually, using, e.g.
  - route command (UNIX systems)
  - route add -net 130.216.244.0/24 130.217.64.2
  - Add a route for 130.216.244.0/24 via 130.217.64.2
- Manual (static) routing becomes more difficult as networks get larger
  - Old networks go away
  - New networks are added
  - Impossible for the Internet

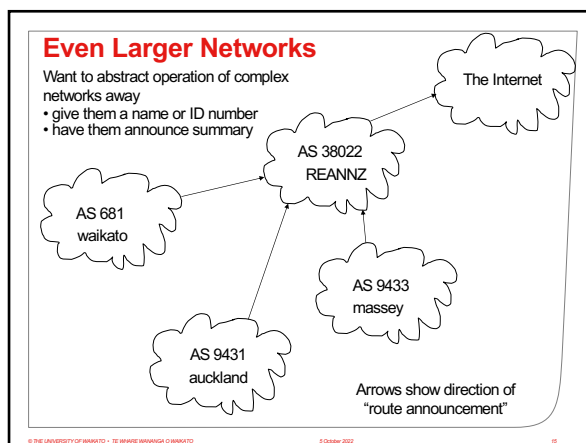
12



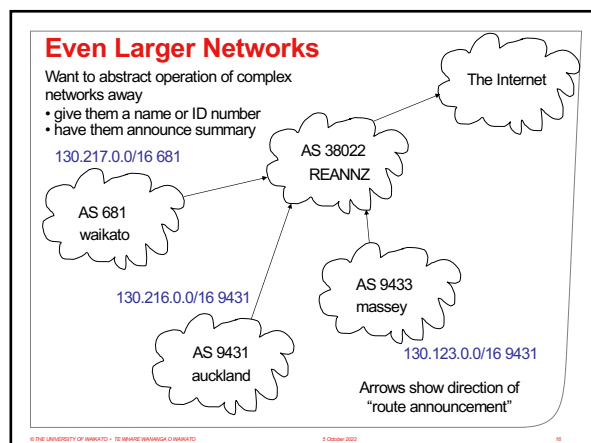
13



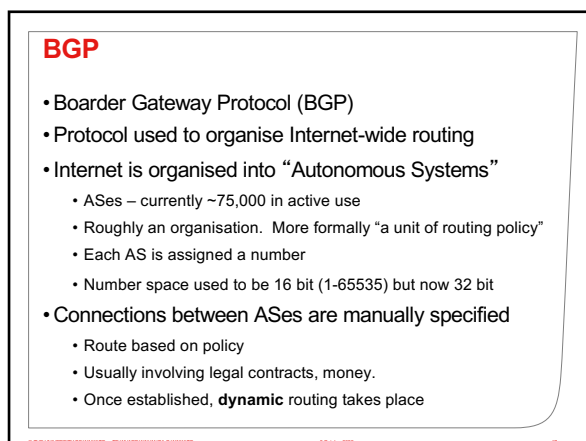
14



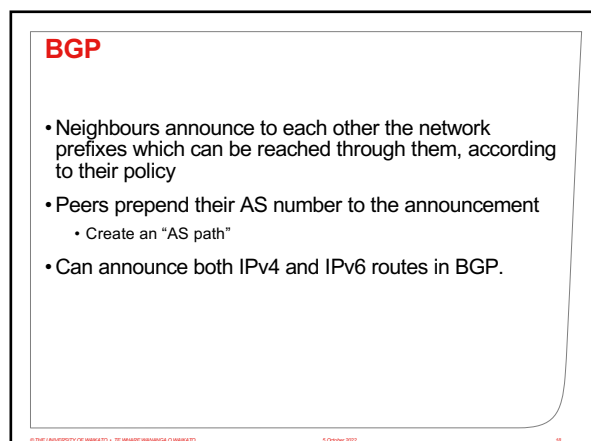
15



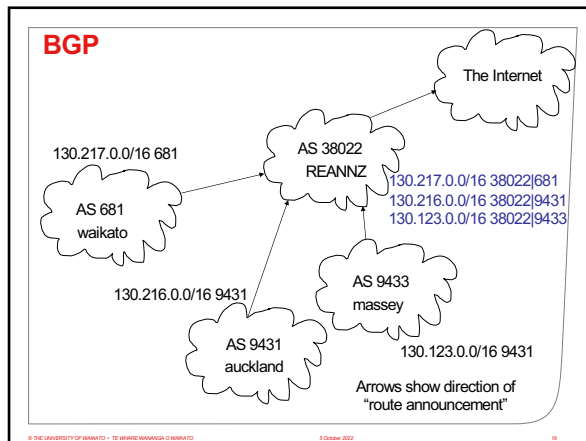
16



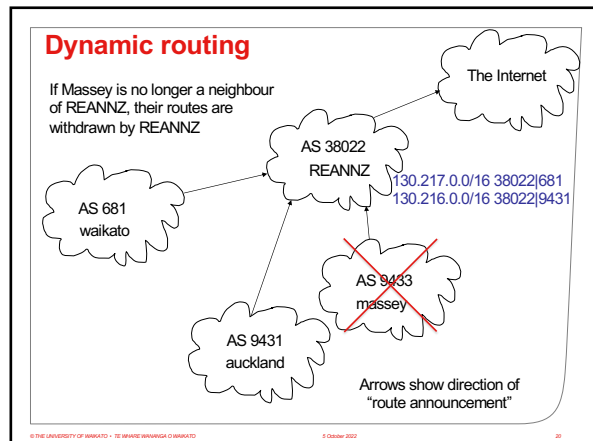
17



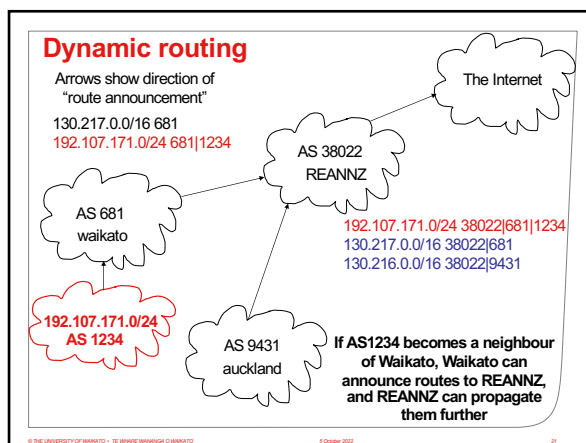
18



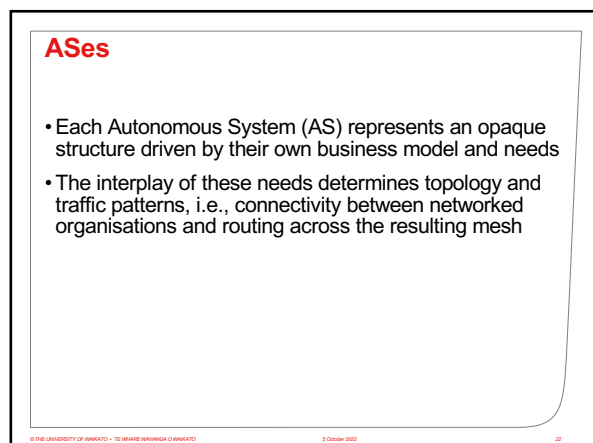
19



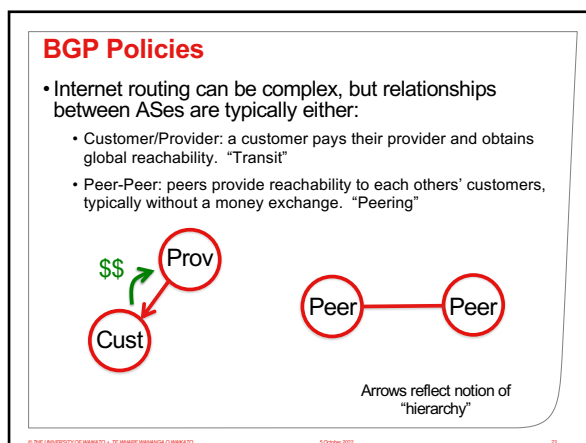
20



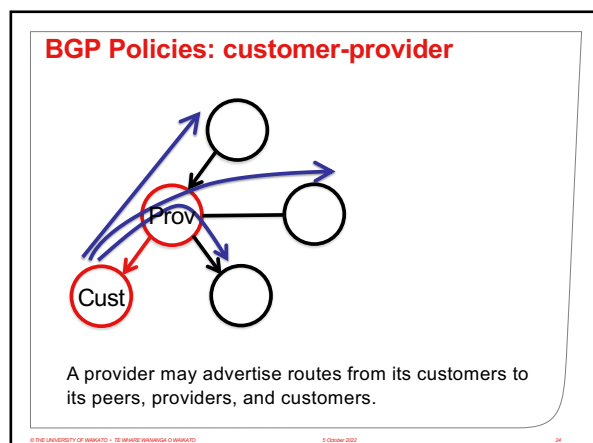
21



22

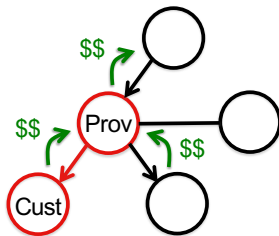


23



24

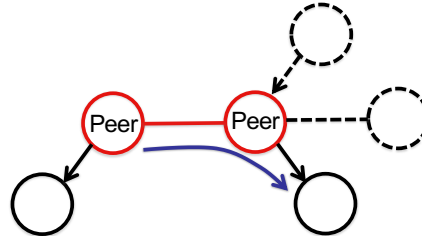
### BGP Policies: customer-provider



A provider may advertise routes from its customers to its peers, providers, and customers.

25

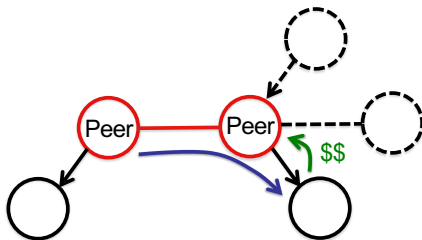
### BGP Policies: peer-peer



A peer should only advertise routes from other peers to its customers. Otherwise, it is providing free "transit"

26

### BGP Policies: peer-peer



A peer should only advertise routes from other peers to its customers. Otherwise, it is providing free "transit"

27

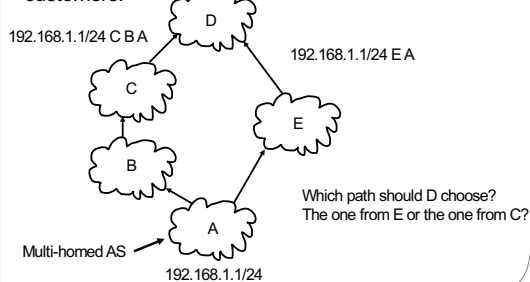
### BGP Policies as "local preference"

- When an operator configures a BGP session with another network, they assign a "local preference" integer value to received routes that specify how the network should prioritise routes relative to other routes for the same prefix it receives
  - Higher values == higher preference
- Typically operators assign higher values to routes received from customers, then peers, then providers.
  - If a network receives the same prefix from a provider and a customer, the network will prefer the route from the customer
- Goal is often to maximise revenue while minimizing cost

28

### BGP

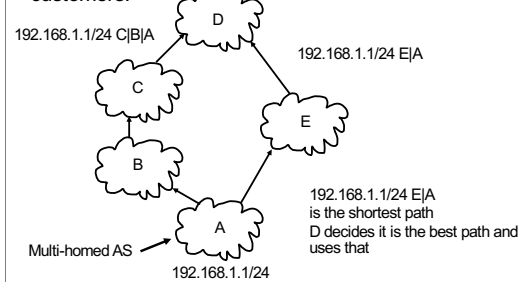
- Routers may receive an advertisement for the same IP prefix from multiple ASes. Assume C and E are both customers.



29

### BGP

- Routers may receive an advertisement for the same IP prefix from multiple ASes. Assume C and E are both customers.



30

## Learning Objectives

- Be able to reason about which route is chosen, given a routing table and destination address
- Appreciate how Internet-scale routing is achieved using the Border Gateway Protocol
  - What an Autonomous System (AS) is
  - Difference between Provider-Customer and Peer-Peer relationships
  - How routing relationships are established
  - How an AS path is formed
  - How dynamic routing takes place

© THE UNIVERSITY OF MANCHESTER • 10 MANCHESTER MANAGED COURSEWORK

© October 2022

31

31

## Summary

- IP Routing tables are organised so that the most specific route is chosen to a destination
- Internet routing is organised using the BGP protocol
  - ASes
  - An AS being a network operated by a single organisation
  - In general, an AS prefers the highest preference route available, then the one with the shortest AS path length. Other rules follow, but these are two fundamental rules
- Future lectures: Interior routing protocols

© THE UNIVERSITY OF MANCHESTER • 10 MANCHESTER MANAGED COURSEWORK

© October 2022

32

32

# COMPX204-22B Practical Networking and Cybersecurity

## Lecture 18 Dynamic Internal Routing Protocols

Matthew Luckie  
mluckie@waikato.ac.nz



10 October 2022

1

## Learning Objectives

- Understand the limits of manual configuration of networks and why automatic routing protocols are useful
- Understand how Dijkstra's algorithm works given a sample network topology
- Understand the factors that influence the "Distance" metric

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

10 October 2022

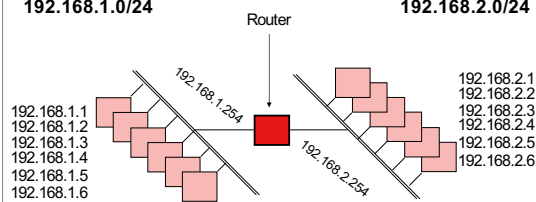
2

2

## Local Area Networks

192.168.1.0/24

192.168.2.0/24



192.168.1.6/24

The first 24 bits of the IP address correspond to the network address

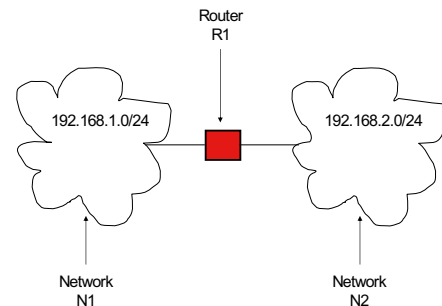
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

10 October 2022

3

3

## Local Area Networks



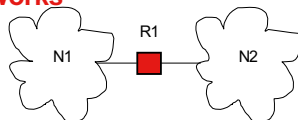
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

10 October 2022

4

4

## Local Area Networks



### • Simple network:

- Hosts in N1 have two routes:
  - One for directly connected hosts in N1
  - One for hosts in N2 via R1
- Hosts in N2 have two routes:
  - One for directly connected hosts in N2
  - One for hosts in N1 via R1
- No redundancy (backup paths).
- Could configure network by hand.

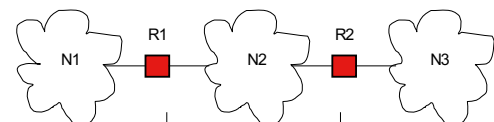
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

10 October 2022

5

5

## What about this one?



Hosts in N1, 3 routes:  
N1 (directly connected)  
N2 via R1  
N3 via R1

-or-

2 routes:  
N1 (directly connected)  
Default via R1

Hosts in N2, 3 routes:  
N2 (directly connected)  
N1 via R1  
N3 via R2

Hosts in N3, 3 routes:  
N3 (directly connected)  
N2 via R2  
N1 via R2

-or-

2 routes:  
N3 (directly connected)  
Default via R2

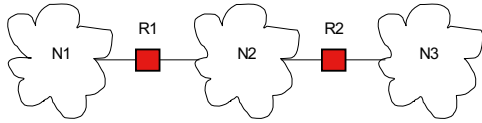
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

10 October 2022

6

6

### What about this one?



- Still a fairly simple network topology.
  - Could still be configured by hand.
  - No redundancy. If anything between R1 and R2 breaks, hosts in N3 and N1 would be unable to communicate
    - would be “partitioned”

© THE UNIVERSITY OF BRISTOL - 10.1016/j.sbspro.2012.01.001

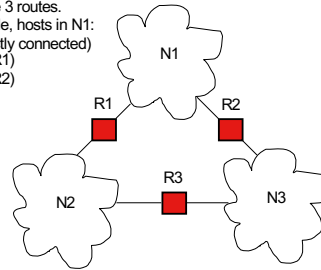
10 October 2012

7

### How about this one?

Hosts have 3 routes.  
For example, hosts in N1:

- N1 (directly connected)
- N2 (via R1)
- N3 (via R2)



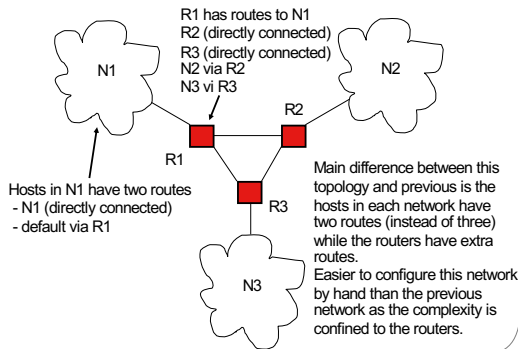
Note: Redundant links!

© THE UNIVERSITY OF BRISTOL - 10.1016/j.sbspro.2012.01.001

10 October 2012

8

### Possibly more likely, though:



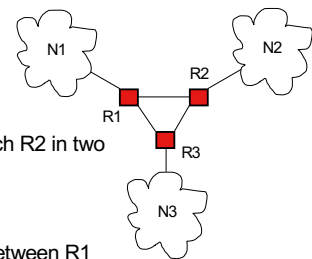
© THE UNIVERSITY OF BRISTOL - 10.1016/j.sbspro.2012.01.001

10 October 2012

9

### Redundant links

- Note that R1 can reach R2 in two ways:
  - Directly connected
  - Via R3.
- However, if the link between R1 and R2 is broken, the backup path will not be automatically used
  - Requires a person to re-configure R1



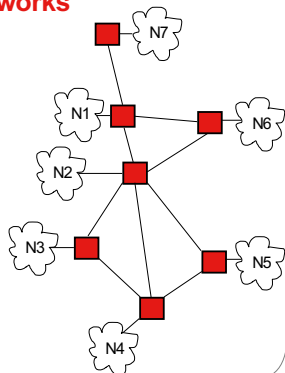
© THE UNIVERSITY OF BRISTOL - 10.1016/j.sbspro.2012.01.001

10 October 2012

10

### More Complex networks

- Starting to get a little more complicated.
- Manual configuration:
  - Tedious
  - Error prone
  - Not able to take advantage of redundant paths in the topology
- Automated configuration desirable



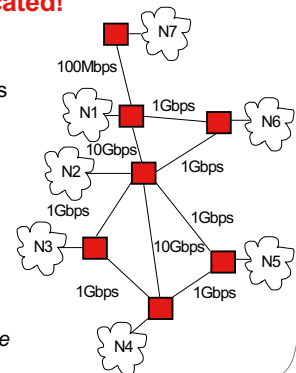
© THE UNIVERSITY OF BRISTOL - 10.1016/j.sbspro.2012.01.001

10 October 2012

11

### Still more complicated!

- Would like optimal paths to be used
- How do we define optimality?
  - Hop count?
  - Transmission delay?
  - Capacity?
  - Available capacity?
  - Monetary cost?
- Abstract metric: *distance*



© THE UNIVERSITY OF BRISTOL - 10.1016/j.sbspro.2012.01.001

10 October 2012

12

## Dynamic routing protocols

- There are many, broken into two categories;
  - Internal routing protocols
  - External routing protocols
- External routing protocol – BGP (previously covered)
- Internal routing protocols:
  - RIP
  - OSPF Going to focus on this one.
  - IS-IS
  - EIGRP

13

## Open Shortest Path First (OSPF)

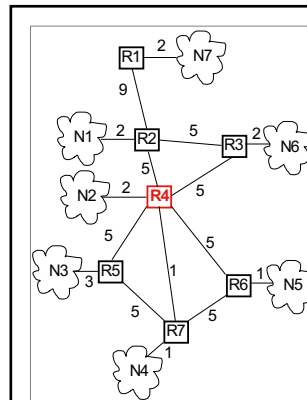
- All routers tell all routers directly connected to them about the links they have and details about the cost of using that link
  - Link-state routing protocol
- All routers tell all routers they are connected to about the links they learn about
  - Process is called 'flooding'
  - Routers end up with enough information to form a graph, like those shown so far
- All routers apply Dijkstra's algorithm to calculate the optimal topology, from where they stand.
- Link failure is detected through lack of a periodic message to let the neighbour they are still there

14

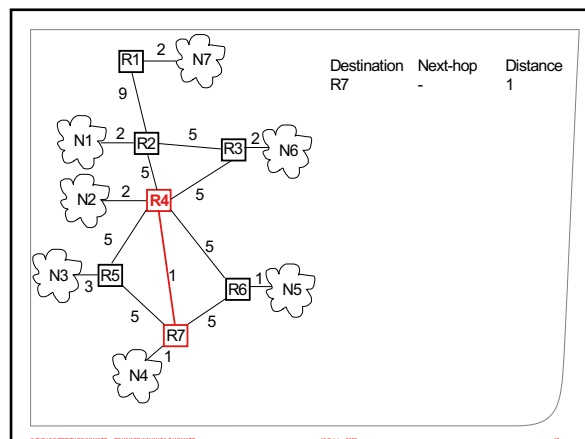
## Dijkstra's algorithm

- Algorithm to find the lowest cost path to all nodes in a graph, from a single node's perspective
- A node running the algorithm iteratively selects the lowest cost path to a node that has not yet been visited until all nodes have been visited
- Produces routing table consisting of
  - Destination
  - Next hop
  - Distance

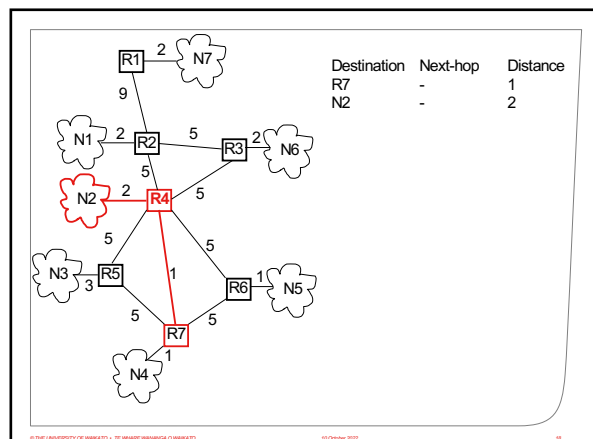
15



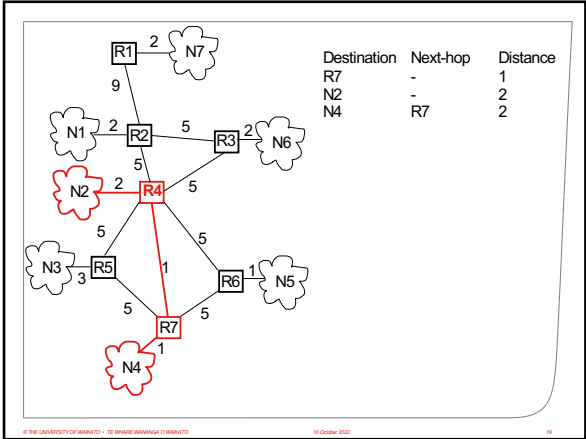
16



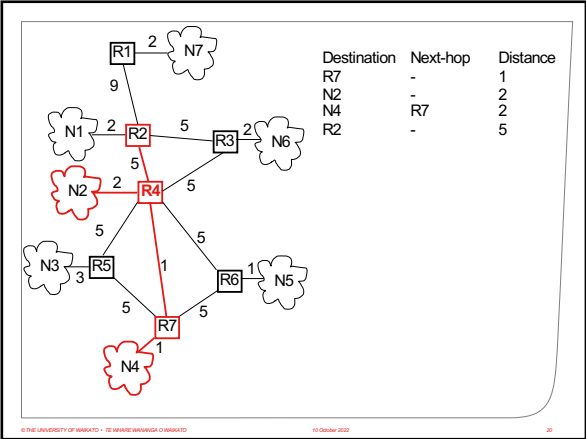
17



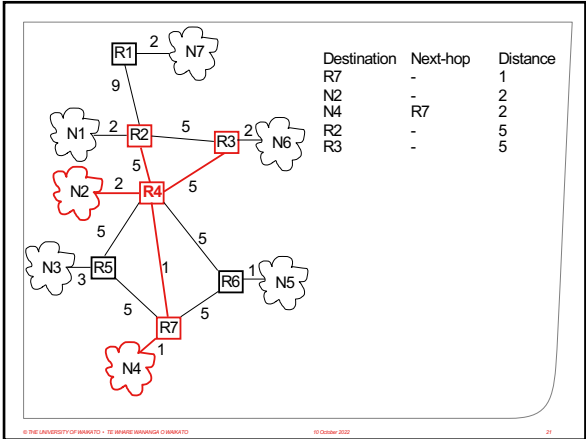
18



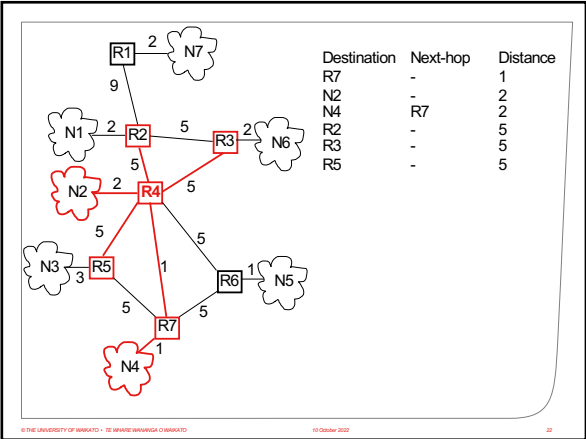
19



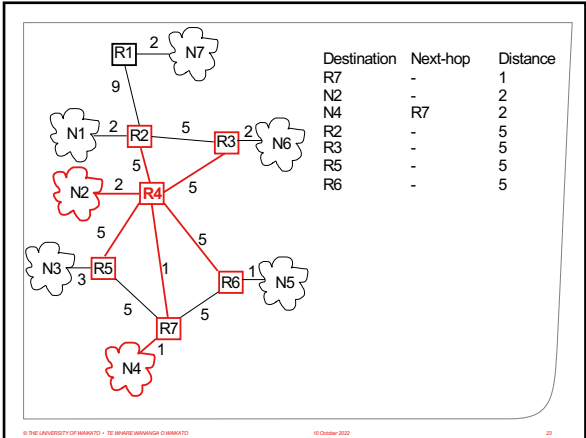
20



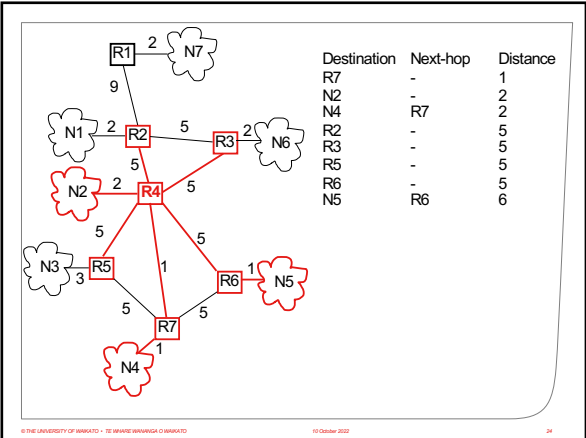
21



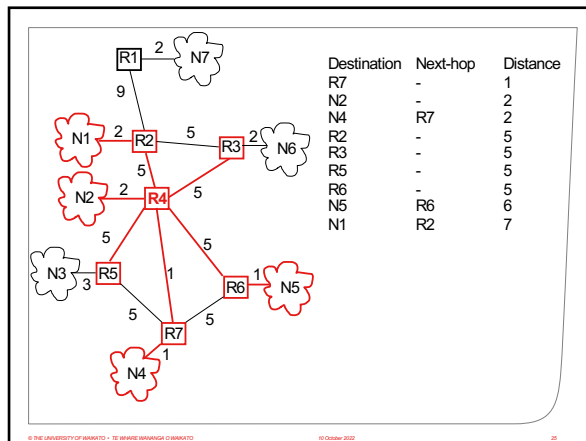
22



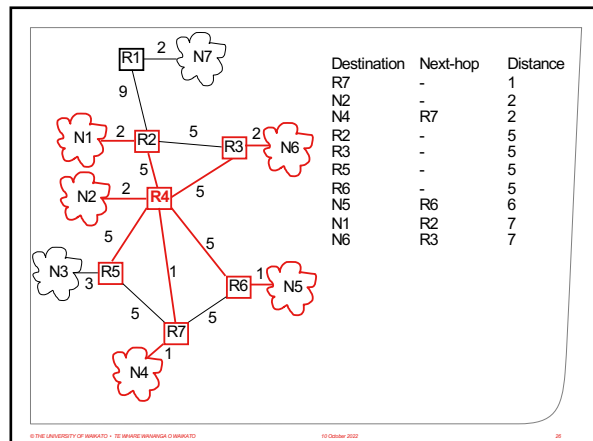
23



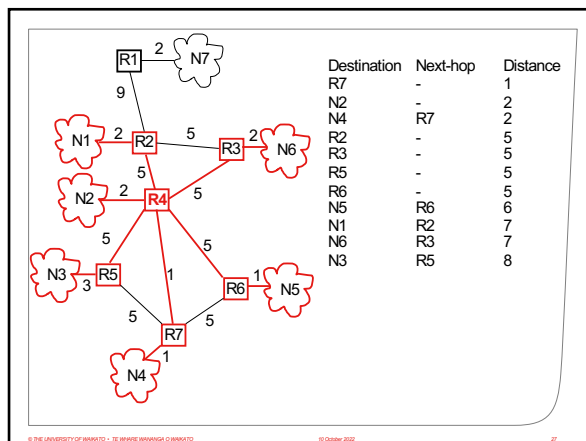
24



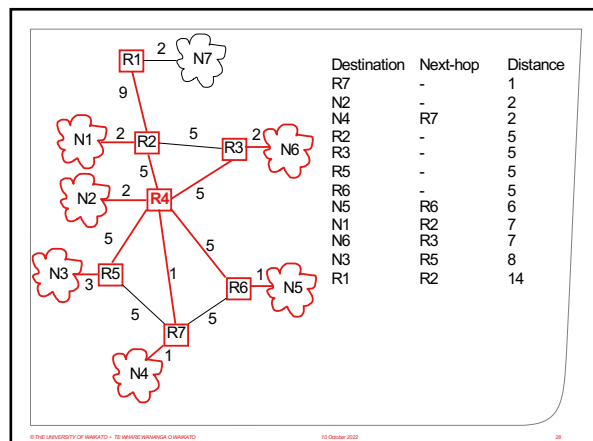
25



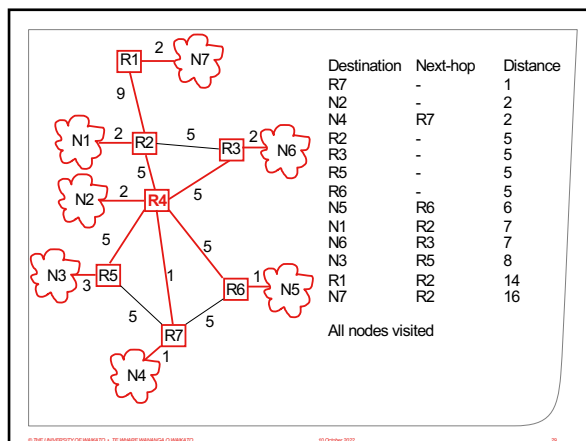
26



27



28

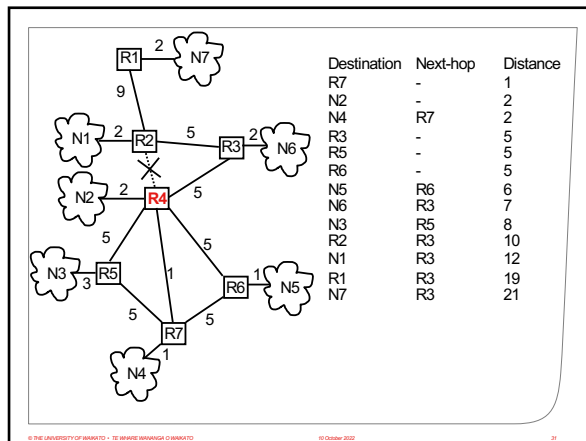


29

### OSPF

- All nodes perform this calculation themselves
- If a link goes away, or changes costs, then the graph is updated and the algorithm has to be run from scratch
- Alternative approach: distance-vector (as with RIP)

30

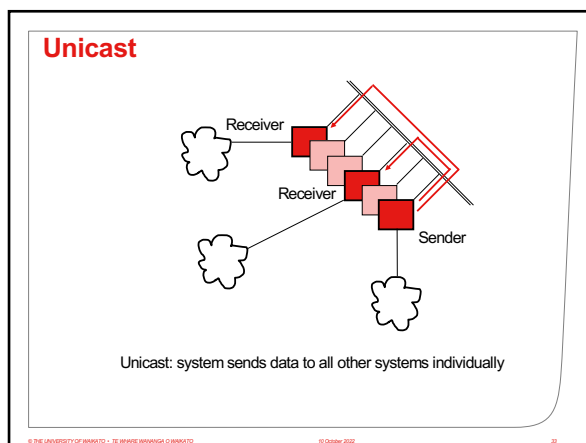


31

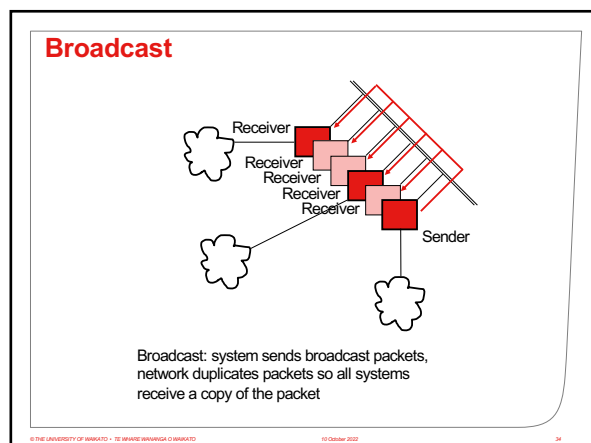
### Multicast

- In OSPF, each router “floods” available routes to all other routers in the network.
- There are multiple ways this could be accomplished:
  - Unicast: router sends data to all other routers individually
  - Broadcast: router sends broadcast packets, network duplicates packets so all systems receive a copy of the packet
  - Multicast: all interested systems join a group; router sends packet once, network duplicates packets to senders as appropriate.
- OSPF uses multicast.

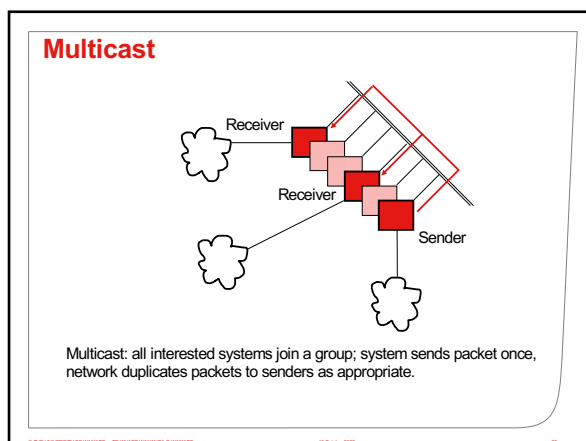
32



33



34



35

### Multicast

- Groups are specified using addresses
- IPv4 multicast addresses are in range
  - 224.0.0.0 to 239.255.255.255
  - i.e., the equivalent of 16 class A address blocks, 1/16<sup>th</sup> of the available address space
- OSPF uses address 224.0.0.5 : link scoped
- 239.0.0.8 is for private use: i.e., a single organisation can use addresses in this block however they want to, but it does not get forwarded to other networks.
  - Addresses in this range are used for things like Universal Plug and Play (UPnP) networking

36

## Multicast

- Can't multicast TCP packets. TCP assumes a single sender / receiver.
- Typically used with UDP packets.
  - Java uses DatagramPacket to construct UDP packets.
  - Java's MulticastSocket extends DatagramSocket, i.e., is a special type of UDP socket.

© THE UNIVERSITY OF MANITOBA - 10 MANITOBA ACADEMIC CREDIT

10 October 2022

37

37

## Java

```
• InetAddress group = InetAddress.getByName(<addr>);
• MulticastSocket ms = new MulticastSocket(<port>);
• byte[] buf = new byte[1472];
• DatagramPacket dp = new DatagramPacket(buf,
 buf.length);

• ms.receive(dp);
• dp.getData(), dp.getLength();
• DatagramPacket pkt = new
 DatagramPacket(<data>, <length>, <addr>, <port>);
• ms.send(pkt);
```

© THE UNIVERSITY OF MANITOBA - 10 MANITOBA ACADEMIC CREDIT

10 October 2022

38

38

## Learning Objectives

- Understand the limits of manual configuration of networks and why automatic routing protocols are useful
- Understand how Dijkstra's algorithm works given a sample network topology
- Understand the factors that influence the "Distance" metric

© THE UNIVERSITY OF MANITOBA - 10 MANITOBA ACADEMIC CREDIT

10 October 2022

39

39

## Summary

- Dynamic routing protocols are useful
  - Less tedious than manually reconfiguring all routers when links change (are added, removed, or otherwise change distance)
  - Less error prone
  - Better at responding to failures
- OSPF is one example of an internal routing protocol
  - Link-state
  - Uses Dijkstra's algorithm
- Multicast: send data once to all interested receivers
  - OSPF uses multicast to send routing information to all routers it is connected to.

© THE UNIVERSITY OF MANITOBA - 10 MANITOBA ACADEMIC CREDIT

10 October 2022

40

40

# COMPX204-22B Practical Networking and Cybersecurity

Lecture 19  
Address Exhaustion, RFC 1918, Network  
Address Translation

Matthew Luckie  
mluckie@waikato.ac.nz



13 October 2022

1

## Overview

- IPv4
- Address exhaustion
- IPv6
- Application-Level Gateways (ALGs)
- Network Address Translation (NAT)

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

13 October 2022

2

## Learning Objectives

- Understand the key differences between IPv4 and IPv6 from a protocol perspective
- Understand what is meant by class-based addressing, and how it contributed to address run-out
- Understand how Network Address Translation works from a protocol perspective

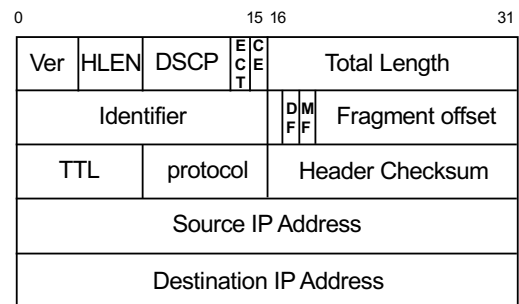
© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

13 October 2022

3

3

## IPv4 Header



© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

13 October 2022

4

4

## IPv4 Address Exhaustion

- A 32 bit address allows for around **4 billion** hosts, ignoring network and broadcast addresses
  - $2^{32} = 4,294,967,296$
- This might seem a lot, until you think about giving an IPv4 address to all cell phones in the world
  - This scenario was not envisaged when IPv4 was defined
- Ignoring this, class-based assignment resulted in inefficient allocation of IPv4 address space
  - Class A: /8 - 1.0.0.0 – 127.255.255.255 (first bit: 0)
  - Class B: /16 - 128.0.0.0 – 191.255.255.255 (first bits: 10)
  - Class C: /24 - 192.0.0.0 – 223.255.255.255 (first bits: 110)

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

13 October 2022

5

5

## Two intermediate solutions

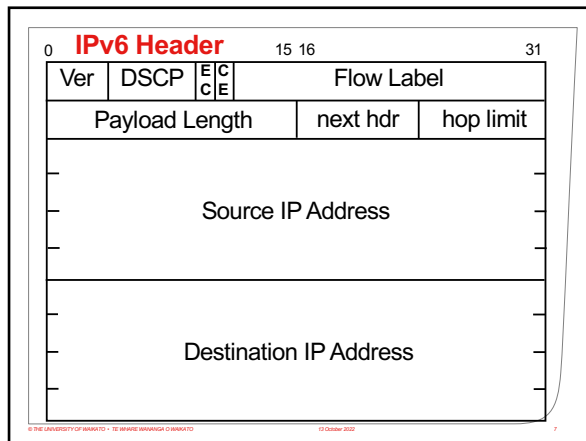
- Classless Inter-domain Routing (CIDR)
  - RFC 1519, September 1993
- RFC 1918 private addresses
  - originally RFC 1594, March 1994
  - Requires application level gateways, or
  - Network address translation (RFC 1631, May 1994)
- Both bought the Internet time before addresses ran out
- Long term solution
  - IP version 6

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

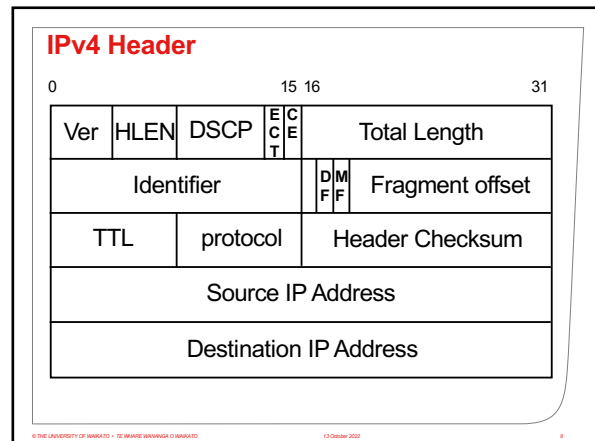
13 October 2022

6

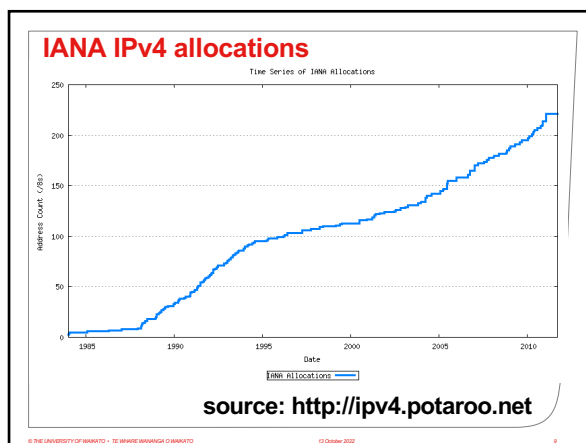
6



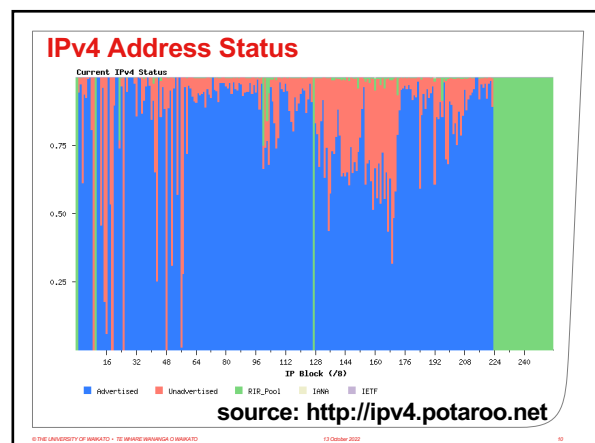
7



8



9



10

### Classless Interdomain Routing (CIDR)

- Up until 1993, allocations were done in classes
  - Class A: /8
  - Class B: /16
  - Class C: /24
- Changes to allocation policy and routing protocols allowed allocations to be made based on actual space required.
  - E.g. allocating 202.53.176.0/20 to FX networks, 4096 addresses
    - Instead of allocating one class B network, for 6% usage
    - Instead of allocating 16 class C networks

11

### RFC 1918 private addresses: motivation

With the proliferation of TCP/IP technology worldwide, including outside the Internet itself, an increasing number of non-connected enterprises use this technology and its addressing capabilities for sole intra-enterprise communications, without any intention to ever directly connect to other enterprises or the Internet itself.

12

### RFC 1918 private address space

- 10.0.0.0/8
  - Class A network
- 172.16.0.0/12
  - Set of 16 contiguous class B networks
- 192.168.0.0/16
  - Set of 256 contiguous class C networks
- Private addresses are, by definition, available for anyone to use for their own networks
- They do not uniquely identify any host in the Internet
- They are not routed on the Internet

13

### Intermission

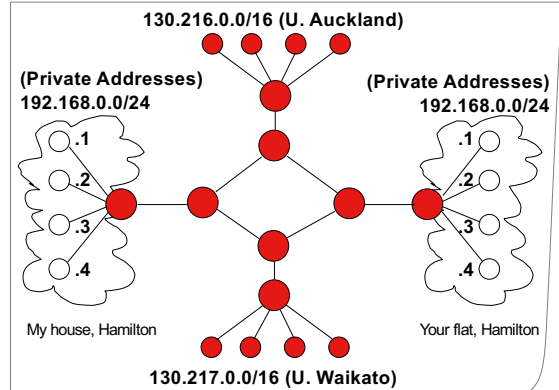
- BGP, The Movie.

14

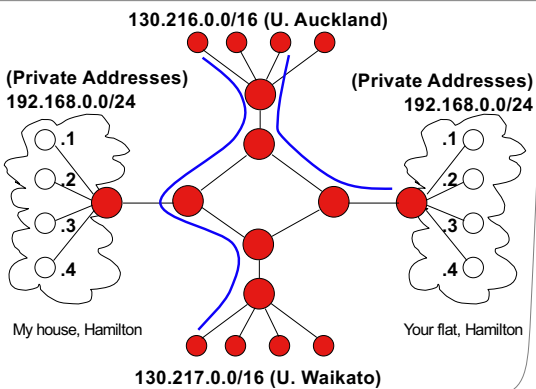
### Internet End-to-End Connectivity

- Ideally, all hosts in the Internet would be able to uniquely identified by their address
  - Allows for simple inter-connectivity of hosts
- The use of RFC 1918 addresses breaks this model

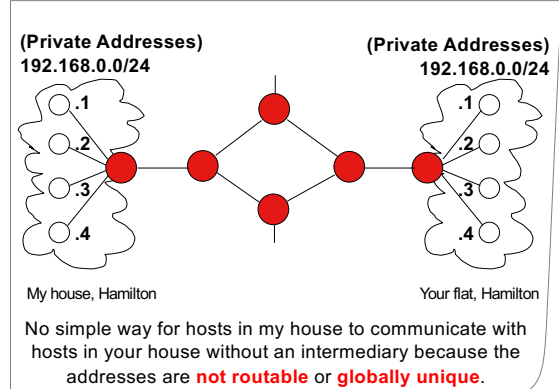
15



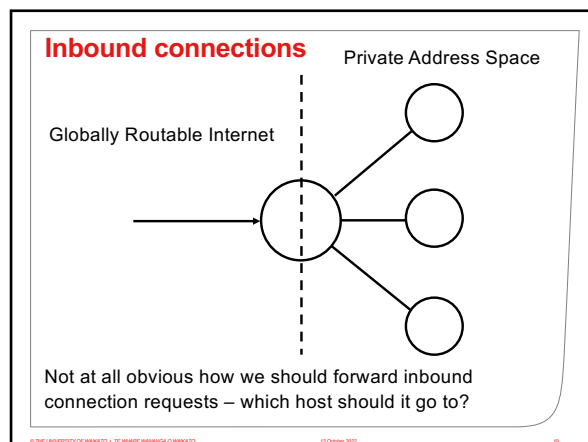
16



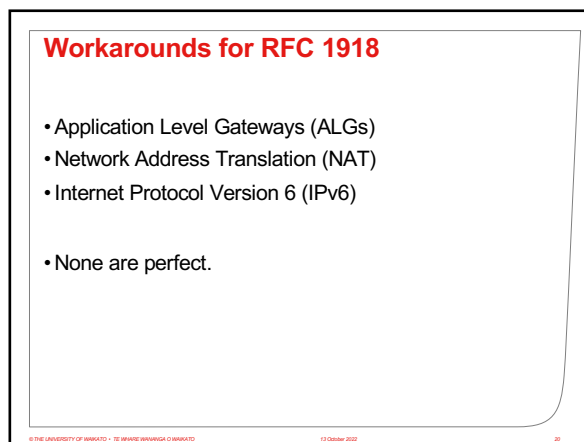
17



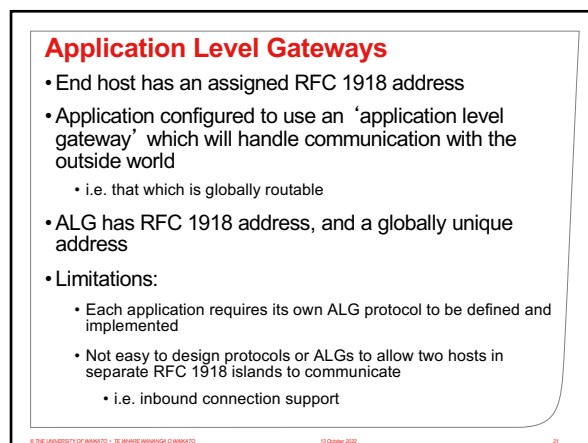
18



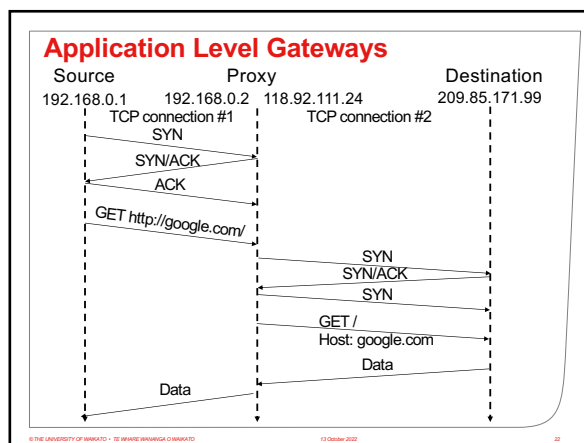
19



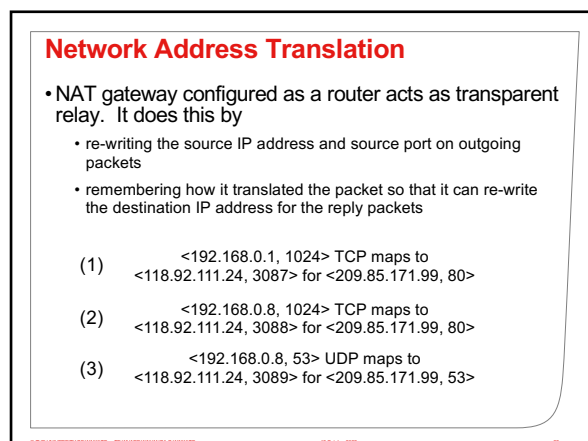
20



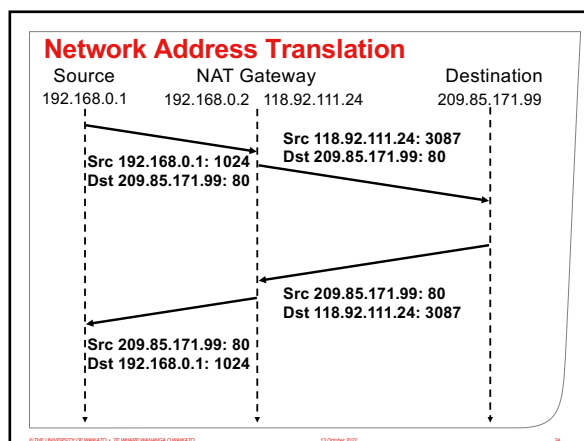
21



22



23



24

## Network Address Translation

- **Advantages:**
  - Fairly simple to implement
  - Usable with UDP, TCP, and ICMP packets
  - Does not require separate implementation for each protocol like an ALG does
- **Disadvantages:**
  - Still does not allow end-to-end connectivity
    - Breaks peer-to-peer applications
    - Does not allow in-bound connections
  - Single point of failure: if NAT gateway breaks then all connections go with it, as it has state associated with each connection

25

## Service Provider NAT

- In New Zealand, we are **usually** assigned a single globally routable IPv4 address whenever we use DSL or fiber connections at home
  - This is not true in all countries
  - i.e., nearly all customers in some countries are assigned an RFC 1918 address
- Soon, ISPs will run out of addresses
- The correct solution is to move to IPv6 which has  $2^{128}$  addresses
  - Most ISPs in NZ will delegate an IPv6 prefix to a customer home router if the home router supports IPv6
- Service Provider NAT (SPNAT) might be required

26

## Summary

- It became obvious fairly on in the early 1990s that the Internet was going to run out of addresses
  - CIDR, Private addresses bought us time
  - NAT is the price we paid
- IPv6 is intended solution
  - Increasing deployment, has been defined for nearly 25 years (1998)
  - Service provider NAT not a good solution.
  - Deployment issues mostly at the edge of the network: enterprise and access networks.
  - Most large transit providers have deployed IPv6

27

## Learning Objectives

- Understand the key differences between IPv4 and IPv6 from a protocol perspective
- Understand what is meant by class-based addressing, and how it contributed to address run-out
- Understand how Network Address Translation works from a protocol perspective

28

# COMPX204-22B Practical Networking and Cybersecurity

## Lecture 20 Elementary Internet Path Debugging

Matthew Luckie  
mluckie@waikato.ac.nz



18 October 2022

1

## This lecture

- Two very basic (but routinely used) tools for debugging problems
  - ping
  - Traceroute (or tracert, on windows)

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

18 October 2022

2

2

## Learning Objectives

- Understand how ping can be used to measure delay and packet loss, and the effect that larger packets have on delay
- Understand how traceroute can be used to give visibility into the router path between two points in the network
- Understand how these useful troubleshooting tools use the Internet Control Message Protocol (ICMP)

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

18 October 2022

3

3

## Ping

- Program to test basic properties of an Internet path
  - Is a host up and working?
  - What is the delay (round-trip-time) between the pair of hosts?
  - How often are packets lost?
  - How variable is the delay on the path?
- These may sound like simple questions, but the latter three are a useful predictor of TCP performance

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

18 October 2022

4

4

## Ping demo

```
$ ping sorcerer.cms.waikato.ac.nz
PING sorcerer.cms.waikato.ac.nz (130.217.250.39): 56 data bytes
64 bytes from 130.217.250.39: icmp_seq=0 ttl=57 time=10.002 ms
64 bytes from 130.217.250.39: icmp_seq=1 ttl=57 time=10.070 ms
64 bytes from 130.217.250.39: icmp_seq=2 ttl=57 time=10.539 ms
64 bytes from 130.217.250.39: icmp_seq=3 ttl=57 time=10.144 ms
64 bytes from 130.217.250.39: icmp_seq=4 ttl=57 time=10.090 ms
64 bytes from 130.217.250.39: icmp_seq=5 ttl=57 time=9.995 ms
64 bytes from 130.217.250.39: icmp_seq=6 ttl=57 time=10.056 ms
64 bytes from 130.217.250.39: icmp_seq=7 ttl=57 time=10.116 ms
64 bytes from 130.217.250.39: icmp_seq=8 ttl=57 time=9.847 ms
64 bytes from 130.217.250.39: icmp_seq=9 ttl=57 time=9.815 ms
64 bytes from 130.217.250.39: icmp_seq=10 ttl=57 time=10.050 ms
64 bytes from 130.217.250.39: icmp_seq=11 ttl=57 time=10.076 ms
64 bytes from 130.217.250.39: icmp_seq=12 ttl=57 time=9.895 ms
64 bytes from 130.217.250.39: icmp_seq=13 ttl=57 time=9.851 ms
^C
--- sorcerer.cms.waikato.ac.nz ping statistics ---
14 packets transmitted, 14 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 9.815/10.039/10.539/0.173 ms
```

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

18 October 2022

5

5

## Ping summary

```
--- sorcerer.cms.waikato.ac.nz ping statistics ---
14 packets transmitted, 14 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 9.815/10.039/10.539/0.173 ms
```

- Basic summary statistics, displaying
  - How many packets were transmitted / received
  - Was there any loss?
  - Minimum delay observed between sending a packet and getting a response
  - Average (mean) delay
  - Maximum delay observed
  - Standard deviation of delays
- Could use this information to get some idea of the TCP retransmit timeout value

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

18 October 2022

6

6

## Experiments with ping: larger packets

```
$ ping -s 1464 sorcerer.cms.waikato.ac.nz
PING sorcerer.cms.waikato.ac.nz (130.217.250.39): 1464 data bytes
1472 bytes from 130.217.250.39: icmp_seq=0 ttl=57 time=10.650 ms
1472 bytes from 130.217.250.39: icmp_seq=1 ttl=57 time=11.159 ms
1472 bytes from 130.217.250.39: icmp_seq=2 ttl=57 time=10.744 ms
1472 bytes from 130.217.250.39: icmp_seq=3 ttl=57 time=10.792 ms
1472 bytes from 130.217.250.39: icmp_seq=4 ttl=57 time=10.988 ms
1472 bytes from 130.217.250.39: icmp_seq=5 ttl=57 time=10.714 ms
1472 bytes from 130.217.250.39: icmp_seq=6 ttl=57 time=11.017 ms
1472 bytes from 130.217.250.39: icmp_seq=7 ttl=57 time=10.980 ms
1472 bytes from 130.217.250.39: icmp_seq=8 ttl=57 time=10.734 ms
1472 bytes from 130.217.250.39: icmp_seq=9 ttl=57 time=10.511 ms
1472 bytes from 130.217.250.39: icmp_seq=10 ttl=57 time=10.745 ms
1472 bytes from 130.217.250.39: icmp_seq=11 ttl=57 time=11.042 ms
1472 bytes from 130.217.250.39: icmp_seq=12 ttl=57 time=10.788 ms
1472 bytes from 130.217.250.39: icmp_seq=13 ttl=57 time=11.072 ms
^C
--- sorcerer.cms.waikato.ac.nz ping statistics ---
14 packets transmitted, 14 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 10.511/10.853/11.159/0.182 ms
```

© THE UNIVERSITY OF WAIKATO - TE WHARE WANGANGI O WAIKATO

10 October 2022

7

## Experiments with ping: larger packets

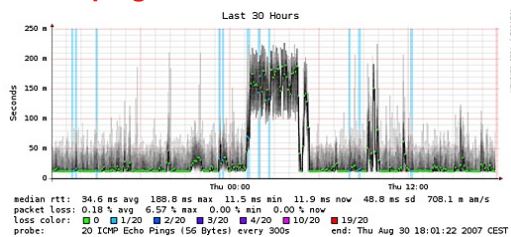
- Small (64 + 20 bytes) packets
  - 9.815/10.039/10.539/0.173 ms
- Large (1472 + 20 bytes) packets
  - 10.511/10.853/11.159/0.182 ms
- Larger packets take longer to be forwarded through the network
  - Though they are more efficient use of the network
  - Your computer at home probably sends and receives ~1500 byte packets
  - 1500: Historical maximum size of Ethernet payload

© THE UNIVERSITY OF WAIKATO - TE WHARE WANGANGI O WAIKATO

10 October 2022

8

## Smokeying



- Smokeping sends 10 ping requests in a row
  - Median is plotted
  - Gray points are the other measured delays
- <https://oss.oetiker.ch/smokeying/>

© THE UNIVERSITY OF WAIKATO - TE WHARE WANGANGI O WAIKATO

10 October 2022

9

## ICMP

- Ping sends an ICMP Echo Request packet into the network, and expects an ICMP Echo Reply packet.
- Internet Control Message Protocol
- Used for
  - Encapsulating error messages
  - Debugging and information

| 0    | 15   | 16       | 31 |
|------|------|----------|----|
| Type | Code | Checksum |    |
| ID   |      | Sequence |    |

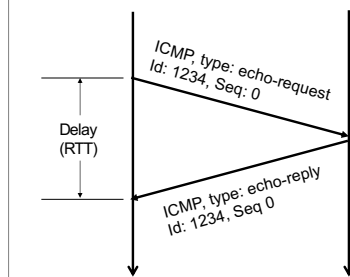
© THE UNIVERSITY OF WAIKATO - TE WHARE WANGANGI O WAIKATO

10 October 2022

10

## ICMP and Ping

- Ping sends an ICMP echo request message (type 8)
- Ping expects an ICMP echo reply message (type 0)



© THE UNIVERSITY OF WAIKATO - TE WHARE WANGANGI O WAIKATO

10 October 2022

11

## Traceroute

- Program to infer the sequence of routers on the forward path
  - Actually, a sequence of network interfaces visited
- Uses ICMP as well!
- Uses DNS entries to tell you the names of the interfaces visited, if they have been stored in the domain name system.
- Tells you the delay measured, though the value of that measurement is somewhat limited (but still useful)

© THE UNIVERSITY OF WAIKATO - TE WHARE WANGANGI O WAIKATO

10 October 2022

12

## Traceroute

```
$ traceroute -q 1 www.google.co.nz
traceroute to www.google.co.nz (216.58.199.67), 64 hops max, 40 byte packets
 1 130.217.248.254 (130.217.248.254) 0.490 ms
 2 gg-fortigate-1.network.waikato.ac.nz (130.217.64.2) 0.316 ms
 3 130.217.2.4 (130.217.2.4) 2.878 ms
 4 210.7.39.9 (210.7.39.9) 0.465 ms
 5 210.7.39.251 (210.7.39.251) 2.979 ms
 6 210.7.39.250 (210.7.39.250) 3.053 ms
 7 et-1-2-0-201.pe1.tkpa.akl.aarnet.net.au (182.255.119.138) 3.164 ms
 8 et-0-1-0.200.pe1.wnppa.akl.aarnet.net.au (113.197.15.68) 4.065 ms
 9 xe-1-2-1.pe1.msct.nsw.aarnet.net.au (113.197.15.66) 27.880 ms
10 et-3-3-0.pe1.rsby.nsw.aarnet.net.au (113.197.15.156) 26.516 ms
11 et-0-3-0.bdr1.msct.nsw.aarnet.net.au (113.197.15.111) 26.411 ms
12 gw1.xe-0-2-1.bdr1.msct.nsw.aarnet.net.au (202.158.205.150) 26.506 ms
13 74.125.37.167 (74.125.37.167) 26.504 ms
14 209.85.248.173 (209.85.248.173) 26.979 ms
15 syd15s01-in-f3.1e100.net (216.58.199.67) 26.885 ms
```

Takapuna (tkpa), Whenuapai (wnppa), New South Wales (NSW), Sydney

13

## Traceroute

```
$ traceroute to cider.caida.org (192.172.226.123), 64 hops max, 40 byte packets
 1 248-gw (130.217.248.251) 0.347 ms
 2 10.5.5.1 (10.5.5.1) 0.307 ms
 3 130.217.2.4 (130.217.2.4) 0.414 ms
 4 210.7.39.9 (210.7.39.9) 0.492 ms
 5 et-1-2-0-202.pe1.tkpa.akl.aarnet.net.au (182.255.119.200) 3.264 ms
 6 et-0-1-0.199.pe1.wnppa.akl.aarnet.net.au (113.197.15.70) 3.965 ms
 7 et-2-0-0.pe1.msct.nsw.aarnet.net.au (113.197.15.76) 26.599 ms
 8 et-2-3-0.pe1.sxt.bkvl.nsw.aarnet.net.au (113.197.15.78) 27.042 ms
 9 et-0-0-0.pe1.a.hnl.aarnet.net.au (113.197.15.99) 119.878 ms
10 et-2-1-0.bdr1.a.sea.aarnet.net.au (113.197.15.201) 171.183 ms
11 cenichpr-1-is-jmb-778.snvaca.pacificwave.net (207.231.245.129) 161.727 ms
12 hpr-lax-hpr3-svl-hpr3-100ge.cenic.net (137.164.25.73) 161.951 ms
13 hpr-sdsc-40ge-lax-hpr.cenic.net (137.164.26.34) 164.319 ms
14 thor-ae0-mx0-ae7.sdsc.edu (192.12.207.62) 164.129 ms
15 cider.caida.org (192.172.226.123) 164.118 ms
```

Note: ~90ms jump in delay here. Why?  
And ~40ms jump in delay in subsequent hop. Why?

14

## IP Header

|                        |          |                 |                 |
|------------------------|----------|-----------------|-----------------|
| 0                      | 15       | 16              | 31              |
| Ver.                   | hlen     | TOS             | Total Length    |
| Identification         |          | flags           | Fragment offset |
| TTL                    | protocol | Header Checksum |                 |
| Source IP Address      |          |                 |                 |
| Destination IP Address |          |                 |                 |

15

## IP Header: needed for 204

|                        |  |          |  |                 |  |              |  |
|------------------------|--|----------|--|-----------------|--|--------------|--|
| 0                      |  | 15       |  | 16              |  | 31           |  |
|                        |  |          |  |                 |  | Total Length |  |
|                        |  |          |  |                 |  |              |  |
| TTL                    |  | protocol |  | Header Checksum |  |              |  |
| Source IP Address      |  |          |  |                 |  |              |  |
| Destination IP Address |  |          |  |                 |  |              |  |

16

## IP Header: TTL

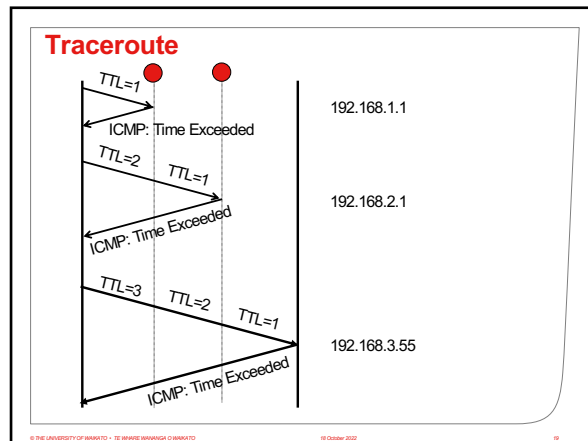
- TTL: time to live
  - Used to prevent an IP packet from getting into a loop and circling forever.
  - Sender chooses a value when sending IP packet
  - Each router decrements the value by 1
  - When the value reaches 1, a router should send an ICMP Time Exceeded message back to the source, saying the packet was discarded
- This is used by traceroute

17

## Traceroute

- Iterative process
- Host sends a series of IP packets into the network, each with an increasing TTL value
- When a packet's TTL reaches one, the router which has the packet returns it to the sender, saying "time exceeded".

18



19

### Traceroute

- Multiple traceroute methods exist
  - ICMP echo (traceroute -I)
  - UDP (traceroute)
  - TCP (tcptraceroute)
- One method may work where others are blocked
  - E.g. web servers are generally responsive to TCP traceroute

20

### Traceroute

- Useful for checking sanity of route
  - Though you can't actually do anything to fix any perceived problem yourself
- Useful for understanding the structure of the Internet
  - Arrangement of nodes (routers) and links
- Major limitation
  - Does not show you the path taken from the destination back to you
  - i.e. traceroute is only telling you half the story
  - Internet routes are often asymmetrical

21

### Summary

- Today's lecture was on elementary network debugging
  - Ping basic statistics on a path to a destination
  - Traceroute arrangement of routers on forward path
- Both use ICMP
- Try using ping and traceroute in the lab

22

### Learning Objectives

- Understand how ping can be used to measure delay and packet loss, and the effect that larger packets have on delay
- Understand how traceroute can be used to give visibility into the router path between two points in the network
- Understand how these useful troubleshooting tools use the Internet Control Message Protocol (ICMP)

23

# COMPX204-22B Practical Networking and Cybersecurity

Lecture 21  
Course Wrap

Matthew Luckie  
mluckie@waikato.ac.nz



20 October 2022

1

## Overview

This paper provides an overview of the technologies, protocols and security issues involved in computer communications.

It will introduce programming issues in communications, including error handling and concurrency.

Practical work will look at the Internet Protocols TCP and IP, and include programming exercises using socket interfaces.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 October 2022

2

2

## Goals

1. Students will be able to describe aspects of the Internet including TCP/IP, DNS, addressing, routing.
2. Students will be able to write small communications systems in Java including parallel programming to implement servers that support multiple simultaneous clients.
3. Students will be able to use Transport Layer Security (TLS) to implement authentication and encryption in software.
4. Students will be able to write unit tests that check for correct software behaviour with untrusted network input.

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 October 2022

3

3

## Contents

- Intro to Linux, Java
- Socket programming, concurrent programming (IO)
- HTTP and Web Servers
- Ethernet
- IPv4, IPv6
- TCP, UDP
- Routing (BGP + OSPF), Multicast, NAT
- Domain names and DNS
- Security
- Network Debugging

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 October 2022

4

4

## Tentative Schedule

|      |         |              |                   |                            |
|------|---------|--------------|-------------------|----------------------------|
| July | 18 – 22 | Intro + Addr | Sockets           |                            |
|      | 25 – 29 | Threads      | Sockets + Threads |                            |
| Aug  | 1 – 5   | Ethernet #1  | Ethernet #2       |                            |
|      | 8 – 12  | DNS / UDP    | Unit Testing      | A1: 12 <sup>th</sup> Intro |
|      | 15 – 19 | TCP #1       | TCP #2            |                            |
|      | 22 – 26 | TCP #3       | Revision          | A2: 26 <sup>th</sup> HTTP  |
| Sep  | 12 – 16 | Test #1      | Kingitanga        |                            |
|      | 19 – 23 | Security #1  | Security #2       |                            |
|      | 26 – 30 | Security #3  | IP Routing        | A3: 30 <sup>th</sup> TFTP  |
| Oct  | 3 – 7   | OSPF / Mcast | BGP               |                            |
|      | 10 – 14 | NAT          | Autoconf          |                            |
|      | 17 – 21 | Debugging    | Course Wrap       | A4: 21 <sup>st</sup> TLS   |

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

20 October 2022

5

5

## Actual Schedule

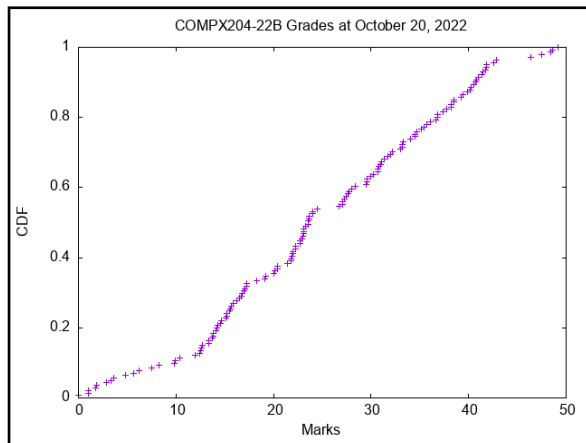
|      |         |              |                   |                            |
|------|---------|--------------|-------------------|----------------------------|
| July | 18 – 22 | Intro + Addr | Sockets           |                            |
|      | 25 – 29 | Threads      | Sockets + Threads |                            |
| Aug  | 1 – 5   | Ethernet #1  | Ethernet #2       |                            |
|      | 8 – 12  | DNS / UDP    | Unit Testing      | A1: 12 <sup>th</sup> Intro |
|      | 15 – 19 | TCP #1       | TCP #2            |                            |
|      | 22 – 26 | TCP #3       | Revision          | A2: 26 <sup>th</sup> HTTP  |
| Sep  | 12 – 16 | Test #1      | Kingitanga        |                            |
|      | 19 – 23 | Security #1  | Security #2       |                            |
|      | 26 – 30 | Security #3  | Security #4       |                            |
| Oct  | 3 – 7   | IP Routing   | OSPF / Mcast      |                            |
|      | 10 – 14 | BGP          | NAT               | A3: 30 <sup>th</sup> TFTP  |
|      | 17 – 21 | Debugging    | Course Wrap       | A4: 21 <sup>st</sup> TLS   |

© THE UNIVERSITY OF WAIKATO - TE WHARE WAIKATO O WAIKATO

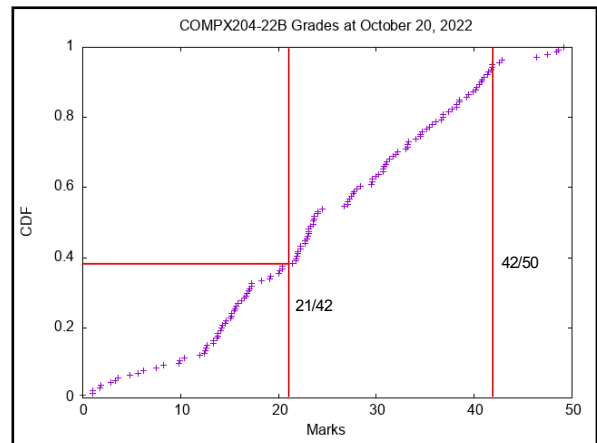
20 October 2022

6

6



7



8

**Tips for the exam**

- Go through previous tests and practice those.
- Go through exams and practice those.

© The University of Queensland - St. Andrew's School of Engineering 20 October 2022 8

9